Telematics Platform User Guide

NADIF for 5G NAD

Document Information	
Filename	TP_F101U-00036_User_Guide_NADIF_5G_NAD
Status	Release
Revision Date	Apr 04, 2022
Version	14.00
Document ID	TP_F101U-00036

Standard Proprietary Terms
THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

Table of contents

1 Introduction ... 9
1.1 Notational Conventions ... 9
2 Setup ... 10
2.1 Tools/Equipment Needed ... 10
2.2 Setup Instructions ... 10
2.3 Setup Troubleshooting ... 11
3 Features/Components ... 11
3.1 Client Registration ... 11
3.1.1 Overview ... 11
3.1.1.1 Platform Content Overview ... 11
3.1.1.2 Product Expectations ... 11
3.1.1.3 Extending Platform Functionality ... 11
3.1.2 Setup ... 11
3.1.2.1 Tools/Equipment Needed ... 11
3.1.2.2 Setup Instructions ... 11
3.1.2.3 Setup Troubleshooting ... 11
3.1.3 Usage ... 11
3.1.3.1 Basic Instructions ... 11
3.1.3.2 Detailed Instructions ... 12
3.1.3.3 Testing/Error Reporting ... 20
3.2 Multiple SIM Subscription Support ... 20
3.2.1 Overview ... 20
3.2.1.1 Platform Content Overview ... 20
3.2.1.2 Product Expectations ... 20
3.2.1.3 Extending Platform Functionality ... 20
3.2.2 Setup ... 20
3.2.2.1 Tools/Equipment Needed ... 20
3.2.2.2 Setup Instructions ... 21
3.2.2.3 Setup Troubleshooting ... 21
3.2.3 Usage ... 21
3.2.3.1 Basic Instructions ... 21
3.2.3.2 Detailed Instructions ... 21
3.2.3.3 Testing/Error Reporting ... 24
3.3 General Request/Response ... 24
3.3.1 Overview ... 24
3.3.1.1 Platform Content Overview ... 24
3.3.1.2 Product Expectations ... 25
3.3.1.3 Extending Platform Functionality ... 25
3.3.2 Setup ... 25

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.3.2.1 Tools/Equipment Needed 25
3.3.2.2 Setup Instructions 25
3.3.2.3 Setup Troubleshooting 25
3.3.3 Usage 25
3.3.3.1 Basic Instructions 25
3.3.3.2 Detailed Instructions 26
3.3.3.3 Testing/Error Reporting 29
3.4 Unsolicited Notification 29
3.4.1 Overview 29
3.4.1.1 Platform Content Overview 29
3.4.1.2 Product Expectations 29
3.4.1.3 Extending Platform Functionality 29
3.4.2 Setup 30
3.4.2.1 Tools/Equipment Needed 30
3.4.2.2 Setup Instructions 30
3.4.2.3 Setup Troubleshooting 30
3.4.3 Usage 30
3.4.3.1 Basic Instructions 30
3.4.3.2 Detailed Instructions 30
3.4.3.3 Testing/Error Reporting 33
3.5 Debug Level 33
3.5.1 Overview 33
3.5.1.1 Platform Content Overview 33
3.5.1.2 Product Expectations 33
3.5.1.3 Extending Platform Functionality 33
3.5.2 Setup 33
3.5.2.1 Tools/Equipment Needed 33
3.5.2.2 Setup Instructions 33
3.5.2.3 Setup Troubleshooting 33
3.5.3 Usage 33
3.5.3.1 Basic Instructions 33
3.5.3.2 Detailed Instructions 34
3.5.3.3 Testing/Error Reporting 34
3.6 DTMF 34
3.6.1 Overview 34
3.6.1.1 Platform Content Overview 34
3.6.1.2 Product Expectations 35
3.6.1.3 Extending Platform Functionality 35
3.6.2 Setup 35
3.6.2.1 Tools/Equipment Needed 35
3.6.2.2 Setup Instructions 35
3.6.2.3 Setup Troubleshooting 35
3.6.3 Usage 35

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.6.3.1 Basic Instructions 35
3.6.3.2 Detailed Instructions 35
3.6.3.3 Testing/Error Reporting 38
3.7 Network 39
3.7.1 Overview 39
3.7.1.1 Platform Content Overview 39
3.7.1.2 Product Expectations 39
3.7.1.3 Extending Platform Functionality 40
3.7.2 Setup 40
3.7.2.1 Tools/Equipment Needed 40
3.7.2.2 Setup Instructions 40
3.7.2.3 Setup Troubleshooting 40
3.7.3 Usage 40
3.7.3.1 Basic Instructions 40
3.7.3.2 Detailed Instructions 41
3.7.3.3 Testing/Error Reporting 47
3.8 On-Device Logging 47
3.8.1 Overview 47
3.8.1.1 Platform Content Overview 47
3.8.1.2 Product Expectations 47
3.8.1.3 Extending Platform Functionality 48
3.8.2 Setup 48
3.8.2.1 Tools/Equipment Needed 48
3.8.2.2 Setup Instructions 48
3.8.2.3 Setup Troubleshooting 48
3.8.3 Usage 48
3.8.3.1 Basic Instructions 48
3.8.3.2 Detailed Instructions 48
3.8.3.3 Testing/Error Reporting 50
3.9 SIM 50
3.9.1 Overview 50
3.9.1.1 Platform Content Overview 50
3.9.1.2 Product Expectations 50
3.9.1.3 Extending Platform Functionality 52
3.9.2 Setup 52
3.9.2.1 Tools/Equipment Needed 52
3.9.2.2 Setup Instructions 52
3.9.2.3 Setup Troubleshooting 52
3.9.3 Usage 52
3.9.3.1 Basic Instructions 52
3.9.3.2 Detailed Instructions 53
3.9.3.3 Testing/Error Reporting 63
3.10 SMS 63

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.10.1 Overview 63
3.10.1.1 Platform Content Overview 63
3.10.1.2 Product Expectations 63
3.10.1.3 Extending Platform Functionality 64
3.10.2 Setup 64
3.10.2.1 Tools/Equipment Needed 64
3.10.2.2 Setup Instructions 64
3.10.2.3 Setup Troubleshooting 64
3.10.3 Usage 64
3.10.3.1 Basic Instructions 64
3.10.3.2 Detailed Instructions 65
3.10.3.3 Testing/Error Reporting 69
3.11 Voice Call 70
3.11.1 Overview 70
3.11.1.1 Platform Content Overview 70
3.11.1.2 Product Expectations 70
3.11.1.3 Extending Platform Functionality 71
3.11.2 Setup 71
3.11.2.1 Tools/Equipment Needed 71
3.11.2.2 Setup Instructions 71
3.11.2.3 Setup Troubleshooting 71
3.11.3 Usage 72
3.11.3.1 Basic Instructions 72
3.11.3.2 Detailed Instructions 72
3.11.3.3 Testing/Error Reporting 81
3.12 Modem Status 81
3.12.1 Overview 81
3.12.1.1 Platform Content Overview 81
3.12.1.2 Product Expectations 81
3.12.1.3 Extending Platform Functionality 83
3.12.2 Setup 83
3.12.2.1 Tools/Equipment Needed 83
3.12.2.2 Setup Instructions 83
3.12.2.3 Setup Troubleshooting 83
3.12.3 Usage 83
3.12.3.1 Basic Instructions 83
3.12.3.2 Detailed Instructions 83
3.12.3.3 Testing/Error Reporting 94
3.13 Standard eCall 94
3.13.1 Overview 94
3.13.1.1 Platform Content Overview 94
3.13.1.2 Product Expectations 95
3.13.1.3 Extending Platform Functionality 96

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.13.2 Setup 97
3.13.2.1 Tools/Equipment Needed 97
3.13.2.2 Setup Instructions 97
3.13.2.3 Setup Troubleshooting 97
3.13.3 Usage 97
3.13.3.1 Basic Instructions 97
3.13.3.2 Detailed Instructions 97
3.13.3.3 Testing/Error Reporting 101
3.14 Private eCall 101
3.14.1 Overview 101
3.14.1.1 Platform Content Overview 101
3.14.1.2 Product Expectations 101
3.14.1.3 Extending Platform Functionality 102
3.14.2 Setup 102
3.14.2.1 Tools/Equipment Needed 102
3.14.2.2 Setup Instructions 102
3.14.2.3 Setup Troubleshooting 102
3.14.3 Usage 102
3.14.3.1 Basic Instructions 102
3.14.3.2 Detailed Instructions 102
3.14.3.3 Testing/Error Reporting 103
3.15 Phone Book Manager 103
3.15.1 Overview 103
3.15.1.1 Platform Content Overview 103
3.15.1.2 Product Expectations 103
3.15.1.3 Extending Platform Functionality 104
3.15.2 Setup 104
3.15.2.1 Tools/Equipment Needed 104
3.15.2.2 Setup Instructions 104
3.15.2.3 Setup Troubleshooting 104
3.15.3 Usage 104
3.15.3.1 Basic Instructions 104
3.15.3.2 Detailed Instructions 104
3.15.3.3 Testing/Error Reporting 105
3.16 Network Recovery Mechanism 105
3.16.1 Overview 105
3.16.1.1 Platform Content Overview 105
3.16.1.2 Product Expectations 105
3.16.1.3 Extending Platform Functionality 105
3.16.2 Setup 105
3.16.2.1 Tools/Equipment Needed 105
3.16.2.2 Setup Instructions 105
3.16.2.3 Setup Troubleshooting 105

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.16.3 Usage 105
3.16.3.1 Basic Instructions 105
3.16.3.2 Detailed Instructions 106
3.16.3.3 Testing/Error Reporting 108
3.17 Power-up Operating Mode 108
3.17.1 Overview 108
3.17.1.1 Platform Content Overview 108
3.17.1.2 Product Expectations 108
3.17.1.3 Extending Platform Functionality 109
3.17.2 Setup 109
3.17.2.1 Tools/Equipment Needed 109
3.17.2.2 Setup Instructions 109
3.17.2.3 Setup Troubleshooting 109
3.17.3 Usage 109
3.17.3.1 Basic Instructions 109
3.17.3.2 Detailed Instructions 109
3.17.3.3 Testing/Error Reporting 110
3.18 RTT (Real Time Text) 110
3.18.1 Overview 110
3.18.1.1 Platform Content Overview 110
3.18.1.2 Product Expectations 110
3.18.1.3 Extending Platform Functionality 111
3.18.2 Setup 111
3.18.2.1 Tools/Equipment Needed 111
3.18.2.2 Setup Instructions 111
3.18.2.3 Setup Troubleshooting 111
3.18.3 Usage 112
3.18.3.1 Basic Instructions 112
3.18.3.2 Detailed Instructions 112
3.18.3.3 Testing/Error Reporting 120
3.19 Persistence configuration 120
3.19.1 Overview 120
3.19.1.1 Platform Content Overview 120
3.19.1.2 Product Expectations 120
3.19.1.3 Extending Platform Functionality 120
3.19.2 Setup 121
3.19.2.1 Tools/Equipment Needed 121
3.19.2.2 Setup Instructions 121
3.19.2.3 Setup Troubleshooting 121
3.19.3 Usage 121
3.19.3.1 Basic Instructions 121
3.19.3.2 Detailed Instructions 121
3.19.3.3 Testing/Error Reporting 121

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

4 Known Issues 122
5 FAQs 122
6 Troubleshooting/Tips and Tricks 122
7 References 122
7.1 Abbreviations/Acronyms/Definitions 122
7.2 Platform Documentation 122
7.3 Website links 122
8 Revision History 123

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

1 Introduction

This document provides usage information for Network Access Device InterFace, or simply NADIF.

NADIF is defined as a simplified functional interface that allows an application program written in

"C" language to communicate with the embedded cellular device. In addition, NADIF supports multiple clients and provides a simple mechanism for asynchronous event notification via a callback function.

The terms NADIF Server and NADIF Client will be referred to throughout this document according to the following definitions:

- NADIF Server is a Linux daemon that runs on the applications processor and exposes public APIs that can be invoked by user applications to access "service groups" provided by an embedded cellular device (embedded modem, or network access device), such as Voice, Data, SMS, cellular Network, etc. Each "service group" can provide multiple services, for example, voice "service group" can provide services like "dial", "answer", "hang-up" etc.
- NADIF Client is an application program that must be registered in NADIF server to use the APIs.

This user guide will cover details about features provided by these services, configuration usage, and dynamic behavior. The target audience for this document is developers and architects who will use the Telematics Platform NADIF services.

The NADIF Server subsystem resides between the user application and modem layer as shown below:

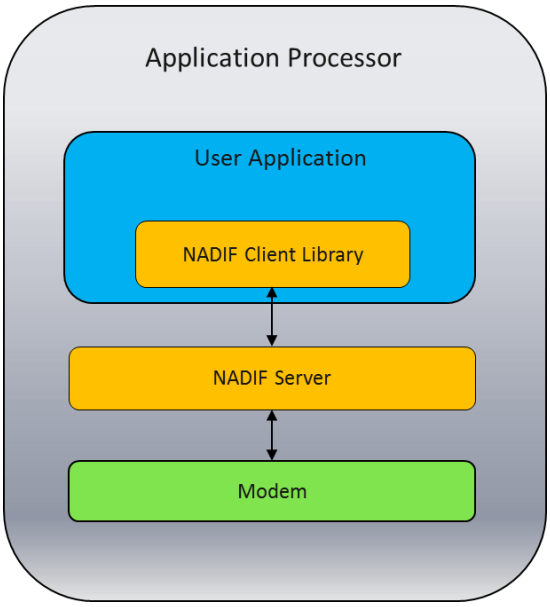


Figure 1: NADIF Context Diagram

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*

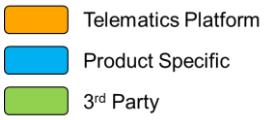
© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend



2 Setup

These instructions are designed to help the user of NADIF, also known as NADIF Client, to set up a project capable of using the services provided by NADIF.

2.1 Tools/Equipment Needed

Not Applicable

2.2 Setup Instructions

- The user of NADIF must use the *NADIF Client Library,* this library is responsible for controlling all socket logic, such as connection, sending request, and listening to notifications and asynchronous response.
- The *NADIF Client Library* is coined as **conti-nadif.** The user of NADIF has to use pkg-config in the client application Makefile to link to the NADIF Client library.
```
LDFLAGS += `pkg-config --libs conti-nadif`
CFLAGS += `pkg-config --cflags conti-nadif`
```
- NADIF Client must include **nadif.h** and **nadif_control.h** in the source file to access common defines used by NADIF.
```
#include "nadif.h"
#include "nadif_control.h"
```
- NADIF Client must be registered in NADIF Server to invoke any NADIF API and receive notifications, see all details in Client Registration.
- In order to invoke any API, NADIF Client must include the appropriate header file according to the service group that the API belongs to. See the Product Expectations subsection of each API service group to find the appropriate header file.
- **Security**: Usage of NADIF APIs require clients (user) to be a member of '**modem**' group (see TP_F101U-00009).

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

2.3 Setup Troubleshooting

Not Applicable

3 Features/Components

3.1 Client Registration

3.1.1 Overview

3.1.1.1 Platform Content Overview

This section describes how the application program can register and de-register a client in NADIF server.

NADIF client must be successfully initialized and registered before requesting any NADIF service.

The NADIF client registration procedure must register a callback to receive notifications and asynchronous responses. A sender ID will be informed to NADIF client to use in subsequent requests.

NADIF server communicates with clients through sockets, making it possible to support multiple NADIF clients in different processes or either multiple NADIF clients running in the same process.

Upon exiting the application, the program should de-register the NADIF client.

3.1.1.2 Product Expectations

The following expectations must be met by Application Program in order to register or de-register a NADIF client as designed:

1. NADIF Client library must be used to communicate with NADIF server as mentioned in the Setup section.
2. Application Program should wait for NADIF Server to be registered in DBus. See Register Client subsection for more details.
3. Application Program must request `NADIF_init_extended()` to register a client in NADIF server for each needed SIM subscription.
4. Application Program should request `NADIF_release()` to de-register a client before exiting application program to release all resource associated.

Note: NADIF_init() is still supported for a Single SIM Subscription.

3.1.1.3 Extending Platform Functionality

Not Applicable

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Not Applicable

3.1.2.2 Setup Instructions

Not Applicable

3.1.2.3 Setup Troubleshooting

Not Applicable

3.1.3 Usage

3.1.3.1 Basic Instructions

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.1.3.2 Detailed Instructions

3.1.3.2.1 Query Capabilities

API/Type Description	Doxygen Path
API to check multi SIM subscription support	Modules |-> Modem |-> NADIF |-> Control |-> **NADIF_query_extended_capabilities()**
Return data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_CLIENT_EXTENDED_CAPABILITIES**
Callback prototype to receive notifications.	Modules |-> Modem |-> NADIF |-> Notifications |-> **NOTIFICATION_FUNC**

The purpose of API *NADIF_query_extended_capabilities()* is to check if device supports Multi-SIM or not. This works in synchronous mode. This API can be called before NADIF_init_extended().

For Multiple SIM enabled platform (say for DSDA), the return type *NADIF_CLIENT_EXTENDED_CAPABILITIES* conveys three parameters describing the current configuration of client namely,

a) *result* : indicating the result of querying the nadif capabilities
(For DSDA to be supported, *result* must be equal to '*NADIF_RESULT_SUCCESS*')

b) *dsda_support* : indicating availability of Dual Sim Dual Active capability.
(For DSDA to be supported, *dsda_support* must be equal to '*TRUE*')

c) *max_subscriptions* : indicating maximum subscription available

(For switching of subscription, minimum value of *max_subscriptions* must be '*2*' *where* the term *max_subscriptions* refers to the maximum number of SIM Slots supported by the module)

3.1.3.2.2 Register Client

API/Type Description	Doxygen Path
API to register a NADIF client	Modules |-> Modem |-> NADIF |-> Control |-> **NADIF_init_extended()**
Return data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**
Callback prototype to receive notifications.	Modules |-> Modem |-> NADIF |-> Notifications |-> **NOTIFICATION_FUNC**

The purpose of API `NADIF_init_extended()` is to register a NADIF client, including Multiple SIM support.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

Before registering a new client, the application program must make sure that NADIF server is already registered in DBus, the following API available in `tpSVCMON.h` can be used (see TP_F101U-00023 Common Library user guide for more details):

```
tpSVCMON_registerServiceStateChange(NADIF_SERVICE, FALSE, callback, &id);
```

***NOTE**: NADIF client must first call `tpSVCMON_init()`.*

To register a new client in NADIF, the application program must call `NADIF_init_extended` () to register on either subscription in any order, assigning a user callback-function to `*callback` parameter to handle notifications and asynchronous responses. It's a paramount need that client is aware of the capabilities of platform using the API `NADIF_query_extended_capabilities()` before trying to register on `NADIF_SIM_SUBSCRIPTION_1`

Upon successful response from `NADIF_init_extended()`, the callback will be successfully registered.

The parameter `*client_id`, provided by `NADIF_init_extended()`, must be used for subsequent requests.

3.1.3.2.3 De-register Client

API/Type Description	Doxygen Path

API to de-register a NADIF client	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Control`
	`|-> NADIF_release()`

The NADIF Client should call `NADIF_release()` to de-register a client before exiting application program to release all resources acquired during the initialization.

See the following two MSCs for more details about registering and de-registering a NADIF Client on two different Subscriptions:

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

For Multiple SIM enabled platform : Single Client Registration sequence for DSDA

For SIM Subscription 'NADIF_SIM_SUBSCRIPTION_0'
NADIF_CLIENT_CONFIGURATION config1;
config1.sim_subscription = NADIF_SIM_SUBSCRIPTION_0;

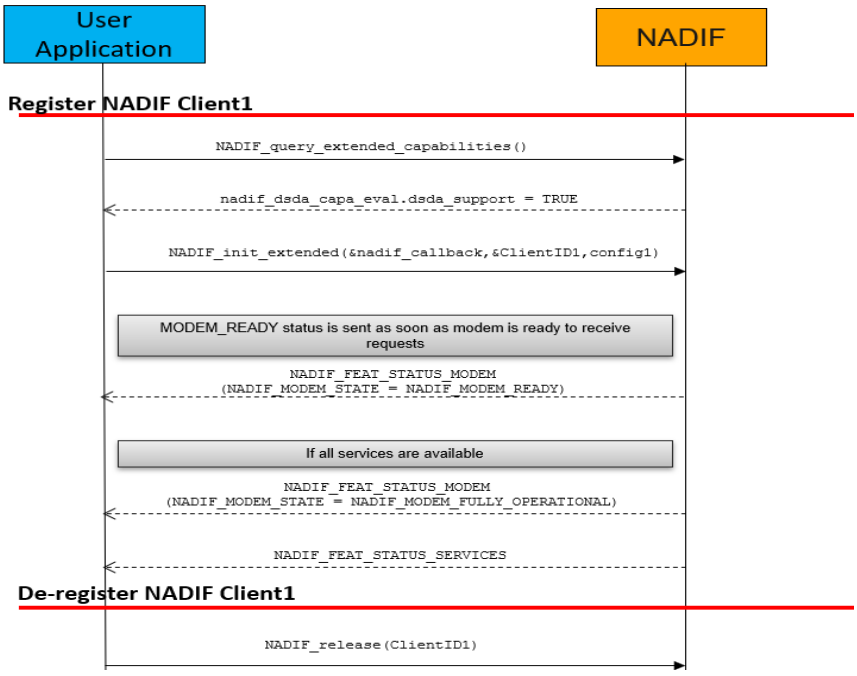


Figure 2: Register / De-register sequence for Multi SIM on Subscription 0

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

For SIM Subscription 'NADIF_SIM_SUBSCRIPTION_1'
NADIF_CLIENT_CONFIGURATION config2;
config2.sim_subscription = NADIF_SIM_SUBSCRIPTION_1;

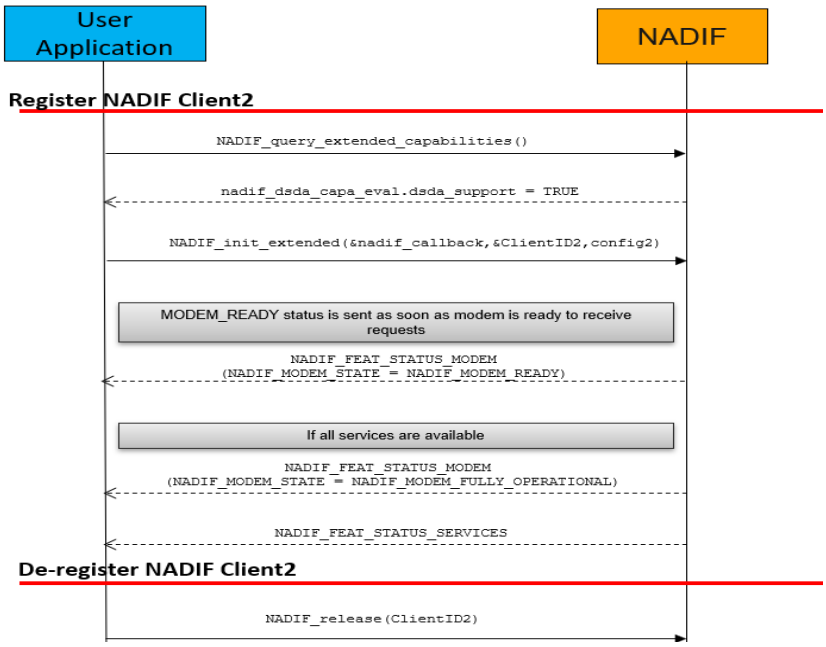


Figure 3: Register / De-register sequence for Multi SIM on Subscription 1

Note: If user wants to receive the call back on 'NADIF_SIM_SUBSCRIPTION_1' , user should first check the availability of subscription 'NADIF_SIM_SUBSCRIPTION_1' using 'NADIF_query_extended_capabilites()'. The API NADIF_query_extended_capabilites() would return the support of DSDA capability via flag 'dsda_support' to the user.

Typically NADIF Server sends three unsolicited notifications after a client is registered.

Once NADIF server is initialized, it sends an unsolicited notification of `NADIF_FEAT_STATUS_MODEM` with the status `NADIF_MODEM_READY`. Immediately after `NADIF_MODEM_READY`, NADIF Server sends another `NADIF_FEAT_STATUS_MODEM`, this time the status indicates whether all the services are available or not:

- `NADIF_MODEM_FULLY_OPERATIONAL`: The Modem is functional and all services are available. In this state all request will be send to the modem
- `NADIF_MODEM_NOT_FULLY_OPERATIONAL`: The Modem is functional but not all services are available. In this state some request will no be process as the service related is not ready.

Lastly, NADIF Server sends an unsolicited notification `NADIF_FEAT_STATUS_SERVICES`, this is a list with the status of all the services, informing whether each of them is available or not.

NOTE: If any service reported by `NADIF_FEAT_STATUS_SERVICES` is different than available, the state reported by `NADIF_FEAT_STATUS_MODEM` will be `NADIF_MODEM_NOT_FULLY_OPERATIONAL`.

See Modem Status section for more details.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.1.3.2.4 Code Example

The following code example shows in details how the Application Program can use the APIs to register/de-register a NADIF client:

Code Example: Register and De-Register a NADIF Client

```
/* Includes */
#include <pthread.h>
#include "tpsys.h"
#include "tpSVCMON.h"
#include "nadif.h"
#include "nadif_control.h"

/* Prototypes */
uint8  nadif_callback(NOTIFICATION_REQUEST_DATA  *rqst_data,  uint32  size,  void
*data);
static    void    handle_service_state_cb(const    gchar*    service_name,    const
tpSVCMON_ServiceState_t _state);
static void waitForServiceReady(void);

/*Local Variables*/
pthread nadif_thread;
static gboolean service_ready = FALSE;
static GMutex service_ready_mutex;
static GCond service_ready_cond;
/*****************************************************************************
**          main
**
**          Performing:
**          - Register a client
**          - DSDA capability check before registering a client
**          - Create a thread to listen to the NADIF Server
**          - De-register a client
**
*****************************************************************************/
int main (int argc, char* const argv[])
{
      /* Variables for DSDA */
      uint32 client_id1 = 0;
      uint32 client_id2 = 0;
      NADIF_CLIENT_EXTENDED_CAPABILITIES  nadif_dsda_capa_eval;
      NADIF_CLIENT_CONFIGURATION config1,config2;
      config1.sim_subscription = NADIF_SIM_SUBSCRIPTION_0;
      config2.sim_subscription = NADIF_SIM_SUBSCRIPTION_1;

      gint monitorID = 0;
      /* Setup IPC
         Note: It only needs to be called if it was not initialized in the
         NADIF client process yet */
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

```
      if(tpSYS_initIPC(E_SYS_IPC_DBUS, NULL,NULL) == E_SYS_IPC_RET_FAILURE)
      {
            /* Unable to setup IPC */
            return 1;
      }
      /* Wait for NADIF Server to be registered in DBus */
      if(tpSVCMON_init() != E_SVCMON_ERROR_NONE){
            /* Unable to initialize SVCMON */
            return 1;
      }
      tpSVCMON_registerServiceStateChange(NADIF_DBUS_SERVICE,FALSE,handle_service
_state_cb, &monitorID);
      waitForServiceReady();

/* DSDA support check before registering a client */
      nadif_dsda_capa_eval = NADIF_query_extended_capabilities();


      if((nadif_dsda_capa_eval.result==NADIF_RESULT_SUCCESS)&&(
          nadif_dsda_capa_eval.dsda_support==TRUE)&&(nadif_dsda_capa_eval.
          max_subscriptions>1))
             printf("DSDA is supported”);

      /* Register clients for multi subscriptions when DSDA is supported */

      /* the callback functions used for both subscriptions is same however
              the user can use different depending upon their design */

      /* Register client for sub0*/

     if (NADIF_RESULT_SUCCESS == NADIF_init_extended(&nadif_callback,
          &client_id1, config1))
     {

            printf("Success on registering nadif client for Subscription %d",
                config1.sim_subscription);
     }
     else
     {
            /* Unable to initialize Sub 0 */
            printf("Unable to register client for Subscription %d",
                    config1.sim_subscription);
     }
      /* Register client for sub1*/
     if (NADIF_RESULT_SUCCESS == NADIF_init_extended(&nadif_callback,
         &client_id2, config2))
     {
            printf("Success on registering nadif client for Subscription %d",
                    config2.sim_subscription);
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

```
        }
        else
        {
                /* Unable to initialize Sub 1 */
                printf("Unable to register client for Subscription %d",
                        config2.sim_subscription);
        }

        while (1)
        {
                /* This is the main loop */
                /* ... */
        }
        /* De-register client1 */
        if(client_id1 != 0)
        {
                NADIF_release(client_id1);
        }
        /* De-register client2 */
        if(client_id2 != 0)
        {
                NADIF_release(client_id2);
        }

        /* De-Init IPC setup */
        tpSYS_deinitIPC();

        return 0;
}

/*******************************************************************************
**
**      nadif_callback
**
**      Performing:
**      - This callback function handles notifications and asynchronous responses.
**      - In the following example, it only checks the modem status
**
**      Parameters:
**      [in]      NOTIFICATION_REQUEST_DATA*    - Typically used to extract
Feature ID. See more details on doxygen.
**      [in]      uint32      - Size of the data pointed by void*
**      [in]      void* - Notification Data. Should be cast to the appropriate
data structure based on the Feature ID.
**
**      Returns
**      uint8 - 0-Success, 1-Failure.
**
*******************************************************************************/
uint8 nadif_callback(NOTIFICATION_REQUEST_DATA *rqst_data, uint32 size, void *data)
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

```
{
      boolean boError = FALSE;
      NADIF_MODEM_STATUS *modem_status_resp;

      /* print subscription id for call back*/
      printf("Response   is   coming   from   subscription   =   %d   ",rqst_data-
      >subscription_id);
      switch(rqst_data->feat_id)
      {
            case NADIF_FEAT_STATUS_MODEM:
            modem_status_resp = (NADIF_MODEM_STATUS*)data;
            /* Modem Status: modem_status_resp->modem_status */
            break;

            default:
            break;
      }

      return (uint8)boError;
}

static void waitForServiceReady() {
      g_mutex_lock(&service_ready_mutex);
      while (!service_ready) {
            g_cond_wait(&service_ready_cond, &service_ready_mutex);
      }
      g_mutex_unlock(&service_ready_mutex);
}

static gpointer client_thread(gpointer data)
{
      g_mutex_lock (&service_ready_mutex);
      service_ready = is_ready;
      g_cond_signal (&service_ready_cond);
      g_mutex_unlock (&service_ready_mutex);
      return NULL;
}

static    void    handle_service_state_cb(const    gchar*    service_name,    const
tpSVCMON_ServiceState_t _state){
      static           tpSVCMON_ServiceState_t           previous_state           =
E_SVCMON_SERVICE_STATE_NOT_READY;

      /* Service Monitor calls this callback from tp-common socket client thread
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

```
        * It's not recommended to do much processing in this function
        * Better to create another thread for the work */
      if(E_SVCMON_SERVICE_STATE_READY == _state){
            if(E_SVCMON_SERVICE_STATE_NOT_READY == previous_state){
                  g_thread_unref(g_thread_new("client_thread",
(GThreadFunc)client_thread, NULL));
            }
      }
      previous_state = _state;
}
```

3.1.3.3 Testing/Error Reporting

Not Applicable

3.2 Multiple SIM Subscription Support

3.2.1 Overview

3.2.1.1 Platform Content Overview

This section describes how the application program can register, de-register multiple clients in NADIF server for multiple SIM subscriptions to access various features namely Voice call, Emergency call, SMS using different radio access technologies like 5G, LTE, WCDMA, GSM. In addition to this, switching of subscription to access specific services is also explained in this section.

Note: Multiple SIM subscriptions are required for Dual SIM Dual Active (DSDA) operation. Only one profile (subscription) may be active at any given time on an eUICC (embedded SIM) that supports multiple profiles, even when multiple SIM subscriptions are supported (i.e., multiple SIM subscription requires 2 or more physical SIMs/eUICCs). Multiple SIM subscriptions are not supported on all platform releases. Consult release notes to determine if multiple SIM subscriptions, including DSDA, are supported on a particular release.

3.2.1.2 Product Expectations

In order to register on Subscription 0 and Subscription 1, the user can refer the sections 3.1.1.2, 3.1.3.2.1, 3.1.3.2.2, and 3.1.3.2.3.

The following expectations must be met by Application Program in order to switch subscription:

1. The user may check and confirm current subscription via API *`NADIF_client_get_current_configuraton()`*.
2. The user can request for switching to a different subscription via `NADIF_client_set_configuration()`.
3. Return value of API `NADIF_client_set_configuration()` indicates subscription switch status (`TRUE` – success, `FALSE` – fail).

3.2.1.3 Extending Platform Functionality

Not Applicable

3.2.2 Setup

3.2.2.1 Tools/Equipment Needed

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.2.2.2 Setup Instructions

Not Applicable

3.2.2.3 Setup Troubleshooting

Not Applicable

3.2.3 Usage

3.2.3.1 Basic Instructions

Not Applicable

3.2.3.2 Detailed Instructions

3.2.3.2.1 Switching of subscription

API/Type Description	Doxygen Path

API to check current registered subscription- id binded to a given client-id.	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Control`
	`|->`
	`NADIF_client_get_current_configuration()`

Return data type of above API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|->` ***`NADIF_CLIENT_CONFIGURATION`***

API to switch subscription for the given client-id.	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Control`
	`|->` **`NADIF_client_set_configuration()`**

The purpose of `NADIF_client_get_current_configuration()` is to get the current subscription-id for a given client ID.

The purpose of `NADIF_client_set_configuration()` is to switch the configuration for the client-id. If the configuration setting is successful, the API will return TRUE(boolean).

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

See a typical MSC for more details about registering of two NADIF Clients on same Subscriptions followed by switching one of the client from one Subscription to another :

Precondition: Registration of two clients on Subscription 0

For SIM Subscription both clients are getting registered on 'NADIF_SIM_SUBSCRIPTION_0'
NADIF_CLIENT_CONFIGURATION config1,config2;
config1.sim_subscription = NADIF_SIM_SUBSCRIPTION_0;
config2.sim_subscription = NADIF_SIM_SUBSCRIPTION_0;

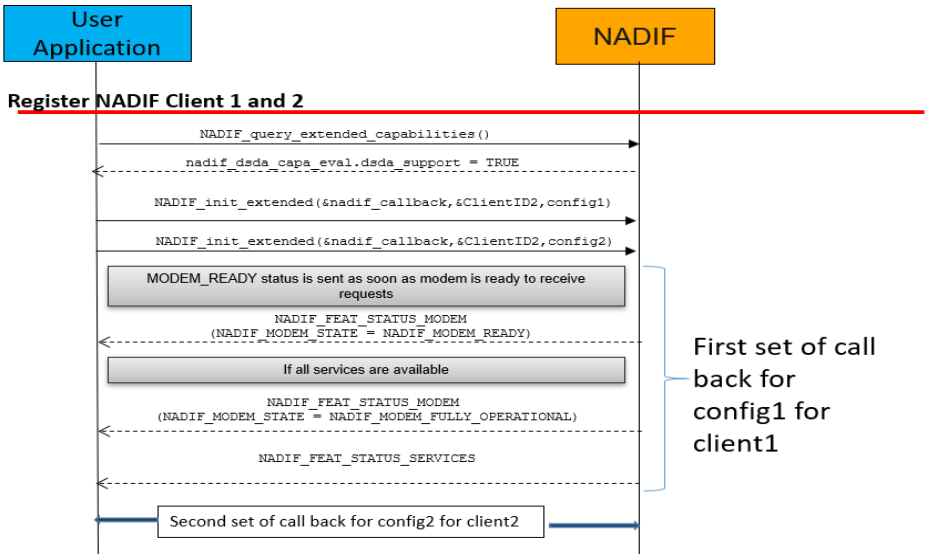


Figure 4: Register sequences for Multi SIM on Subscription 0

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

Client2 switch from Subscription 0 to 1:

Query returned clients' current subscription is NADIF_SIM_SUBSCRIPTION_0
Client2 wants to switch from NADIF_SIM_SUBSCRIPTION_0 **to** NADIF_SIM_SUBSCRIPTION_1.
NADIF_CLIENT_CONFIGURATION config1,config2;
config.sim_subscription = NADIF_SIM_SUBSCRIPTION_1;

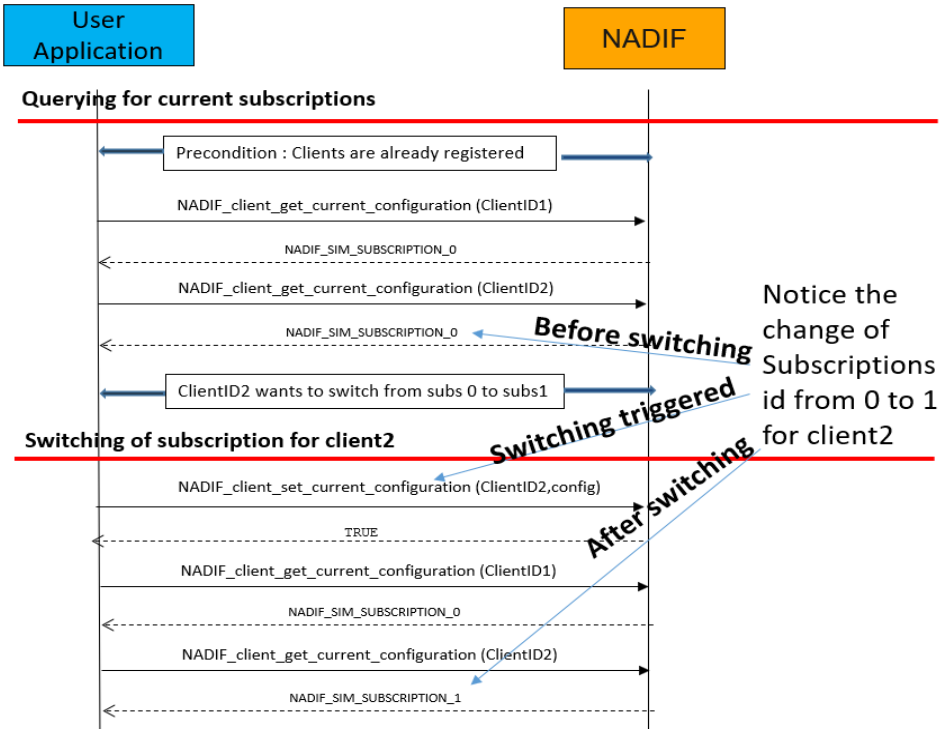


Figure 5: client2 switch to Subscription 1

3.2.3.2.2 Features supporting Multiple SIM subscription

Feature	Supported/Not supported
Debug Level	Not supported
Data Services	Not supported
DTMF	Supported
Network	Supported Exception - API's supporting primary subscription only: • NADIF_net_set_spy_mode()
On-Device Logging	Not supported
SIM	Supported Exception - API's supporting primary subscription only:

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

Feature	Supported/Not supported
	• NADIF_sim_get_current_profile_info() • NADIF_sim_get_profile_info() • NADIF_sim_era_switch_profile() • NADIF_sim_set_ecall_only_emulation() • NADIF_sim_get_ecall_only_emulation() • NADIF_sim_set_remote_profile_switch_lock_config() • NADIF_sim_get_remote_profile_switch_lock_config()
SMS	Supported
Voice Call	Supported
Modem Status	Supported
Standard eCall	Supported Exception - API's supporting primary subscription only: • NADIF_ecall_initiate() • NADIF_ecall_update_msd() • NADIF_ecall_cancel() • NADIF_ecall_get_call_info() • NADIF_ecall_answer() • NADIF_ecall_callback_mode_rat_config()
Private eCall	Supported Exception - API's supporting primary subscription only: • NADIF_ecall_stop_msd() • NADIF_start_transmit_msd()
External MIPI switch Control	Not supported
Phone Book Manager	Supported
Network Recovery Mechanism	Supported
Power-up Operating Mode	Not supported
RTT (Real Time Text)	Supported
Persistence Configuration	Supported

Note: API supporting multiple SIM subscription will be processed on the SIM subscription that the NADIF client is bound to.

3.2.3.3 Testing/Error Reporting

Not Applicable

3.3 General Request/Response

3.3.1 Overview

3.3.1.1 Platform Content Overview

This section generically describes how the application program can request a service from NADIF and receive the corresponding response.

NADIF client must be successfully initialized and registered before requesting any NADIF service, see more details in Client Registration section.

The NADIF APIs are divided into service groups, such as Voice, SMS, Network, Modem Status, etc. Each service group provides multiple functionalities, for example, the Voice service group provides services like dial, answer, hang-up, etc.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

All responses to the requested services are asynchronous and they are handled in the assigned callback-function during the client registration.

Note: The response notification is only notified to the NADIF client that is invoking the API.

3.3.1.2 Product Expectations

The following expectations must be met by NADIF Client in order to request a service from NADIF Server and receive the response as designed:

1. NADIF Client must include the header file according to the service group that the API to be invoked belongs to. See the Product Expectations subsection of each API service group to find the appropriate header file.
2. NADIF Client must evaluate the API return (`NADIF_STATUS`) to verify whether the request has been sent to NADIF Server successful or not.
3. NADIF Server sends the response asynchronously after processing the request, then the NADIF Client must perform the following steps in the registered callback-function:
4. Filter the corresponding response by the enumeration `NADIF_FEATURE_ID`.
5. Cast the data of the received response to the appropriate structure according to the notification data type described in Doxygen.
6. Check the result of the response in `NADIF_RESULT` element in data, if it is `NADIF_RESULT_SUCCESS` then the response data of the requested service is valid.

More details about the NADIF types referenced in this section see the doxygen links in the following tables:

Type Description	Doxygen Path

Enumeration of request status	`Modules`
	`  |-> NADIF`
	`  |-> NADIF Common Types`
	`  |-> NADIF_STATUS`

Enumeration of feature ID	`Modules`
	`  |-> NADIF`
	`  |-> NADIF Common Types`
	`  |-> NADIF_FEATURE_ID`

Enumeration of error response codes	`Modules`
	`  |-> NADIF`
	`  |-> NADIF Common Types`
	`  |-> NADIF_RESULT`

3.3.1.3 Extending Platform Functionality

Not Applicable

3.3.2 Setup

3.3.2.1 Tools/Equipment Needed

Not Applicable

3.3.2.2 Setup Instructions

Not Applicable

3.3.2.3 Setup Troubleshooting

Not Applicable

3.3.3 Usage

3.3.3.1 Basic Instructions

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.3.3.2 Detailed Instructions

The message sequences to request and response are similar for all APIs. The invoked API returns whether the request has been sent successfully or not to NADIF Server. In case of success, NADIF Server will process the requested service and send the response asynchronously to NADIF Client.

The enumeration `NADIF_FEATURE_ID` contains all IDs for notifications for the asynchronous responses, the IDs for API notifications have the prefix `NADIF_FEAT_STATUS` and reside in the range between `NADIF_FEAT_END_UNSOL_NOTIFICATIONS to NADIF_FEAT_MAX_LIST`.

Refer to the following table to cast the API requested to the corresponding response notification:

Type Description	Doxygen Path

Enumeration of Feature ID	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_FEATURE_ID`

The following MSC demonstrates a generic message sequence of requesting a service and receiving a response from NADIF Server.

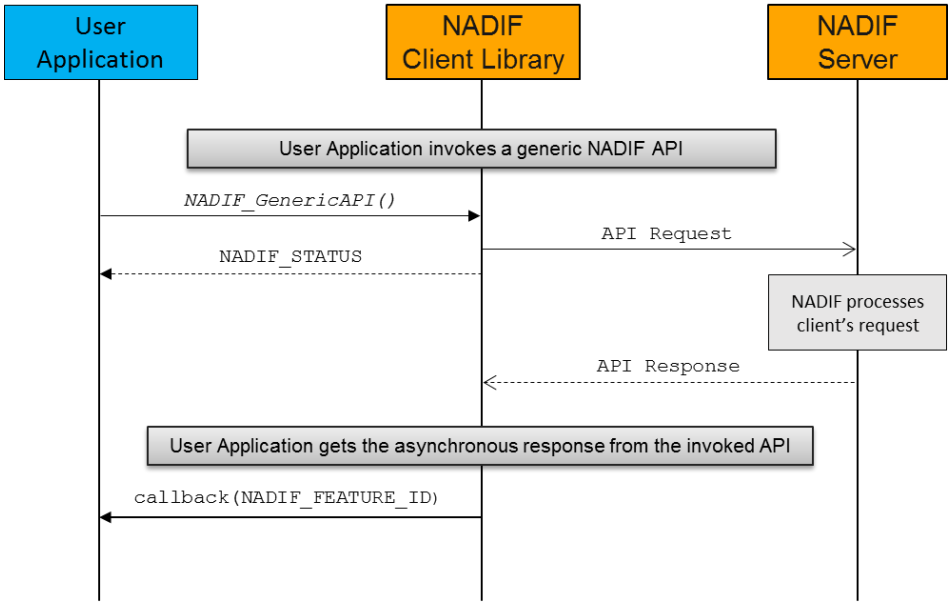


Figure 6: Generic Request Sequence

The API to get the modem serial number (IMEI) will be used as an example to implement a typical NADIF request and response.

See more details about the API to get modem serial number in the following doxygen links:

API/Type Description	Doxygen Path
API to query serial number	`Modules`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_get_serial_number**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_GET_SERIAL_NUMBER_RESP**

The following code example shows in details how NADIF Client can invoke an API and get the asynchronous response:

Code Example: Get Serial Number

```
/* Includes */
#include "nadif.h"
#include "nadif_control.h"
#include "nadif_modem_status.h"

/******************************************************************************
**
**          Get Modem Serial Number (IMEI)
**
**          Performing:
**          - Get modem serial number to demonstrate request/response flow.
**
**          Parameters:
**          [in]  uint32 - Client ID informed by NADIF_init_extended() during the
NADIF     Client registration.
**
******************************************************************************/
void GetSerialNumber (uint32 ClientID)
{
      NOTIFICATION_REQUEST_DATA rqst;
      rqst.sender_id = ClientID;
      status = NADIF_modem_get_serial_number(&rqst,NULL);
      if(status == NADIF_STATUS_SUCCESS)
      {
            /* Request sent successfully */
      }
      else
      {
            /* Request failed */
      }
}
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

```
/*****************************************************************************
**
**          nadif_callback
**
**          Performing:
**          - This callback function handles notifications and asynchronous
responses.
**          - This callback has to be registered by NADIF_Init_extended.
**
**          Parameters:
**          [in]  NOTIFICATION_REQUEST_DATA*    -   Typically   used   to   extract
Feature ID. See more details on doxygen.
**          [in]  uint32      - Size of the data pointed by void*
**          [in]  void* - Notification Data. Should be cast to the appropriate data
structure based on the Feature ID.
**
**          Returns
**          uint8 - 0-Success, 1-Failure.
**
*****************************************************************************/
uint8 nadif_callback(NOTIFICATION_REQUEST_DATA *rqst_data, uint32 size, void *data)
{
      boolean boError = FALSE;
      NADIF_GET_SERIAL_NUMBER_RESP *serial_resp;

      /* Filter by NADIF_FEATURE_ID */
      switch(rqst_data->feat_id)
      {
            case NADIF_FEAT_MODEM_GET_SERIAL_NUMBER:
            /*Cast the data of the received response to the appropriate structure*/
            serial_resp = (NADIF_GET_SERIAL_NUMBER_RESP *)(data);
            /* Check NADIF_RESULT */
            if (serial_resp->result == NADIF_RESULT_SUCCESS)
            {
                  /* Response: NADIF_FEAT_MODEM_GET_SERIAL_NUMBER */
                  /* IMEI: serial_resp->serial_number */
                  /* IMEI Software Version: serial_resp->imei_sv */
            }
            break;
            /* Other notifications should be added here as needed */
            /* case ...  */
            default:
```

Formatted: German (Germany)

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

```
            break;
        }

        return (uint8)boError;
}
```

3.3.3.3 Testing/Error Reporting

Not Applicable

3.4 Unsolicited Notification

3.4.1 Overview

3.4.1.1 Platform Content Overview

Unsolicited notifications are events sent by NADIF Server to report an internal status change or notify the user of any indication received by the cellular network, such as incoming call, incoming SMS, change of network registration status, etc. Unsolicited notifications are typically sent to all NADIF clients, however, they can be disabled individually for each client.

This section describes in detail how NADIF Client handles the unsolicited notifications and how to enable or disable them.

Note: Unsolicited Notification will be notified to all the NADIF clients.

3.4.1.2 Product Expectations

3.4.1.2.1 Receive Unsolicited Notification

The following expectations must be met by NADIF Client in order to receive the unsolicited notification as designed:

1. NADIF Client must use the same asynchronous response mechanism as General Request/Response to receive unsolicited notifications, so the registered callback-function must filter the unsolicited notification by `NADIF_FEATURE_ID`.
2. NADIF Client must include the header file according to the service group that includes the unsolicited notification in order to cast the received data to the corresponding notification data type.

3.4.1.2.2 Enable/Disable Unsolicited Notification

NADIF Client must include the header file `nadif_control.h` to access the APIs to enable/disable unsolicited notification. There are two APIs capable to perform this feature, as shown below:

1. NADIF Client can request `NADIF_set_unsolicited_feat_report()` to enable/disable an unsolicited notification individually.
2. NADIF Client can request `NADIF_set_unsolicited_feat_report_list()` to enable/disable a list of unsolicited notifications.

NOTES:

1. *Only the client that is invoking the API has the unsolicited notification disabled/enabled, other NADIF clients will not affected.*
2. Only the unsolicited notification requested to disable or enable is changed, other reports will not be affected.

3.4.1.3 Extending Platform Functionality

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.4.2 Setup

3.4.2.1 Tools/Equipment Needed

Not Applicable

3.4.2.2 Setup Instructions

Not Applicable

3.4.2.3 Setup Troubleshooting

Not Applicable

3.4.3 Usage

3.4.3.1 Basic Instructions

Not Applicable

3.4.3.2 Detailed Instructions

3.4.3.2.1 Unsolicited Notification

The enumeration `NADIF_FEATURE_ID` contents all IDs for unsolicited notifications and asynchronous responses, the IDs for unsolicited notifications have the prefix `NADIF_FEAT_STATUS` and reside in the range between `NADIF_FEAT_UNKNOWN` to `NADIF_FEAT_END_UNSOL_NOTIFICATIONS`.
This document does not describe all unsolicited notifications. Some notifications were designed to be used by specific platform modules, however, they are broadcast to all NADIF clients. User application does not need to take any action for undocumented notifications.

Refer to the following table to cast the received data to the corresponding notification data type:

API/Type Description	Doxygen Path

Enumeration of Feature ID	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_FEATURE_ID`

3.4.3.2.2 Code Example

The following code example shows how NADIF Client handles an unsolicited notification, in this example an incoming SMS is demonstrated. Note that the same callback used to handle asynchronous response is used for unsolicited notifications.

Code Example: Unsolicited Notification (SMS)

```
/* Includes */
#include "nadif.h"
#include "nadif_control.h"
#include "nadif_sms.h"

/*****************************************************************************
**
**          nadif_callback
**
**          Performing:
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

```
**          - This callback function handles notifications and asynchronous
responses.
**          - This callback has to be registered by NADIF_Init_extended.
**
**          Parameters:
**          [in]  NOTIFICATION_REQUEST_DATA*    -   Typically   used   to   extract
Feature ID. See more details on doxygen.
**          [in]  uint32      - Size of the data pointed by void*
**          [in]  void* - Notification Data. Should be cast to the appropriate data
structure based on the Feature ID.
**
**          Returns
**          uint8 - 0-Success, 1-Failure.
**
*****************************************************************************/
uint8 nadif_callback(NOTIFICATION_REQUEST_DATA *rqst_data, uint32 size, void
*data)
{
      boolean boError = FALSE;
      NADIF_SMS_DATA *sms_data;
      /* Filter by NADIF_FEATURE_ID */
      switch(rqst_data->feat_id)
      {
            case NADIF_FEAT_STATUS_SMS:
            /* Cast data to the appropriate structure */
            sms_data = (NADIF_SMS_DATA *)(data);
            /* SMS Status */
            switch(sms_data->sms_status)
            {
                  case NADIF_SMS_NEW:
                  /* New SMS has been received */
                  break;
                  case NADIF_SMS_MEMORY_FULL:
                  /* SMS memory is full */
                  break;
                  case NADIF_SMS_UNKNOWN_STATUS:
                  /* SMS status is unknown */
                  break;
                  default:
                  /* Invalid enumeration value */
                  break;
            }
            break;
            /* Other notifications should be added here as needed */
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

```
            /* case ...  */
            default:
            break;
        }
        return (uint8)boError;
}
```

3.4.3.2.3 Enable/Disable Unsolicited Notification

All unsolicited notifications are enabled by default when the NADIF Client is registered. It is recommended that NADIF client disable all unsolicited notifications that will not be handled by it, avoiding unnecessary callback calls. See the following APIs to perform this action:

3.4.3.2.3.1 Set NADIF Event Report

API/Type Description	Doxygen Path

Structure of Event Report Request	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Control`
	`|-> NADIF_EVENT_REPORT_REQ`

API to enable/disable a single unsolicited notification.	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Control`
	`|-> NADIF_set_unsolicited_feat_report`

This API is capable to enable/disable a single unsolicited notification reported by NADIF.

The user must enter the unsolicited notification (`NADIF_FEATURE_ID`) and whether the report will be enabled or disabled, see the doxygen path above for more details.

NOTE: This API does not have an asynchronous response, it has an immediate effect as soon as it returns `NADIF_STATUS_SUCCESS`.

3.4.3.2.3.2 Set NADIF Event Report List

API/Type Description	Doxygen Path

Structure of Event Report Request	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Control`
	`|-> NADIF_EVENT_REPORT_REQ`

API to enable/disable a list of unsolicited notification.	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Control`
	`|-> NADIF_set_unsolicited_feat_report_list`

This API is capable to enable/disable a list of unsolicited notification reported by NADIF.

The user must enter the size of the list and an array of `NADIF_EVENT_REPORT_REQ`, containing a list of the unsolicited notification (`NADIF_FEATURE_ID`) and whether the report will be enabled or disabled, see the doxygen path above for more details.

NOTE: This API does not have an asynchronous response, it has an immediate effect as soon as it returns `NADIF_STATUS_SUCCESS`.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.4.3.3 Testing/Error Reporting

Not Applicable

3.5 Debug Level

3.5.1 Overview

3.5.1.1 Platform Content Overview

This section describes how to set or query the debug trace level in NADIF.

All NADIF traces are logged in Linux file system, there is a common file where most of the Telematic software modules store the logs.

The API to set the debug level described in this section affects the traces reported by NADIF Server, and the settings can be a combination of the following levels: Info, Verbose, and Error.

3.5.1.2 Product Expectations

The following expectations must be met by NADIF Client in order to set or query the debug trace level as designed:

1. NADIF Client must include the header file `nadif_control.h` to access the APIs to set/query the debug trace level.
2. NADIF Client is expected to use `NADIF_set_debug_level()` to set the debug level.
3. NADIF Client is expected to use `NADIF_get_debug_level()` to query the current debug level.

3.5.1.3 Extending Platform Functionality

The log file is stored in ***/var/log/messages*** and the NADIF tracing should be parsed by the tag ***"NAD Interface"***.

See the following example:

```
user.info NAD Interface[512]: mnc = 020, mcc = 334
user.info NAD Interface[512]: nadif_nas_ind
user.info NAD Interface[512]: QMI_NAS_NETWORK_TIME_IND_MSG_V01:
user.info NAD Interface[512]: year:2018 month:2 day:19 hour:22 minute:21 second:45 day_of_week:0
user.info NAD Interface[512]: time_zone:-24
user.info NAD Interface[512]: daylt_sav_adj:0
```

Figure 7: NADIF log messages

3.5.2 Setup

3.5.2.1 Tools/Equipment Needed

Not Applicable

3.5.2.2 Setup Instructions

Not Applicable

3.5.2.3 Setup Troubleshooting

Not Applicable

3.5.3 Usage

3.5.3.1 Basic Instructions

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.5.3.2 Detailed Instructions

3.5.3.2.1 Set Debug Level

API/Type Description	Doxygen Path
API to set debug level	Modules |-> Modem |-> NADIF |-> Control |-> **NADIF_set_debug_level()**
Structure of debug level type	Modules |-> Modem |-> NADIF |-> Control |-> **NADIF_SET_DEBUG_LEVEL_REQ**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to set the debug level for NADIF Server. A bitmask configures the debug level, which allows setting multiple levels simultaneously (Info, Verbose and Error).

3.5.3.2.2 Query Debug Level

API/Type Description	Doxygen Path
API to query debug level	Modules |-> Modem |-> NADIF |-> Control |-> **NADIF_get_debug_level()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Control |-> **NADIF_GET_DEBUG_LEVEL_RESP**

The notification of this API reports which levels of debugging are set (Info, Verbose and Error).

3.5.3.3 Testing/Error Reporting

Not Applicable

3.6 DTMF

3.6.1 Overview

3.6.1.1 Platform Content Overview

This section describes how to generate and detect a DTMF tone during a voice call.

There are two APIs responsible for generating a DTMF tone on the far end, start/stop requests. A notification is received by the NADIF Client when a DTMF start/stop request is sent to the network, allowing a better control of the DTMF playing time.

NADIF does not support burst DTMF (i.e., DTMF string), it only supports a single DTMF at a time.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.6.1.2 Product Expectations

NADIF Client must include the header file `nadif_dtmf.h` to access the APIs for DTMF.

3.6.1.2.1 DTMF tone generation

The following expectations must be met by NADIF Client in order to generate a DTMF tone as designed:

1. A voice call must be established.
2. NADIF Client must use `NADIF_dtmf_send()` to start generating a DTMF tone on the far end.
3. NADIF Client must wait for the `NADIF_FEAT_STATUS_DTMF` notification with the `dtmf_play` status `NADIF_DTMF_PLAY_START`.
4. NADIF Client must wait for the period of time for DTMF tone generation on the far end (DTMF tone playing time).
5. NADIF Client must invoke `NADIF_dtmf_stop()` to stop playing the DTMF tone.
6. NADIF Client must wait for the `NADIF_FEAT_STATUS_DTMF` notification with the `dtmf_play` status `NADIF_DTMF_PLAY_STOP`.
7. NADIF client must wait a period of time between tones to send another tone, NADIF Server will not do any timing.

3.6.1.2.2 DTMF detection

The following expectations must be met by NADIF Client in order to detect a DTMF tone as designed:

1. NADIF Client must use `NADIF_dtmf_set_tone_detection()` to enable the DTMF tone detection.
2. A voice call must be established.
3. NADIF Client must evaluate the unsolicited notification `NADIF_FEAT_STATUS_RX_DTMF_DETECTED` to receive the DTMF tone sent by the far end.

3.6.1.3 Extending Platform Functionality

Not Applicable

3.6.2 Setup

3.6.2.1 Tools/Equipment Needed

Not Applicable

3.6.2.2 Setup Instructions

Not Applicable

3.6.2.3 Setup Troubleshooting

Not Applicable

3.6.3 Usage

3.6.3.1 Basic Instructions

Not Applicable

3.6.3.2 Detailed Instructions

3.6.3.2.1 DTMF tone generation

API/Type Description	Doxygen Path

Structure of DTMF Data.	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> DTMF`
	`  |-> NADIF_DTMF_DATA`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path

API to start a DTMF tone	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> DTMF`
	`|-> **NADIF_dtmf_send()**`

API to stop a DTMF tone	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> DTMF`
	`|-> **NADIF_dtmf_stop()**`

Notification data type of APIs	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> **NADIF_RESULT**`

The API `NADIF_dtmf_send()` allows NADIF Client to start playing a DTMF tone on the far end. The DTMF tone must be stopped invoking `NADIF_dtmf_stop`() and NADIF Client must control the tone playing time through `NADIF_FEAT_STATUS_DTMF` notification.

The unsolicited notification `NADIF_FEAT_STATUS_DTMF` gives relevant information about the DTMF tone that has been sent to modem network for playing on remote side. The `NADIF_DTMF_PLAY_TYPE` element reports `NADIF_DTMF_PLAY_START` whenever a DTMF tone has sent to play on the remote, and it reports `NADIF_DTMF_PLAY_STOP` after stopping to play a DTMF tone.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

See a typical use case in the following MSC:

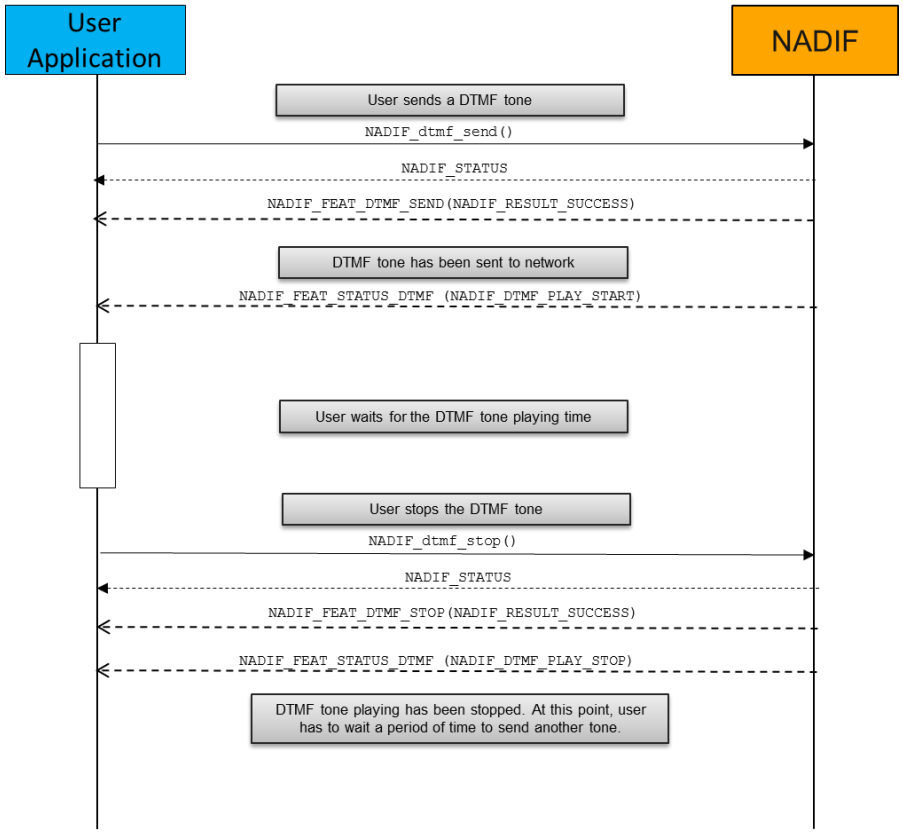


Figure 8: DTMF tone generation sequence

NOTE: NADIF Client must time the DTMF tone playback. When playback needs to end, NADIF Client must invoke `NADIF_dtmf_stop()`. If this API is not invoked to stop tone playback, then cellular network behavior is unpredictable.

3.6.3.2.2 DTMF detection

API/Type Description	Doxygen Path

API to enable DTMF detection	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> DTMF`
	`  |-> NADIF_dtmf_set_tone_detection()`

Notification data type of API	`Modules` `  |-> Modem`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path

	`|-> NADIF`
	`|-> DTMF`
	`|-> NADIF_DTMF_DATA`

The API `NADIF_dtmf_set_tone_detection()` allows the NADIF Client to enable or disable the DTMF tone detection.

The incoming tone is reported through the unsolicited notification `NADIF_FEAT_STATUS_RX_DTMF_DETECTED` when a DTMF tone is received. Only one DTFM tone is reported at a time, NADIF Client must evaluate the parameter `digit_buffer` in `NADIF_DTMF_DATA` to identify the received tone.

See more details in the following MSC:

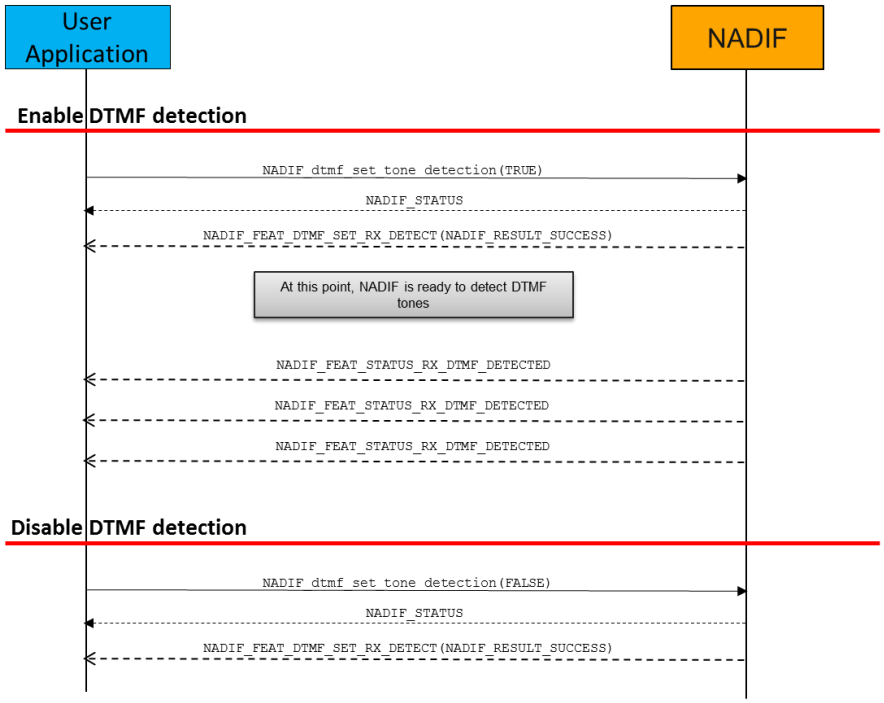


Figure 9: DTMF detection sequence

3.6.3.3 Testing/Error Reporting

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.7 Network

3.7.1 Overview

3.7.1.1 Platform Content Overview

This section describes features related to the cellular-network, such as network registration, network information (MNC and MCC), signal quality, neighboring cells, network scanning, etc.

3.7.1.2 Product Expectations

NADIF Client must include the header file `nadif_network.h` to access any API described in this section.

3.7.1.2.1 Network Registration Setting

NADIF provides means to set the cellular-network registration mode, which allows to perform the registration automatically or manually.

The following expectations must be met by NADIF Client in order to use this feature:

1. The registration mode is expected to be automatic by default. After a power cycle, the registration mode is always returned to automatic.
2. NADIF Client is expected to use `NADIF_net_set_registration()` to set the registration mode to automatic or manual.

3.7.1.2.2 Network Registration Status

NADIF Client is expected to be notified through the unsolicited notification `NADIF_FEAT_STATUS_NETWORK` whenever a network registration status is changed.

NADIF Client can use `NADIF_net_get_registration_status()` to query the registration status and network information.

3.7.1.2.3 Last Registered Network Info

NADIF provides means to query the MNC and MCC of the last registered network. The following expectations must be met by NADIF Client in order to use this feature:

1. The network must be registered at least once.
2. NADIF Client is expected use `NADIF_net_get_last_registered_network_info()` to request the last registered network information.

3.7.1.2.4 Neighboring Cells

NADIF Client is expected to receive the number of neighboring cells by the unsolicited notification `NADIF_FEAT_STATUS_NB_CELLS_NUM` every 5 seconds.

NADIF Client can also use `NADIF_net_get_nb_cells_number()` to query number of neighboring cells of the registered network.

3.7.1.2.5 Signal Quality

NADIF Client is expected to receive the signal quality by the unsolicited notification `NADIF_FEAT_STATUS_SIGNAL_QUALITY` every 5 seconds.

NADIF Client can also use `NADIF_net_get_signal_quality()` to query the current signal quality.

3.7.1.2.6 Network Scan

NADIF Client is expected to use `NADIF_net_perform_scan()` to perform a network scan.

3.7.1.2.7 SPY Mode

NADIF Client is expected to use `NADIF_net_spy_mode()` to enable or disable the SPY mode.

3.7.1.2.8 Cellular DRX

NADIF Client is expected to use `NADIF_net_get_drx()` to query the Cellular DRX cycle coefficient.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.7.1.2.9 IMS Setting

NADIF Client is expected to use `NADIF_ims_set_config()` to enable or disable the IMS support dynamically.

3.7.1.2.10 IMS Status

The following expectations must be met by NADIF Client in order to query the IMS status:

1. NADIF Client must include the header file `nadif_data_services.h` to access this feature.
2. NADIF Client is expected to use `NADIF_data_get_ims_status()` to query IMS status.

IMS status is also reported by the unsolicited notification `NADIF_FEAT_STATUS_IMS_SERVICES_STATUS` whenever the IMS status is changed.

3.7.1.2.11 Last Rejection Cause

NADIF provides means to query upto last 5 network rejection cause codes since modem power cycle. NADIF Client is expected use NADIF_net_get_last_reject_causes`()` to request for upto last 5 rejection cause received from the network.

3.7.1.2.12 Set High RF Capability

NADIF Client is expected to use NADIF_network_set_high_capability() to set which SIM subscription access the high RF capability stack (access to all RATs).

3.7.1.2.13 Get High RF Capability

NADIF Client is expected to use NADIF_network_get_high_capability() to get which SIM subscription access the high RF capability stack (access to all RATs).

3.7.1.2.14 High RF Capability Status

NADIF Client is expected to be notified through the unsolicited notification NADIF_FEAT_STATUS_NETWORK_HIGH_CAPABILITY whenever the status of high RF capability configuration is changed.

3.7.1.3 Extending Platform Functionality

Not Applicable

3.7.2 Setup

3.7.2.1 Tools/Equipment Needed

Not Applicable

3.7.2.2 Setup Instructions

Not Applicable

3.7.2.3 Setup Troubleshooting

Not Applicable

3.7.3 Usage

3.7.3.1 Basic Instructions

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.7.3.2 Detailed Instructions

3.7.3.2.1 Network Registration Setting

API/Type Description	Doxygen Path
Structure of Registration Data.	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_SET_REG_RQST**
API to set registration	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_net_set_registration()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows the NADIF Client to set the network registration mode to automatic or manual selection.

The automatic mode, `NADIF_REG_AUTO`, allows performing the network registration using all Radio Access Technologies (GSM, WCDMA, LTE etc.).

The manual mode, `NADIF_REG_MANUAL`, must specify the preferred Radio Access Technologies to perform the registration, by the `net_select` parameter, which is a bitmask to allow multiple RATs.

The manual mode also allows setting a preferred band by `band_select` parameter, if no change is needed, set it to 0 (`NADIF_BAND_UNKNOWN`). See the doxygen path above for more details.

NOTES:

1. *The parameters `net_select` and `band_select` do not have any effect on automatic mode.*
2. *It is recommended to leave the registration mode as automatic, manual mode may affect the performance of the network registration.*

3.7.3.2.2 Network Registration Status

API/Type Description	Doxygen Path
Structure of Network Type	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_NETWORK_TYPE**
API to query registration status	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_net_get_registration_status()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_GET_REG_STATUS_RESP**

It is recommended that the NADIF Client monitor the network registry through this API and/or the unsolicited notification `NADIF_FEAT_STATUS_NETWORK`. Network status gives relevant information about the network where the modem is registered, such as the current registration status for voice and packet, type of network technology

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

registered, Mobile Country Code, Mobile Network Code, Data Capability, network active band, network active channel, roaming information, etc

The unsolicited notification `NADIF_FEAT_STATUS_NETWORK` is sent whenever a network registration status is changed, the `status_change_flag` parameter in `NADIF_REG_STATUS_DATA` is a bitmask to inform that one (or more) status has changed.

The API `NADIF_net_get_registration_status()` allows NADIF Client to query the registration status of any network technology (GSM, WCDMA, LTE, etc.) by informing the parameter `NADIF_NETWORK_TYPE`.

The Network Registration Status notification reports the current status of the network registration (i.e., Registered, Not Registered, Limited and Roaming) for voice switched (CS) and packet switched (PS) individually.

When the registration status is `NADIF_REG_STAT_REGISTERED` or `NADIF_REG_STAT_REGISTERED_ROAMING`, the LAC, CID, MNC, MCC, network active band, and network active channel will also be reported if available.

Signal Quality

It is recommended that NADIF Client monitor the signal quality to verify that the modem is able to register on the network. The minimum signal strength required may vary depending on the sensitivity of the hardware.

Network monitoring

The NADIF client can monitor the registered network by constantly checking the MNC and MCC. Changes in the MNC and MCC may indicate that the network registration has changed, this is a typical use case for border areas.

Registration Rejected

In case of network registration denied or deregistration, the reject reason will be reported if available by the parameter `reject_cause`. Reject reason values are specified in 3GPP TS 24.008 Sections 10.5.3.6 and 10.5.5.14, and 3GPP TS 24.301 Section 9.9.3.9.

Network registration rejected call can wake up the application processor while it is suspended.

3.7.3.2.3 Last Registered Network Info

API/Type Description	Doxygen Path

API to query the last registered network info.	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Network`
	`|-> NADIF_net_get_last_registered_network_info()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Network`
	`|-> NADIF_NET_GET_LAST_REGISTERED_INFO_RESP`

This API reports the MMC and MNC from the last registered network. This information might be useful in boot-up time, informing very early to the NADIF Client which network the modem will try to register.

NOTE: If the notification of this API is `NADIF_RESULT_SUCCESS` and the `plmn_info` element of `NADIF_NET_GET_LAST_REGISTERED_INFO_RESP` in the notification contains default values, this indicates that the modem has not yet successfully registered with a cellular network.

3.7.3.2.4 Neighboring Cells

API/Type Description	Doxygen Path

API to query Neighboring Cells	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Network`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	`|-> NADIF_net_get_nb_cells_number()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Network`
	`|-> NADIF_GET_NB_CELLS_NUMBER_RESP`

The notification of this API or the unsolicited notification `NADIF_FEAT_STATUS_NB_CELLS_NUM` report the number of neighboring cells of the network which is registered.

3.7.3.2.5 Signal Quality

API/Type Description	Doxygen Path

API to query Signal Quality	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Network`
	`|-> NADIF_net_get_signal_quality()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Network`
	`|-> NADIF_GET_SIG_QUAL_RESP`

The notification of this API or the unsolicited notification `NADIF_FEAT_STATUS_SIGNAL_QUALITY` report the current RSCP and RSRP. It also informs a normalized RSSI, raw RSSI, BER and RSCP/RSRP for the registered RATs.

3.7.3.2.6 Network Scan

API/Type Description	Doxygen Path

Structure of Network Scan Request	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Network`
	`|-> NADIF_NET_SCAN_RQST`

API to request Network scan	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Network`
	`|-> NADIF_net_perform_scan()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Network`
	`|-> NADIF_NET_SCAN_RESP`

This API allows NADIF client to perform a scan on the visible networks according to the RAT informed in `NADIF_NET_SCAN_RQST`. The notification of this API reports a list with information of each network found, such as MCC, MNC, network status and description.

NOTE: Network scan may take a few seconds to report the result.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.7.3.2.7 SPY Mode

API/Type Description	Doxygen Path
Structure of Network SPY Request	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_NET_SET_SPY_MODE_RQST**
API to set SPY Mode	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_net_set_spy_mode()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF client to enable or disable the spy mode. Spy mode is typically used by the eCall feature, where the modem is only registered during an emergency call. When Spy Mode is enabled it keeps monitoring serving cell and all surrounding cells. Nevertheless, network registration can be achieved in a very short time if an emergency call needs to be performed.

3.7.3.2.8 Cellular DRX

API/Type Description	Doxygen Path
API to query Cellular DRX	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_net_get_drx()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_GET_DRX_CYCLE_RESP**

Cellular DRX allows the modem to periodically switch off its receiver for some time before listening again to the control channel to check if the network has a new data for the modem. A long DRX cycle is much more power efficient, but it increases latency time for receiving an incoming data.

The notification of this API reports the Cellular DRX cycle coefficient negotiated with the network.

3.7.3.2.9 IMS Setting

API/Type Description	Doxygen Path
Structure of IMS setting Request	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_NET_IMS_SET_CONFIG_RQST**
API to set IMS	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_ims_set_config()**
Notification data type of API	Modules

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to enable or disable the IMS support dynamically.

When LTE is the registered network and IMS is disabled, all voice calls are circuit-switched calls. It means, when a call is initiated (incoming or outgoing), the modem will switch to WCDMA (or GSM) service to complete the voice call.

IMS support has to be enabled to allow VoLTE (voice over LTE), in this case, the IMS VoIP service also needs to be available.

3.7.3.2.10 IMS Status

API/Type Description	Doxygen Path
API to query IMS status	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_data_get_ims_status()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Data Services |-> **NADIF_GET_IMS_STATUS_RESP**

The notification of this API or the unsolicited notification `NADIF_FEAT_STATUS_IMS_SERVICES_STATUS` report the current status of the IMS service. It also contains whether the IMS service is registered or not, and an individual status of each service over IMS, such as SMS, VoIP, Video Telephony, User terminal and Video Sharing.

IMS status is useful to check whether the VoLTE (Voice over LTE) is available, in this case, VoIP service will be `NADIF_SERVICE_AVAILABLE`.

3.7.3.2.11 Last Rejection Cause

API/Type Description	Doxygen Path
API to query Last reject cause	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_net_get_last_reject_causes()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_GET_LAST_REJECT_CAUSE_RESP**

This API allows the NADIF Client to query the upto last 5 rejected causes received from the network.

The rejected causes will be cleared after any modem power cycle that occur, including subsystem controlled modem reboot since it is expected that modem will perform a fresh network scan and registration process after a power cycle.

3.7.3.2.12 Set High RF Capability

API/Type Description	Doxygen Path
Enum of Subscription ID	Modules

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|>> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_SIM_SUBSCRIPTION_ID**
API to set High RF capability	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_network_set_high_capability ()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows the NADIF Client to set which SIM subscription access the high RF capability using API NADIF_network_set_high_capability which will be by default set to primary subscription (NADIF_SIM_SUBSCRIPTION_0).

The low rf capability will be automatically assigned to another sim which is not high RF capability configured.
Notes:

- High capability means the RF capability is able to attach to full network stack 5G,LTE,WCDMA,GSM.
- Low capability means the RF capability is able to attach to limited network stack LTE and GSM.
- This API is applicable for only Dual SIM supported device and if not it returns failure.

3.7.3.2.13 Get High RF Capability

API/Type Description	Doxygen Path
API to query High RF capability	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_network_get_high_capability ()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_GET_NETWORK_HIGH_CAPABILITY_RESP**

This API allows NADIF Client to get which SIM subscription access the high RF capability using API NADIF_network_get_high_capability for which we get the response through NADIF_GET_NETWORK_HIGH_CAPABILITY_RESP.

3.7.3.2.14 High RF Capability Indication

API/Type Description	Doxygen Path
Structure of notification data	Modules |-> Modem |-> NADIF |-> Network |-> **NADIF_FEAT_STATUS_NETWORK_HIGH_CAPABILITY**

The unsolicited notification `NADIF_FEAT_STATUS_NETWORK_HIGH_CAPABILITY` provides the status of high RF capability configuration change.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.7.3.3 Testing/Error Reporting

Not Applicable

3.8 On-Device Logging

3.8.1 Overview

3.8.1.1 Platform Content Overview

On-device logging is a feature which collects diagnostics data from the modem to store it as .qmdl files in the app processor's filesystem for modem behavior analysis. The log data is recorded using a log file rotation mechanism, that is, a new file is created when the size limit of the file is reached. When the maximum number of files and the size limit are reached the oldest file is deleted and then a new file is created.

On-device logging can be configured to collect specific diagnostic log packets through a file containing a log mask.

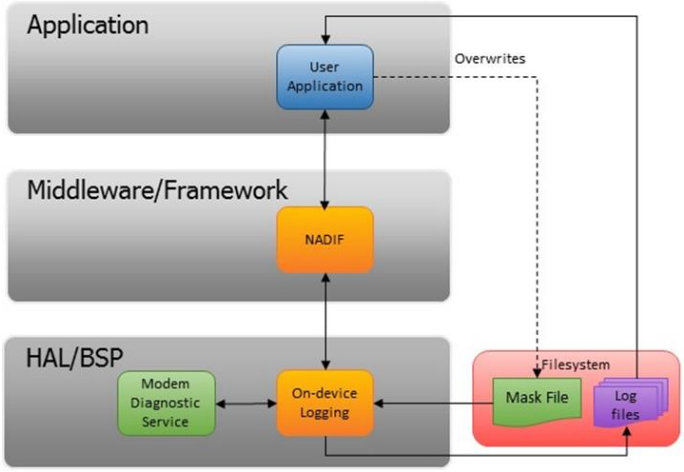


Figure 10: On-Device Logging context diagram

NOTE: On Device Logging feature is only available when Continental HAL is used.

3.8.1.2 Product Expectations

The following expectations must be met by NADIF Client in order to use the on-device logging feature as designed:

1. NADIF Client must include the header file `nadif_on_device_logging.h` to access the APIs.
2. On-device logging is disabled at every power-up cycle.
3. NADIF Client is expected to use `NADIF_logging_mode()` to enable or disable the on-device logging feature.
4. NADIF Client is expected to use `NADIF_get_on_device_logging_parameteres()` to query the settings for on-device logging.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.8.1.3 Extending Platform Functionality

3.8.1.3.1 Log Mask

The log mask is the file which contains the configuration that tells on-device logging the specific diagnostic log packets that will be collected. The device already has a mask loaded, ready to use, containing the default mask, which is useful to investigate most of the issues.

The user can overwrite the default mask that is located in:

/data/on_device_logging/on_device_logging_mask.cfg

NOTE: Telematics Platform has to be consulted to create a custom log mask.

3.8.2 Setup

3.8.2.1 Tools/Equipment Needed

Not Applicable

3.8.2.2 Setup Instructions

Not Applicable

3.8.2.3 Setup Troubleshooting

Not Applicable

3.8.3 Usage

3.8.3.1 Basic Instructions

Not applicable

3.8.3.2 Detailed Instructions

3.8.3.2.1 Enable/disable on-device logging

API/Type Description	Doxygen Path

Structure of logging mode Request	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> on device logging`
	`  |-> NADIF_LOGGING_MODEM_RQST_TYPE`

API to set the logging mode	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> on device logging`
	`  |-> NADIF_logging_mode()`

Notification data type of API	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> NADIF Common Types`
	`  |-> NADIF_RESULT`

This API allows NADIF Client to enable or disable the on-device logging. Note that file system is the only mode supported by on-device logging, so NADIF Client has to enter the parameter `logging_modes` as `NADIF_CUST_DM_LOGGING_FILESYSTEM` to enable the logging. It is also mandatory to enter the parameters of *number of log files* and *maximum size of the log file*.

While on-device is running, .qmdl files are created when the maximum size is reached.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

In the following example, the number of log files is set to 5 and the *maximum size of the log file* is 1000000.

```
[root@mdm9x28 /]#ls -la /data/log
ls -la /data/log
drwxr-xr-x    3 root     root           736 Oct 11 00:38 .
drwxrwxr-x   12 root     data          1384 Oct 11 00:25 ..
drwxrwxrwx    2 root     root           240 Oct 11 00:27 cores
-rw-r--r--    1 root     root      18113623 Oct 11 00:39 messages
-rw-r--r--    1 root     root        999997 Oct 11 00:35 qxdm_log.0000000000.qmdl
-rw-r--r--    1 root     root        999992 Oct 11 00:36 qxdm_log.0000000001.qmdl
-rw-r--r--    1 root     root        999985 Oct 11 00:37 qxdm_log.0000000002.qmdl
-rw-r--r--    1 root     root        999987 Oct 11 00:38 qxdm_log.0000000003.qmdl
-rw-r--r--    1 root     root        876761 Oct 11 00:39 qxdm_log.0000000004.qmdl
```

Figure 11: New log files are created when the size limit is reached

```
[root@mdm9x28 /]#ls -la /data/log
ls -la /data/log
drwxr-xr-x    3 root     root           736 Oct 11 00:40 .
drwxrwxr-x   12 root     data          1384 Oct 11 00:25 ..
drwxrwxrwx    2 root     root           240 Oct 11 00:27 cores
-rw-r--r--    1 root     root      18162177 Oct 11 00:41 messages
-rw-r--r--    1 root     root        999985 Oct 11 00:37 qxdm_log.0000000002.qmdl
-rw-r--r--    1 root     root        999987 Oct 11 00:38 qxdm_log.0000000003.qmdl
-rw-r--r--    1 root     root        999954 Oct 11 00:39 qxdm_log.0000000004.qmdl
-rw-r--r--    1 root     root        999989 Oct 11 00:40 qxdm_log.0000000005.qmdl
-rw-r--r--    1 root     root        731706 Oct 11 00:41 qxdm_log.0000000006.qmdl
```

Figure 12: Oldest files, with indexes 0 and 1, were deleted to store new logs 5 and 6

3.8.3.2.2 Log Files

The logs will be stored in:

- /var/log/
  - Symbolic link to logs.
- /data/log/
  - Real path to logs

The logs will be stored with the name format *qxdm_log.0000000000.qmdl*, where the number between *qxdm_log.* and *.qmdl* represents the number of each log file, this value increments when a new log is created. The lowest number is the oldest file.
The log files can be downloaded using adb pull:

```
adb pull  <Origin> <Destination>
```

e.g.

```
adb pull /data/log/qxdm_log.0000000099.qmdl  C:\Logs
```

3.8.3.2.3 On-device logging settings

API/Type Description	Doxygen Path
API to query the logging mode	Modules |-> Modem |-> NADIF |-> on device logging |-> **NADIF_get_on_device_logging_parameters()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> on device logging |-> **NADIF_GET_LOGGING_PARAMETERS_RESP**

The notification of this API returns the settings for on-device logging, such as mode, number of log files and maximum size of the log file.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.8.3.2.4 Device Disk Space Consideration

If the device only has 100 MB of available space, the user should not allow on-device logging to consume all the space since other applications save information in our system. As result, it will cause a severe system malfunction since other applications also store information in the device.

3.8.3.3 Testing/Error Reporting

Not applicable

3.9 SIM

3.9.1 Overview

3.9.1.1 Platform Content Overview

This section describes features supported by NADIF related to Subscriber Identity Module (SIM) or eUICC connected to the modem. See the following standard features supported by NADIF:

- PIN management – request to unlock the SIM or unblock the PIN.
- SIM status – request for the current status of the SIM.
- SIM information – retrieving SIM contents like phone number, IMS, serial number and others.
- SIM power settings – request to power up or down the SIM.
- APDU commands – method for sending APDU messages directly to the SIM to access specifics applet.
- SIM set eCall only emulation- Request to enable or disable eCall only emulation.
- SIM get eCall only emulation- Query the current status of eCall only emulation.
- Read/write SIM Transparent EF – read/write access to any transparent elementary file

NADIF also has a feature set designed for eUICC to support multiple profiles, such as:

- Profile information – retrieving information from the profiles.
- ERA Local profile switch – request to switch to ERA or regular profile.
- Refresh operation (allow/block) – request to allow or block SIM updates, including profile updates and profile switch.

Profile is a group of data, files, and applications on an eUICC that is used to obtain access to a single mobile network operator (i.e., cellular service provider).

Only one profile on an eUICC can be active at a time. Dual standby (DSDS) is not supported

NADIF Client is notified when there is refresh operation, such as profile file change, profile switch or refresh failure.

A refresh operation triggered by the network can wake up the application processor while it is suspended.

3.9.1.2 Product Expectations

NADIF Client must include the header file `nadif_sim.h` to access any API described in this section.

If Hardware variant supports Dual SIM, SIM slot parameter present in any of the APIs described in this section will be ignored.

3.9.1.2.1 Set SIM Power

NADIF Client is expected to use `NADIF_sim_set_power()` to power up or down the SIM.

3.9.1.2.2 SIM State

NADIF Client is expected to use `NADIF_sim_get_state()` to query the current state of the SIM.

NADIF Client is also expected to be notified through the unsolicited notification `NADIF_FEAT_STATUS_SIM` whenever the status of the SIM is changed.

3.9.1.2.3 PIN management

NADIF Client is expected to use `NADIF_sim_set_state()` to manage operations related to the PIN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.9.1.2.4 SIM Information

NADIF Client is expected to use `NADIF_sim_get_info()` to obtain various types of information of the SIM.

3.9.1.2.5 SIM APDU

NADIF Client is expected to use `NADIF_sim_send_apdu()` to send APDU commands to the SIM.

3.9.1.2.6 Read SIM/UICC transparent Elementary File

NADIF Client is expected to use NADIF_sim_read_transparent_file() to have read access to any transparent elementary file in the SIM card.

3.9.1.2.7 Write SIM/UICC transparent Elementary File

NADIF Client is expected to use NADIF_sim_write_transparent_file() to have write access to any transparent elementary file in the SIM card.

3.9.1.2.8 Profile Information

NADIF Client is expected to use `NADIF_sim_get_profile_info()` to query the SIM Profile information.

3.9.1.2.9 Current Profile Information

NADIF Client is expected to use `NADIF_sim_get_current_profile_info()` to query the current Profile information.

3.9.1.2.10 ERA Local Profile Switch

NADIF Client is expected to use `NADIF_sim_era_switch_profile()` to switch the SIM profile to either ERA (Russia emergency call) or regular profile.

3.9.1.2.11 Get Remote Profile Switch Lock Configuration

NADIF Client is expected to use `NADIF_sim_get_remote_profile_switch_lock_config()` to get the remote profile switch lock configuration.

Note: This API of remote sim lock is only supported for Idemia SIM cards, if other brand of SIM is used it will not work.

3.9.1.2.12 Set Remote Profile Switch Lock Configuration

NADIF Client is expected to use `NADIF_sim_set_remote_profile_switch_lock_config()` to set the remote profile switch lock configuration.

Note: This API of remote sim lock is only supported for Idemia SIM cards, if other brand of SIM is used it will not work.

3.9.1.2.13 Refresh Allow

NADIF Client is expected to use `NADIF_sim_refresh_allow()` to allow or block the SIM updates.

3.9.1.2.14 Refresh Flag Status

NADIF Client is expected to use `NADIF_sim_get_refresh_flag_status()` to query the SIM refresh flag status.

3.9.1.2.15 Refresh Notifications

NADIF client is expected to receive one or more of the following unsolicited notifications during a refresh operation:

1. NADIF_FEAT_STATUS_SIM_REFRESH_IND_NOTIFICATION
2. NADIF_FEAT_STATUS_SIM_PROFILE_SWITCH_NOTIFICATION
3. NADIF_FEAT_STATUS_SIM_FILE_CHANGE_NOTIFICATION
4. NADIF_FEAT_STATUS_SIM_REFRESH_FAILURE_NOTIFICATION

3.9.1.2.16 Physical sim slot selection

NADIF client is expected to use `NADIF_sim_set_physical_slot()` to switch the physical sim slot.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

Note: The switch of the physical sim slot does not take effect until a modem power cycle.

3.9.1.2.17 Get active physical sim slot

NADIF client is expected to use `NADIF_sim_get_active_physical_slot()` to get the physical sim slot that is currently active.

Note this is a complementary API to `NADIF_sim_set_physical_slot()` since it's effect needs a power cycle to be effective.

If this API is called when the SIM slot has been changed, but the power cycle has not been performed yet. Then the active physical slot will defer to the physical slot set.

3.9.1.2.18 SIM set eCall only emulation

The following expectations must be met by NADIF Client in order to request for ecall only emulation:

1. Sim profile configuration should be eCall Plus (Commercial profile with eCall), since eCall only emulation cannot be enabled if the SIM profile is already eCall Only.
2. Sim should be in Ready state.
3. NADIF Client is expected to use `NADIF_sim_set_ecall_only_emulation()` to set the status of eCall only emulation.
4. This API will trigger the SIM power cycle. NADIF client is expected to wait for sim ready status before calling any Modem-MNO related service that require MNO connection

3.9.1.2.19 SIM get eCall only emulation

The following expectations must be met by NADIF Client in order to query the current ecall emulation status:

1. Sim profile configuration should be eCall Plus (Commercial profile with eCall), since eCall only emulation would not be enabled if the SIM profile is already eCall Only.
2. Client is expected to use `NADIF_sim_get_ecall_only_emulation()` to get the status of eCall only emulation.

The status of get ecall only emulation request is notified by the `NADIF_SIM_GET_ECALL_ONLY_EMU_RESP notification`.

3.9.1.3 Extending Platform Functionality

Not applicable

3.9.2 Setup

3.9.2.1 Tools/Equipment Needed

Not applicable

3.9.2.2 Setup Instructions

Not applicable

3.9.2.3 Setup Troubleshooting

Not applicable

3.9.3 Usage

3.9.3.1 Basic Instructions

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.9.3.2 Detailed Instructions

3.9.3.2.1 Set SIM Power

API/Type Description	Doxygen Path
Structure of setting the SIM power.	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_SIM_SET_POWER_RQST**
API to set SIM power	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_set_power()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to power up or down the SIM. The SIM is powered by default at modem power up.

The SIM power down can be performed when the modem is switched off or when it is set to Airplane mode.

3.9.3.2.2 SIM State

API/Type Description	Doxygen Path
API to query SIM state	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_get_state()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> SMS |-> **NADIF_SIM_GET_STATE_RESP**

The notification of this API or the unsolicited notification `NADIF_FEAT_STATUS_SIM` return the current status of the SIM, i.e., PIN locked, PIN unlocked, PIN blocked, ready, error, etc.

If the SIM state is PIN locked or PIN blocked, NADIF Client can use the API for PIN management to unlock SIM or unblock the PIN.

If the SIM state is an error, the element `sim_error_status`, which is part of `NADIF_SIM_GET_STATE_RESP`, indicates the type of the error.

SIM must be in `NADIF_SIM_READY` to subscribe to the network services.

If the SIM status is busy, the element sim_busy_status, which is part of NADIF_SIM_GET_STATE_RESP, will be set to 1 (sim_busy_status = 1). SIM status busy indicates that the SIM drivers will not respond to APDU commands. The normal operation resumes when sim_busy_status is cleared(sim_busy_status= 0). To know the SIM status busy client can query the API NADIF_sim_get_state or check for the boolean flag sim_busy_status reference to unsolicited notification NADIF_FEAT_STATUS_SIM.

Note: If sim busy status persists for more than 180seconds retry mechanism should be triggered by performing SIM power cycle using API NADIF_sim_set_power().If the SIM doesn't recover perform modem power cycle.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

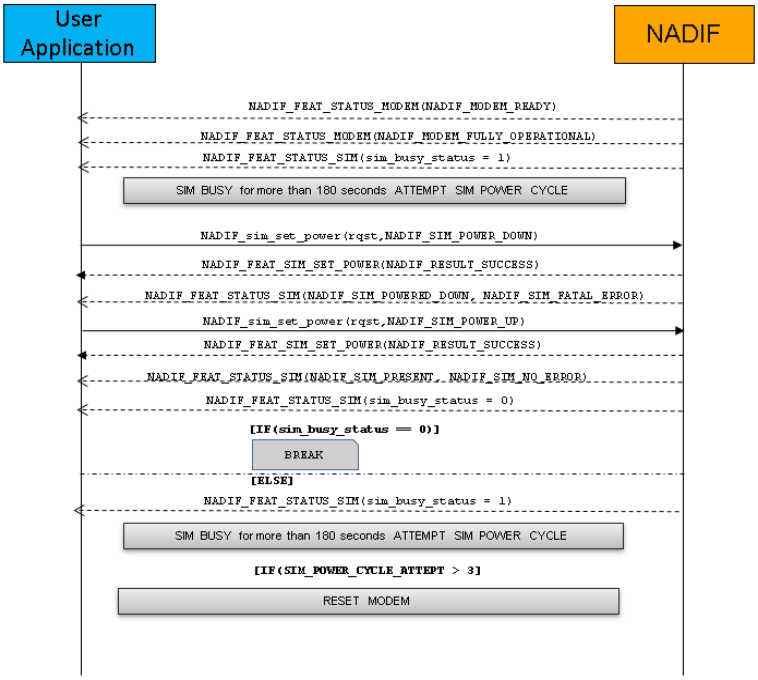


Figure 13: SIM BUSY STATUS Recovery sequence

3.9.3.2.3 PIN management

API/Type Description	Doxygen Path
Structure of requesting the state of the SIM.	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_SIM_SET_STATE_RQST**
API to set SIM state	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_set_state()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

The API `NADIF_sim_set_state()` manages the operations related to PIN and PUK of the SIM, see the main use cases supported by this API based on the current SIM status (`NADIF_SIM_STATUS`):

Use Case 1: `NADIF_SIM_PIN_LOCKED`

To unlock the SIM card, NADIF Client must enter *NADIF_SIM_UNLOCK_PIN_1 or NADIF_SIM_UNLOCK_PIN_2* in the parameter `set_state_option` and the valid PIN code in the parameter *pin*.

Use Case 2: `NADIF_SIM_PIN_BLOCKED`

To unblock the PIN1 or PIN2, NADIF Client must enter *NADIF_SIM_UNBLOCK_PIN_1 or NADIF_SIM_UNBLOCK_PIN_2* in the parameter `set_state_option`, a new PIN code in the parameter *pin,* and the valid PUK code in the parameter *puk*.

This API can be used to lock the SIM by entering the option `NADIF_SIM_ENABLE_PIN_1` or `NADIF_SIM_ENABLE_PIN_2` in the parameter `set_state_option`, in this case, the SIM PIN will be requested at modem power up.

To disable the SIM PIN request at modem power up, the user must enter `NADIF_SIM_DISABLE_PIN_1` or `NADIF_SIM_ENABLE_PIN_2` in the parameter `set_state_option`.

NOTES:

1. *The `pin` parameter must be entered for all options (`set_state_option`) used by this API.*
2. *PIN Management API is not typically used in telematic products, since the PIN is usually kept unlocked.*

3.9.3.2.4 SIM Information

API/Type Description	Doxygen Path

Enumeration of SIM info option	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> SIM`
	`  |-> NADIF_SIM_INFO_OPTION`

API to query SIM info	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> SIM`
	`  |-> NADIF_sim_get_info()`

Notification data type of API	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> SMS`
	`  |-> NADIF_SIM_GET_INFO_RESP`

This API allows the NADIF Client to query various types of information from the SIM.

The notification of this API returns the information according to the option selected by the parameter `sim_info_option`, which is a bitmask to allow the request for multiple options. Check below which options are supported:

- SIM status
- SIM phone number
- SIM IMS
- SIM serial number
- eUICC identifier
- eUICC Service Provider Name

Note that some of this information can also be requested using the APIs described in the Modem Status section.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.9.3.2.5 SIM APDU

API/Type Description	Doxygen Path

API to send APDU	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> SIM`
	`  |-> NADIF_sim_send_apdu()`

Notification data type of API	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> SMS`
	`  |-> NADIF_SIM_SEND_APDU_RESP`

This API allows NADIF Client to send APDU commands to the SIM/eUICC. The notification of this API provides the APDU response.

The APDU is an array of bytes that is passed to the SIM/eUICC directly without any manipulation/formatting by NADIF or the modem. NADIF Client is responsible for creating and appropriately formatting APDU message.

The APDU response from the SIM/eUICC is not parsed by NADIF or modem other than extracting basic status information (i.e., status word 1 and status word 2). Parsing of response contents must be done by the calling application.

To send an APDU message, NADIF Client must enter the array of bytes for Application ID (AID) and the APDU command.

Application ID

The application ID must be specified according to the APDU command that it belongs to. The SIM/eUICC specification should be consulted to check the available applications.

See some examples of application ID for eUICC, according to SGP.02 v3.2 – Remote Provisioning Architecture for Embedded UICC Technical Specification:

Application Name	AID
ISD-R	A0 00 00 05 59 10 10 FF FF FF FF 89 00 00 01 00
ECASD	A0 00 00 05 59 10 10 FF FF FF FF 89 00 00 02 00
ISD-P Executable Load File	A0 00 00 05 59 10 10 FF FF FF FF 89 00 00 0D 00
ISD-P Executable Module	A0 00 00 05 59 10 10 FF FF FF FF 89 00 00 0E 00

Table 1: eUICC Application ID

NOTE: NADIF client has to specify the Application ID (AID) each time the `NADIF_sim_send_apdu()` API is invoked.

APDU Command

The APDU message command consists of the following fields:

Field Name	Description
CLA	Instruction class - indicates the type of command (mandatory)
INS	Instruction code - indicates the specific command (mandatory)
P1	Instruction parameter(1) for the command (mandatory)
P2	Instruction parameter(2) for the command (mandatory)
Lc	Number of bytes (Nc)
Nc	Command data
Le	Maximum number of response bytes expected (Nr)

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

Table 2: APDU Command

Field Name	Description
Nr	Response data
SW1-SW2	Command processing status

Table 3: APDU Response

Example

The following example shows the array of bytes to get the Issuer Identification Number (IIN) from ECASD:

AID (ECASD): A0 00 00 05 59 10 10 FF FF FF FF 89 00 00 02 00

APDU Command to get IIN: 00 CA 00 42

The response includes SW1 and SW2 status word, in this case, 90 00 indicates success.

ADPU Response: 00 63 68 56 **90 00**

3.9.3.2.6 Read SIM/UICC transparent Elementary file

API/Type Description	Doxygen Path

API to query the SIM Profile	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> SIM`
	`|->NADIF_sim_read_transparent_file()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> SIM`
	`|->NADIF_SIM_READ_TRANSPARENT_FILE_RESP`

This API allows NADIF Client to have read access to any elementary transparent file in the SIM card.

- `file_id_info` - File Identifier info all fields must be specified, `file_id_info` contain `file_id`, `file_path_len` and `file_path`. `file_path` must be the complete path of the file, which is a sequence block of 2 bytes(e.g., 0x3F00 0x7FFF - `file_path`[0] = 0x00,`file_path`[1] = 0x3F `file_path`[2] = 0xFF,`file_path`[3] = 0x7F).
- `offset` - Offset for the read operation, if offset is 0, file will be read from the beginning
- `read_data_len` - Length of the content to be read from the SIM card Transparent File. The value 0 is used to read the complete file. The Maximum read length limits to 4096

Example

The following example shows how to fill-up the file_id_info to get the FPLMN list from SIM:

```
file_id_info.file_id = 0x6F7B;
file_id_info.file_path_len = 4;
file_id_info.file_path[0] = '\x00';
file_id_info.file_path[1] = '\x3f';
file_id_info.file_path[2] = '\xff';
file_id_info.file_path[3] = '\x7f';
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.9.3.2.7 Write SIM/UICC transparent Elementary file

API/Type Description	Doxygen Path
API to update transparent elementary file	Modules |-> Modem |-> NADIF |-> SIM |->**NADIF_sim_read_transparent_file()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> SIM |->**NADIF_SIM_WRITE_TRANSPARENT_FILE_RESP**

This API allows NADIF Client to have write access to any elementary transparent file in the SIM card and access by the path.

- `file_id_info` - - File Identifier info all fields must be specified, `file_id_info` contain `file_id`, `file_path_len` and `file_path`. `file_path` must be the complete path of the file, which is a sequence block of 2 bytes(e.g., 0x3F00 0x7FFF - `file_path`[0] = 0x00,`file_path`[1] = 0x3F `file_path`[2] = 0xFF,`file_path`[3] = 0x7F).
- `offset` – Offset for the write operation, if offset is 0, file will be written from the beginning
- `write_data_len` - Length of the content to be written to the SIM card Transparent elementary File. The value 0 is used to write the complete file. The Maximum write length limits to 4096
- `write_data` - data that to be written to the SIM transparent elementary file

3.9.3.2.8 Profile Information

API/Type Description	Doxygen Path
API to query the SIM Profile	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_get_profile_info()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> SMS |-> **NADIF_SIM_GET_PROFILE_RESP**

The notification of this API returns a list of all profiles available on the SIM. Each element of the list contains the following items:

- Profile ID.
- Profile type, i.e., regular, emergency or unknown.

This API also provides which profile is active.

3.9.3.2.9 Current Profile Information

API/Type Description	Doxygen Path
API to query the current SIM Profile	Modules |-> Modem |-> NADIF |-> SIM

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|-> **NADIF_sim_get_current_profile_info()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> SMS |-> **NADIF_GET_SIM_CURRENT_PROFILE_INFO_RESP**

The notification of this API returns profile information from the active SIM. See the items reported by this API:

- SIM phone number
- SIM IMS
- SIM serial number
- SIM Carrier Name

3.9.3.2.10 ERA Local Profile Switch

API/Type Description	Doxygen Path
API to switch SIM Profile	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_era_switch_profile()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to switch between a single non-ERA (commercial) profile and a designated ERA emergency profile on an eUICC for supporting Russia emergency call (ERA).

Note that this API is used by PSAP Call to support ERA-GLONASS feature, switching to Emergency profile.

3.9.3.2.11 Get Remote Profile Switch Lock Configuration

API/Type Description	Doxygen Path
API to get the Remote Profile Switch lock configuration	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_get_remote_profile_switch_lock_config()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_SIM_GET_REMOTE_PROFILE_SWITCH_LOCK_CONFIG_RESP**

The notification of this API provides the remote profile switch lock status (locked/unlocked).

NOTE: This API of remote sim lock is only supported for Idemia SIM cards, if other brand of SIM is used it will not work.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.9.3.2.12 Set Remote Profile Switch Lock Configuration

API/Type Description	Doxygen Path
API to set the Remote Profile Switch lock configuration	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_set_remote_profile_switch_lock_config()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to lock/unlock remote profile switch. When remote profile switch is locked, the SIM card will reject/block any remote requests for a SIM profile switch (e.g., request from a SIM Subscription Manager). When remote profile switch is unlocked, remote requests for a SIM profile switch are allowed. Locking out remote requests for SIM profile switch is recommended when performing critical operations (e.g., emergency calls, software update).

NOTE: This API of remote sim lock is only supported for Idemia SIM cards, if other brand of SIM is used it will not work.

3.9.3.2.13 Refresh Allow

API/Type Description	Doxygen Path
API to set the refresh flag	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_refresh_allow()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to enable or disable the SIM refresh operation, including profile updates and profile switch.

NOTES:

1. *As NADIF supports multiple clients, there is a logic to ensure that the Refresh operation will be blocked if any of the NADIF Clients requests the prohibition.*
2. *All NADIF clients allow the Refresh operation by default, so this API should be invoked if the NADIF client needs to block the refresh operation.*
3. *The refresh operation will be pending until unblocked by all NADIF Clients or modem power cycle.*

3.9.3.2.14 Refresh Flag Status

API/Type Description	Doxygen Path
API to query the refresh flag status	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_get_refresh_flag_status()**
Notification data type of API	Modules

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|> Modem |-> NADIF |-> SIM |-> NADIF_SIM_GET_REFRESH_OK_FLAG_RESP

The notification of this API reports whether the SIM Refresh operation is allowed or not. This API is useful for checking if another NADIF client is blocking the Refresh operation.

3.9.3.2.15 Refresh Notifications

The following unsolicited notifications can be received during a refresh operation:

NADIF_FEAT_STATUS_SIM_REFRESH_IND_NOTIFICATION

This unsolicited notification is reported whenever the refresh operation is completed due to eUICC/SIM initialization, eUICC/SIM app reset or network session reset.

NADIF_FEAT_STATUS_SIM_PROFILE_SWITCH_NOTIFICATION

This unsolicited notification is reported whenever the refresh operation is completed due to eUICC/SIM reset caused by profile switch.

NADIF_FEAT_STATUS_SIM_FILE_CHANGE_NOTIFICATION

This unsolicited notification is reported whenever a parameter included in the profile information changes in the current eUICC/SIM profile.

NADIF_FEAT_STATUS_SIM_REFRESH_FAILURE_NOTIFICATION

This unsolicited notification is reported when a refresh operation/profile switch failure occurs during a SIM OTA profile update or local profile switch.

NOTE: These notifications are sent only if the refresh operation is allowed.

3.9.3.2.16 Physical sim slot selection

API/Type Description	Doxygen Path
API to set physical SIM slot	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_set_physical_slot ()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_FEAT_SET_PHYSICAL_SIM_SLOT**

This API allows NADIF Client to select the physical sim slot to which it is required to switch.

Note: This API requires a system reboot to take effect.

3.9.3.2.17 Get active physical sim slot

API/Type Description	Doxygen Path
API to query the active physical SIM slot	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_get_active_physical_slot ()**
Notification data type of API	Modules |-> Modem

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|> NADIF |-> SIM |-> **NADIF_FEAT_GET_ACTIVE_PHYSICAL_SIM_SLOT**

This API allows NADIF Client to query the current active physical sim slot .

If this API is called when the SIM slot has been changed, but the power cycle has not been performed yet. Then the active physical slot will defer to the physical slot set.

3.9.3.2.18 SIM set eCall only emulation

API/Type Description	Doxygen Path
API to enable or disable the eCall only emulation.	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_set_ecall_only_emulation ()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to enable or disable eCall only emulation thus emulating the eCall only behavior when the SIM profile is eCall Plus (Commercial profile with eCall). The API returns NADIF_RESULT_NO_EFFECT when the SIM is already eCall only profile and NADIF_RESULT_SIM_NOT_INITIALIZED when the SIM is not ready.

This emulation is persistent across power cycles and the API operates according to the following table:

SIM Configuration	eCall Emulation	Current Operation Mode/ Behavior
eCall Only	OFF	eCall Only
eCall Only	ON	eCall Only
eCall Plus	OFF	eCall Plus
eCall Plus	ON	eCall Only

Note: The behavior of the API is the same for ERA GLONASS and EU eCall, while the emulation is enabled the operation mode will be Emergency profile for ERA GLONASS and eCall only for EU eCall.

NADIF client can check the unsolicited notification NADIF_FEAT_STATUS_SIM or use the API NADIF_sim_get_state to check the sim ready status.

3.9.3.2.19 SIM get eCall only emulation

API/Type Description	Doxygen Path
API to get the current status of eCall only emulation.	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_get_ecall_only_emulation ()**
Notification data type of API	|-> Modem |-> NADIF |-> SIM |-> **NADIF_SIM_GET_ECALL_ONLY_EMU_RESP**

This API allows NADIF Client to get the current status of eCall only emulation. If the SIM profile is already eCall Only, in this case the status is returned as NADIF_SIM_ECALL_ONLY_EMULATION_NOT_ALLOWED.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.9.3.3 Testing/Error Reporting

Not applicable

3.10 SMS

3.10.1 Overview

3.10.1.1 Platform Content Overview

This section describes how to send, read and delete SMS messages. See the following features supported by NADIF:

- Messages limited to 140 bytes or 160 ASCII characters (packed into 140 bytes).
- Incoming SMS messages are stored on the internal memory of the modem and they are handled by index. Typically the modem can store 255 incoming messages, but this value may depend on the modem vendor. Outgoing SMS messages are typically not stored, so NADIF does not support them.
- An incoming SMS can wake up the application processor while it is suspended.
- Support for setting and querying the Service Center Address (SCA).
- SMS over IMS is supported by the APIs described in this section, whenever the IMS is registered and SMS service is available.
- Support for sending and reading received SMS PDU (raw SMS message including header)

3.10.1.2 Product Expectations

NADIF Client must include the header file `nadif_sms.h` to access any API described in this section.

3.10.1.2.1 SMS cell broadcast

The following expectations must be met by NADIF Client in order to setup BCM and receives it:

1. NADIF Client is expected to use NADIF_sms_set_broadcast_config_req to set the BCM configuration
2. NADIF Client is expected to use NADIF_sms_get_broadcast_config to get the BCM configuration
3. NADIF Client must have received SMS cell broadcast message from the network, if it is enabled via NADIF_sms_set_broadcast_config_req. Status id is 'NADIF_SMS_BROADCAST_MESSAGE'. Reported contents provided in "data" element of NADIF_SMS_DATA.
4. NADIF Client must have received SMS ETWS message from network, if it is enabled via NADIF_sms_set_broadcast_config_req. Status id is 'NADIF_SMS_ETWS_MESSAGE'. Reported contents provided in "etws_msg" of NADIF_SMS_ETWS_MESSAGE.

3.10.1.2.2 Send SMS

The following expectations must be met by NADIF Client in order to send an SMS as designed:

1. The modem must be registered on the cellular network
2. NADIF Client is expected to use `NADIF_sms_send()` to send an SMS.

3.10.1.2.3 Indication for SMS Delivery report

NADIF Client is expected to be notified through the unsolicited notification `NADIF_FEAT_STATUS_SMS` whenever an indication for the delivery report of the Mobile originated SMS is received.

3.10.1.2.4 Incoming SMS

NADIF Client is expected to be notified through the unsolicited notification `NADIF_FEAT_STATUS_SM` whenever an incoming SMS is received.

3.10.1.2.5 Read SMS

The following expectations must be met by NADIF Client in order to read an SMS as designed:

1. NADIF Client must have received an incoming SMS notification (`NADIF_FEAT_STATUS_SM`) previously.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

2. NADIF client must know the index of the message which it wants to read. The index number is obtained by the unsolicited notification `NADIF_FEAT_STATUS_SM` or invoking `NADIF_sms_list()` to read the index list available.
3. NADIF Client is expected to use `NADIF_sms_read()` to read decoded /formatted SMS.
4. NADIF Client has to decode the SMS message if NADIF_sms_read_pdu() used.
5. After reading an incoming SMS it is recommended to delete it, avoiding a full inbox and possible loss of messages.

3.10.1.2.6 List SMS

NADIF Client is expected to use `NADIF_sms_list()` to query the index list of incoming messages available.

3.10.1.2.7 Delete SMS

The following expectations must be met by NADIF Client in order to delete an SMS as designed:

1. NADIF client must know the index of the message which wants to delete.
2. NADIF Client is expected to use `NADIF_sms_delete()` to delete an SMS.

3.10.1.2.8 Set SCA

NADIF Client is expected to use `NADIF_sms_set_sca()` to set the SCA.

3.10.1.2.9 Query SCA

NADIF Client is expected to use `NADIF_sms_get_sca()` to query the SCA.

3.10.1.2.10 Set SMS cell Broadcast Configuration

This API allows NADIF Client to set SMS broadcast message configuration through the modem, see the main use cases supported by this API:

Use Case 1: BCM activation

To enable the BCM, NADIF Client must set bcm_enable to NADIF_SMS_BCM_ENABLE.

Use Case 2: BCM deactivation

To disable the BCM, NADIF Client must set bcm_enable to NADIF_SMS_BCM_DISABLE.

Use Case 3: Select/added services id

To add specific service id, NADIF Client must set to_service_id/from_service_id and specify the corresponding service as selected

3.10.1.3 Extending Platform Functionality

Not applicable

3.10.2 Setup

3.10.2.1 Tools/Equipment Needed

Not applicable

3.10.2.2 Setup Instructions

Not applicable

3.10.2.3 Setup Troubleshooting

Not applicable

3.10.3 Usage

3.10.3.1 Basic Instructions

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.10.3.2 Detailed Instructions

3.10.3.2.1 Send SMS

API/Type Description	Doxygen Path
Structure of SMS Data.	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_SMS_DATA**
API to send SMS	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_sms_send()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to send an SMS. The following parameters must be populated in NADIF_SMS_DATA to send a message successfully when the data format is text or binary or UCS2:

- `addr` – Destination address of SMS/Phone Number
- `data` - SMS data/payload
- `data_len` – Length of the SMS data
- `data_format` – Format of the SMS data, i.e., text or binary.
- `time_type` – Relative or absolute timestamp format.
- `time` – Must be filled according to the `time_type`, see the instruction in doxygen.

The following parameters may be optional:

- `sms_priority` – SMS priority, i.e., Normal, Interactive, urgent and emergency.
- `sca` – Service Center Address.

The following parameter must be populated in NADIF_SMS_DATA when the data format is Transparent PDU.

- `data` – SMS PDU in ASCII Hex format NULL terminated string.
- `data_format` – Format of the SMS data, i.e., transparent PDU.

All other parameters are ignored.

3.10.3.2.2 Incoming SMS

The unsolicited notification `NADIF_FEAT_STATUS_SM` is sent whenever an incoming SMS is received. It gives relevant information about the incoming SMS, such as error indicator, index, SMS data, status, type, format, etc. See more details in the following doxygen path:

Type Description	Doxygen Path
Structure of notification data	Modules |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_SMS_DATA**

The Following elements are the most important to be verified by the NADIF Client after receiving this unsolicited notification:

- `NADIF_SMS_STATUS`: This element indicates `NADIF_SMS_NEW` when an incoming SMS has been received, `NADIF_SMS_BROADCAST_MESSAGE` when a broadcast message has been

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

received, NADIF_SMS_ETWS_MESSAGE when an ETWS message has been received. The status will be reported as NADIF_SMS_MEMORY_FULL if the SMS memory is full. In case, the stored SMS messages need to be deleted.

- NADIF_SMS_TYPE: This element indicates the current situation of the SMS, for example, it will be NADIF_SMS_MT_UNREAD after receiving an incoming SMS. The type changes to NADIF_SMS_MT_READ after the NADIF Client reads the SMS via NADIF_sms_read().
- index: Unique number to handle a received SMS (MT). This index is used to read and delete a specific SMS message. NADIF_sms_list() can also be used to query the index list available.

3.10.3.2.3 Indication for SMS Delivery report

Type Description	Doxygen Path
Structure of notification data	Modules |-> NADIF |-> Short Message Service (SMS) |-> ***NADIF_SMS_DATA***

This API allows NADIF Client to receive Indication for SMS Delivery report for Mobile originated SMS and read the PDU data of the received report.

The Following elements are the most important to be verified by the NADIF Client after receiving this unsolicited notification:

- NADIF_SMS_STATUS: This status will be reported as NADIF_SMS_DELIVERY_REPORT if the Indication for SMS Delivery report is received.
- data_len: length of SMS Delivery report indication.
- format: NADIF_SMS_DATA_TRANSPARENT_PDU is received for Indication of the SMS Delivery report.

3.10.3.2.4 Read SMS

API/Type Description	Doxygen Path
API to read SMS	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_sms_read()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_SMS_READ_RESP**

This API allows NADIF Client to read the decoded data of the received SMS. Note that the unsolicited notification NADIF_FEAT_STATUS_SM is reported when an SMS is received, and it informs the index of the message.

The status of the message NADIF_SMS_READ_RESP->sms_type indicates whether the messages were read or not, after reading via NADIF_sms_read() the status is changed from NADIF_SMS_MT_UNREAD to NADIF_SMS_MT_READ.

See a typical MSC for more details about sending and reading SMS in the following doxygen link:

Doxygen Path
Middleware |-> **NADIF Message Sequence Charts - Send/Read SMS**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.10.3.2.5 Read SMS PDU

API/Type Description	Doxygen Path
API to read SMS PDU	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_sms_read_pdu()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_SMS_READ_RESP_PDU**

This API allows NADIF Client to read the PDU data of the received SMS. Note that the unsolicited notification `NADIF_FEAT_STATUS_SM` is reported when an SMS is received, and it informs the index of the message.

The status of the message `NADIF_SMS_READ_RESP->sms_type` indicates whether the messages were read or not, after reading via `NADIF_sms_read_pdu()` the status is changed from `NADIF_SMS_MT_UNREAD` to `NADIF_SMS_MT_READ`.

See a typical MSC for more details about sending and reading SMS PDU in the following doxygen link:

Doxygen Path
Middleware |-> **NADIF Message Sequence Charts – Send/Read SMS PDU**

3.10.3.2.6 List SMS

API/Type Description	Doxygen Path
API to list SMS	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_sms_list()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_SMS_LIST_RESP**

This API allows NADIF Client to read the index list of the SMS messages available. The user can filter the list by `NADIF_SMS_MT_UNREAD`, `NADIF_SMS_MT_READ` or `NADIF_SMS_MT_ALL`.

It is recommended to use this API after system startup to verify that there is no incoming unread message

3.10.3.2.7 Delete SMS

API/Type Description	Doxygen Path
API to delete a SMS	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_sms_delete()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

This API allows NADIF Client to delete an SMS message by entering an index.

3.10.3.2.8 Set SCA

API/Type Description	Doxygen Path
Structure of SCA Data.	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_SMS_SCA_DATA**
API to set SCA	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_sms_set_sca()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to set the SCA number that is used by the modem when sending or storing SMS messages. If the `sca_number` parameter provided is prefixed with a plus sign (+), the `sca_type` defaults to 145, regardless of whether the type is specified. If the `sca_type` is not provided and `sca_number` is not prefixed with a (+), the address type defaults to 129.

The AT command equivalent to this command is AT+CSCA (refer to 3GPP TS 27.005).

3.10.3.2.9 Query SCA

API/Type Description	Doxygen Path
API to query SCA	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_sms_get_sca()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_GET_SMS_SCA_RESP**

This API allows NADIF Client to query the current SCA number, it returns the string of the SCA number and type, `sca_number` and `sca_type`, respectively.

The AT command equivalent to this command is AT+CSCA (refer to 3GPP TS 27.005).

3.10.3.2.10 Set SMS cell Broadcast Configuration

API/Type Description	Doxygen Path
Structure of broadcast config request data	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS) |-> **NADIF_SMS_BCM_CONFIG_DATA**
API to set SMS Broadcast Configuration through the modem	Modules |-> Modem |-> NADIF |-> Short Message Service (SMS)

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	`|-> NADIF_sms_set_broadcast_config_req()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_RESULT`

This API allows NADIF Client to set SMS broadcast message configuration through the modem, see the main use cases supported by this API:

Use Case 1: BCM activation

To enable the BCM, NADIF Client must set bcm_enable to NADIF_SMS_BCM_ENABLE.

Use Case 2: BCM deactivation

To disable the BCM, NADIF Client must set bcm_enable to NADIF_SMS_BCM_DISABLE.

Use Case 3: Select/add services id

To add specific service id, NADIF Client must set to_service_id/from_service_id and specify the corresponding service as selected

3.10.3.2.11 Get SMS cell Broadcast Configuration

API/Type Description	Doxygen Path

Structure of broadcast config request data	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Short Message Service (SMS)`
	`|-> NADIF_SMS_GET_BCM_CONFIG_RESP`

API to get SMS Broadcast Configuration through the modem	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Short Message Service (SMS)`
	`|-> NADIF_sms_get_broadcast_config()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_GET_BCM_CONFIG_RESP`

This API allows the client to query and get the SMS cell Broadcast Configuration through modem. The notification of this API NADIF_FEAT_SMS_GET_BCM_CFG return the current status of the SMS cell Broadcast Configuration, i.e., broadcast from service ids, broadcast to service ids, ids selected, or not selected.

Before the reading of the configuration data, the client should check the result of NADIF_RESULT, which indicates whether the request was a success or failure.

3.10.3.3 Testing/Error Reporting

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.11 Voice Call

3.11.1 Overview

3.11.1.1 Platform Content Overview

This section describes the support provided by NADIF for incoming and outgoing voice calls via cellular network. See the following features supported by NADIF:

- Dial an outgoing call - request to initiate a voice call specifying a calling number (i.e., call origination).
- Dial an outgoing voice call over IMS with custom user data transmission using SIP messages
- Answer an incoming call - request to answer an incoming call (ringing).
- Hang-up a call in progress - request to hang-up/disconnect active voice call.
- Manage multiple calls
- Provide call status - unsolicited notification message to indicate whenever the call state has changed: dialing, ringing, waiting for answer, conversation, call end with reason, etc.
- Provide all call status - unsolicited notification message to indicate the multiple call status whenever the call state has changed: dialing, ringing, waiting for answer, conversation, call end with reason, etc.
- Query call status – request to query the current call status.
- Query all call status – request to query the current call status of all active calls.
- An incoming voice call can wake up the application processor while it is suspended.
- NADIF supports circuit-switched call and VoLTE (voice call over LTE), which is handled automatically by the modem.

Notes:

1. *When IMS/VoIP service is disabled all voice calls are circuit-switched calls. So, when a call is initiated (incoming or outgoing) and the modem is in service on LTE, the modem will switch to WCDMA (or GSM) service to complete the voice call. When switching to WCDMA, all active packet data sessions are handed over to WCDMA and will continue. When switching to GSM, active packet data sessions may be dropped or may be suspended, depending on network capability for multiple contexts in GPRS/EDGE.*
2. *Voice supplementary services are additional services provided by a cellular service provider beyond a single party voice call. Voice supplementary services include line identification (CLIP, CLIR, COLP, COLR), call forwarding, call waiting, call hold, multiparty calls, call barring, and explicit call transfer. These services are not supported by NADIF. NADIF only provides support for one active voice call to a single party at a time.*

3.11.1.2 Product Expectations

NADIF Client must include the header file `nadif_voice.h` to access any API described in this section.

3.11.1.2.1 Dial an outgoing call

The following expectations must be met by NADIF Client in order to send a SMS as designed:

1. The modem must be registered on the cellular network
2. NADIF Client is expected to use `NADIF_voice_dial()` to dial an outgoing call.

3.11.1.2.2 Dial an outgoing call over IMS with custom user data

The following expectations must be met by NADIF Client in order to call over IMS with custom user data:

1. The modem must be registered on the cellular network with IMS support
2. NADIF Client is expected to use NADIF_voice_dial_custom_data_over_ims() to dial an outgoing call.

3.11.1.2.3 Answer an incoming call

The following expectations must be met by NADIF Client in order to answer an incoming call as designed:

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

1. NADIF Client must have received an unsolicited notification `NADIF_FEAT_STATUS_VOICE` with the status of `NADIF_CALL_INCOMING`.
2. NADIF client must know the call ID of the incoming call. The incoming `call_id` is informed by `NADIF_FEAT_STATUS_VOICE`.
3. NADIF Client is expected to use `NADIF_voice_answer()` to answer an incoming voice call.

3.11.1.2.4 Disconnect a call

The following expectations must be met by NADIF Client in order to hang-up/disconnect a call in progress or reject an incoming call as designed:

1. NADIF client must know the call ID of the call to be disconnected.
2. NADIF Client is expected to use `NADIF_voice_hangup()` to disconnect a voice call.

3.11.1.2.5 Manage multiple calls

The following expectations must be met by NADIF Client in order to manage the multiple calls as designed:

1. NADIF Client must have received an unsolicited notification `NADIF_FEAT_VOICE_ALL_CALLS_STATUS` with the num_of_calls more than one.
2. NADIF client must know the status of each call and the call Id if required. The `call_status` and `call_id` are informed by `NADIF_FEAT_VOICE_ALL_CALLS_STATUS` .
3. NADIF Client is expected to use NADIF_voice_manage_calls `()` to manage the multiple calls.

3.11.1.2.6 Call status

The following expectations must be met by NADIF Client in order to query the current call status as designed:

1. NADIF client must know the call ID of the call.
2. NADIF Client is expected to use `NADIF_voice_get_call_status()` to query the call status.

The Call status is also reported by the unsolicited notification `NADIF_FEAT_STATUS_VOICE`.

3.11.1.2.7 All Call status

The following expectations must be met by NADIF Client in order to query the current call status as designed:

1. NADIF Client is expected to use `NADIF_voice_get_all_call_status()` to query the call status, when Call ID of the call is unknown.

The status of all call status request is notified by the `NADIF_GET_ALL_CALL_STATUS_RESP notification`.

3.11.1.2.8 Call end reason

The following expectations must be met by NADIF Client in order to know the reason when the call is ended or dropped:

1. The call end reason is reported by the unsolicited notification `NADIF_FEAT_STATUS_VOICE` casting the data type `NADIF_CALL_STATUS_DATA`.

3.11.1.3 Extending Platform Functionality

Not applicable

3.11.2 Setup

3.11.2.1 Tools/Equipment Needed

Not applicable

3.11.2.2 Setup Instructions

Not applicable

3.11.2.3 Setup Troubleshooting

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.11.3 Usage

3.11.3.1 Basic Instructions

Not applicable

3.11.3.2 Detailed Instructions

3.11.3.2.1 Dial an outgoing call

API/Type Description	Doxygen Path

API to dial	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> Voice`
	`  |-> **NADIF_voice_dial()**`

Notification data type of API	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> NADIF Common Types`
	`  |-> **NADIF_RESULT**`

This API allows NADIF Client to initiate and dial an outgoing call to a specific number. Note that the notification `NADIF_FEAT_STATUS_VOICE` is sent whenever the call status changes.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

See a typical use case for an outgoing call in the following MSC:

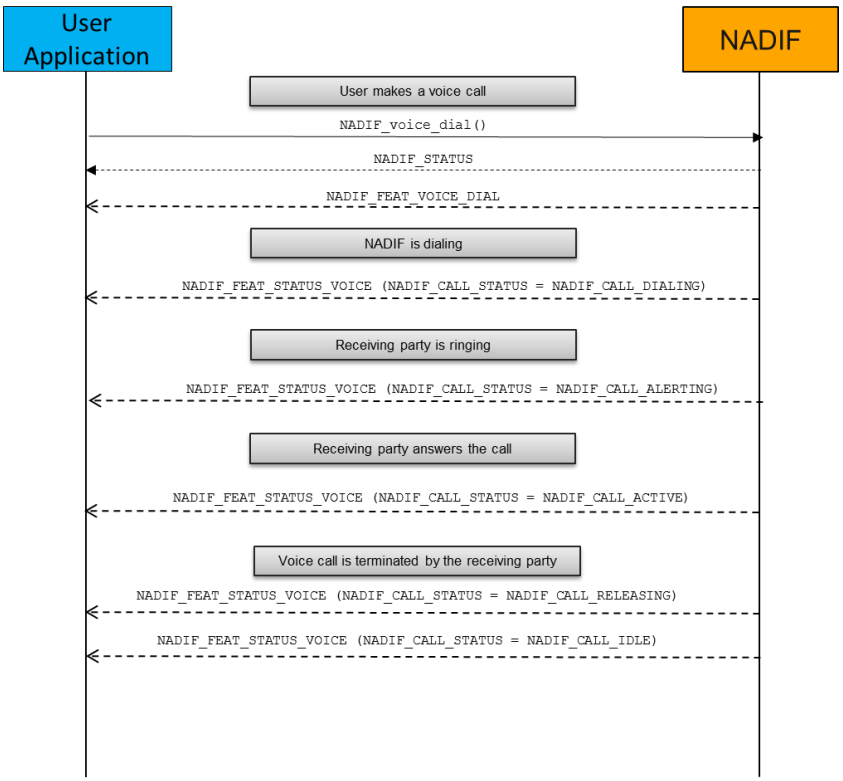


Figure 14: Outgoing Call Sequence

3.11.3.2.2 Dial an outgoing call over IMS with custom user data

API/Type Description	Doxygen Path
Dial a voice call over IMS with custom user data transmission using SIP	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_voice_dial_custom_data_over_ims()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

This API allows NADIF Client to initiate and dial an outgoing call to a specific number via IMS and send custom user data to remote endpoint (Ex: call center) over SIP messages during the call. The custom user data provides via API input parameters. Note that the notification `NADIF_FEAT_STATUS_VOICE` is sent whenever the call status changes. Notification `NADIF_FEAT_STATUS_VOICE_CUST_IMS_DATA_TRANSMISSION` indicates the data transmission result status.

The following parameters must be populated in NADIF_VOICE_CUST_USER_DATA to send custom user data:

1. `cust_user_data_content_type_len` – Content type length of the custom user data.
2. `cust_user_data_content_type` – Content type of the custom user data.
3. `cust_user_data_len` – Custom user data length.
4. `cust_user_data` – Custom user data sent over SIP header.
5. `phone_number` – Phone number string.
6. `cust_user_sip_accept_len` – Accept payload of SIP accept data.
7. `cust_user_sip_accept` - sip accept over SIP header.

Note: This functionality has to be supported by MNO, so that OEM can negotiate with the MNO for the SIP messages.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

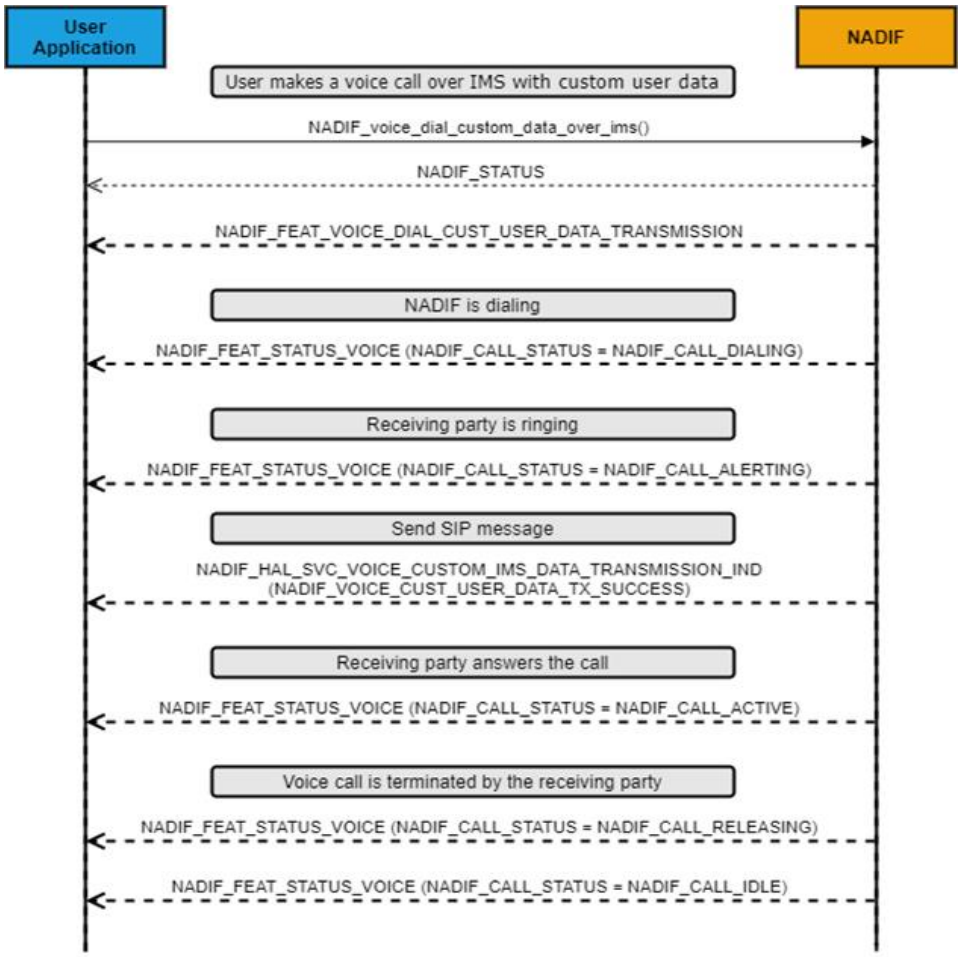


Figure 15: Incoming Call Sequence

3.11.3.2.3 Answer an incoming call

API/Type Description	Doxygen Path

API to answer an incoming call	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> Voice`
	`  |-> NADIF_voice_answer()`

Notification data type of API	`Modules` `  |-> Modem`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path

	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_RESULT`

This API allows NADIF Client to answer an incoming call. Note the unsolicited notification `NADIF_FEAT_STATUS_VOICE` is reported when there is an incoming call (ringing). This notification also informs the call ID, which must be used to answer the call by `NADIF_voice_answer()`.

See a typical use case for an incoming call in the following MSC:

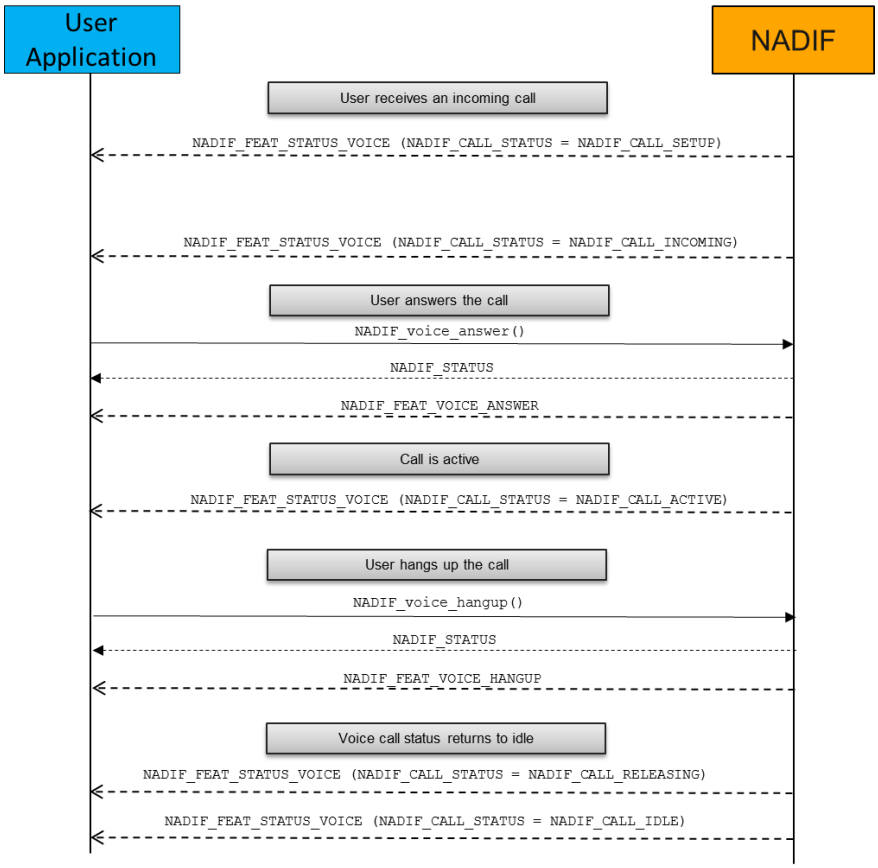


Figure 16: Incoming Call Sequence

NOTE: The unsolicited notification `NADIF_FEAT_STATUS_VOICE` is sent whenever there is change in call status.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.11.3.2.4 Disconnect a call

API/Type Description	Doxygen Path
API to disconnect a call	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_voice_hangup()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to hang-up/disconnect a voice call in progress or reject an incoming call.

3.11.3.2.5 Manage multiple calls

API/Type Description	Doxygen Path
API to Manage the multiple calls	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_voice_manage_calls ()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_MANAGE_CALLS_RES**

This API allows NADIF Client to manage multiple calls by using following supplementary service applicable during the call.

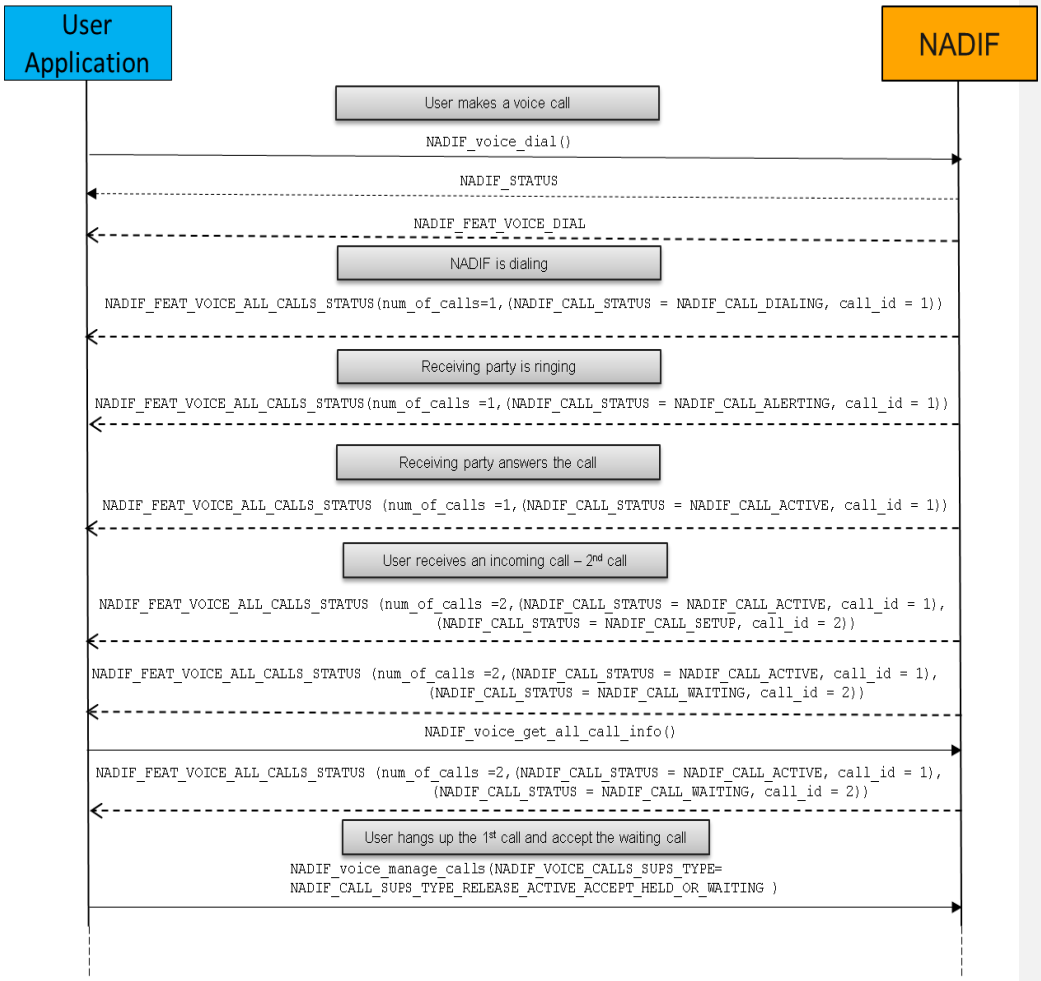
- NADIF_CALL_SUPS_TYPE_RELEASE_HELD_OR_WAITING : Release held or waiting call
- NADIF_CALL_SUPS_TYPE_RELEASE_ACTIVE_ACCEPT_HELD_OR_WAITING : Release active call and accept held or waiting call
- NADIF_CALL_SUPS_TYPE_HOLD_ACTIVE_ACCEPT_WAITING_OR_HELD : Hold active call and accept waiting or held call
- NADIF_CALL_SUPS_TYPE_HOLD_ALL_EXCEPT_SPECIFIED_CALL : Hold all calls except a specified one
- NADIF_CALL_SUPS_TYPE_END_ALL_CALLS :End all calls

Note the unsolicited notification `NADIF_FEAT_VOICE_ALL_CALLS_STATUS` is reported when there is any change in the call status. This notification informs the call status and call ID of all the calls, which can be used to manage the call by `NADIF_voice_manage_calls()`. Conference call is not supported.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

See a typical use case for releasing the active call and accepting incoming call in the following MSC:



© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

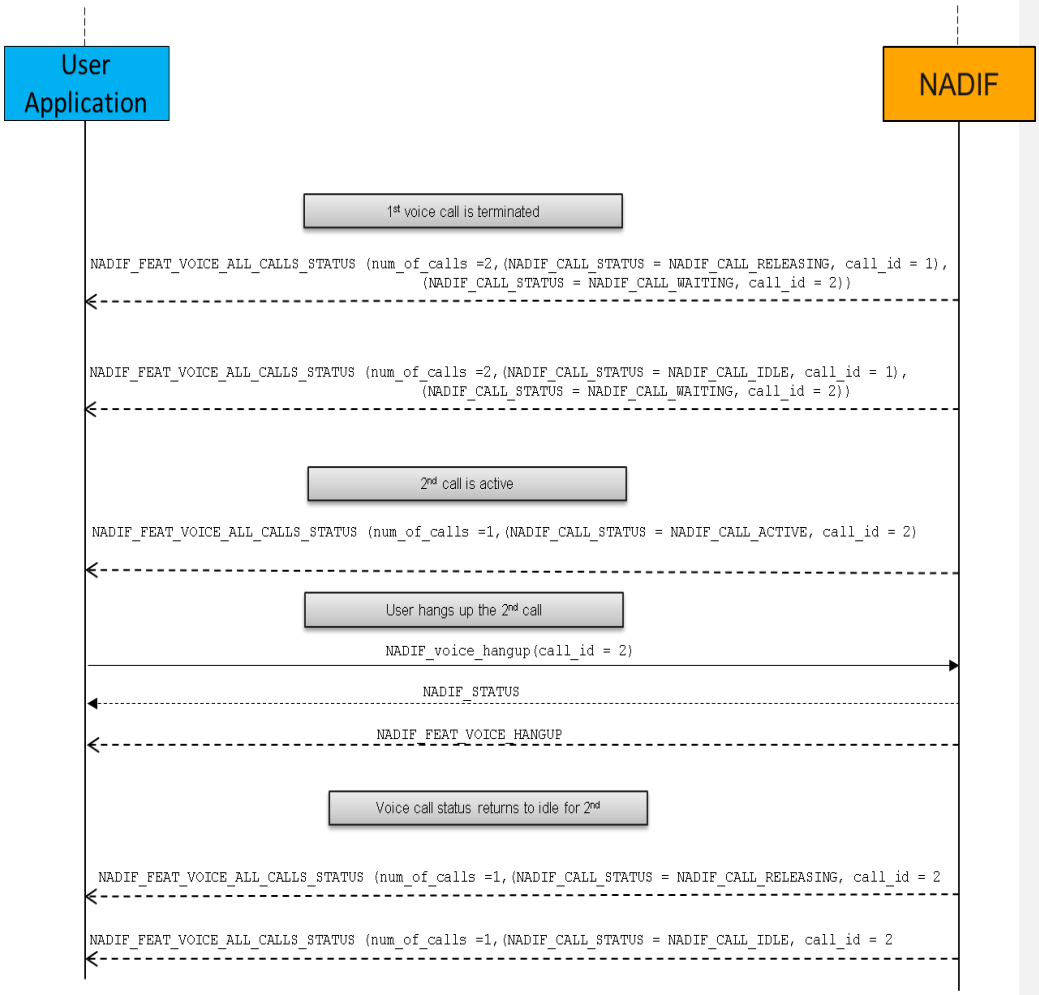


Figure 17: Release active call and accepting held or waiting call

3.11.3.2.6 Call status

API/Type Description	Doxygen Path
API to query call status	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_voice_get_call_status()**
Notification data type of API	Modules

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path

	`|-> Modem`
	`|-> NADIF`
	`|-> Voice`
	`|-> **NADIF_GET_CALL_STATUS_RESP**`

Call status gives relevant information about an incoming or outgoing voice call, such as voice call status, call direction, phone number, reason for ending the call etc. NADIF Client must evaluate the `NADIF_CALL_STATUS` to identify what is the current state of the voice call, for example, dialing, incoming, held, active etc.

The call status is reported by the unsolicited notification `NADIF_FEAT_STATUS_VOICE` or by querying via `NADIF_voice_get_call_status()`.

The unsolicited notification is typically sent whenever there is any change in the voice call status, this status is provided by `NADIF_CALL_STATUS` element in notification data.

3.11.3.2.7 All call status

API/Type Description	Doxygen Path

API to query call status	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Voice`
	`|-> **NADIF_voice_get_all_call_status()**`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Voice`
	`|-> **NADIF_GET_ALL_CALL_STATUS_RESP**`

All call status gives relevant information about an incoming or outgoing voice call, such as call ID,voice call status, call direction, phone number, reason for ending the call etc. NADIF Client must evaluate the `NADIF_CALL_STATUS` to identify what is the current state of the voice call, for example, dialing, incoming, held, active etc.

The call status is reported by the unsolicited notification `NADIF_FEAT_STATUS_VOICE` or by querying via `NADIF_voice_get_all_call_status()`.

The unsolicited notification is typically sent for individual voice calls whenever there is any change in the voice call status, this status is provided by `NADIF_CALL_STATUS` element in notification data.

3.11.3.2.8 Call end reason

It is recommended to check the element `call_end_reason` of this notification to determine if there was an error to terminate the call. See the following doxygen path for call end reasons:

Type Description	Doxygen Path

Enumeration of call end reason	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Type`
	`|-> **NADIF_CALL_END_REASON**`

The call end reason is reported by the unsolicited notification `NADIF_FEAT_STATUS_VOICE` casting the data type `NADIF_CALL_STATUS_DATA`.

Refer to the following Message Sequence Chart from Doxygen Path to know the flow of call end reason notification:

Type Description	Doxygen Path
MSC	`Modules` `|-> Modem`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

Type Description	Doxygen Path

	`|-> NADIF`
	`|-> Message Sequence Charts`
	`|-> NADIF_voice_dial`
	`|-> Call failed or dropped by the Network`

3.11.3.3 Testing/Error Reporting

Not applicable

3.12 Modem Status

3.12.1 Overview

3.12.1.1 Platform Content Overview

This section describes the support provided by NADIF to obtain various status information from the modem.

3.12.1.2 Product Expectations

NADIF Client must include the header file `nadif_modem_status.h` to access any API described in this section.

3.12.1.2.1 Modem Status

NADIF Client is expected to receive the unsolicited notification `NADIF_FEAT_STATUS_MODEM` whenever the modem status is changed. This notification is typically sent after a new NADIF Client registration.

`NADIF_MODEM_STATE` description:

1. `NADIF_MODEM_NOT_READY`. Modem is not ready to communicate. This indicates that the modem is not in a state where it can accept commands from NADIF, hence NADIF will not send any commands to the modem.
2. `NADIF_MODEM_FAILURE`. There is a failure to communicate with the modem. This will be reported after NADIF has attempt to communicate with modem and has failed after retries. Also NADIF can detect is there is any malfunction in the modem, e.g. Modem Failure, in this scenarios NADIF will also report this state.
3. `NADIF_MODEM_READY`. This state is reported to all clients whenever they start to communicate with NADIF. By doing this NADIF let clients know that NADIF is running.
4. `NADIF_MODEM_NOT_FULLY_OPERATIONAL`. The Modem is functional but not all services are available. In this state some request will no be process as the service related is not ready.
5. `NADIF_MODEM_FULLY_OPERATIONAL`. The Modem is functional and all services are available. In this state all request will be send to the modem.
6. `NADIF_MODEM_RESET`. Modem reset detected. NADIF and Modem likely out of sync. NADIF does not attempt to recover modem connection. Any reset recovery is at discretion of NADIF client.
7. `NADIF_MODEM_OFF`. Modem is shut down. This indicates the modem is not in a state where it can accept commands from NADIF. The board must be power-cycled in order to re-establish communications

3.12.1.2.2 Modem services status

NADIF Client is expected to receive the unsolicited notification `NADIF_FEAT_STATUS_SERVICES` after a new NADIF Client registration.

3.12.1.2.3 IMSI

The following expectations must be met by NADIF Client in order to query the IMSI as designed:

1. SIM must be in Ready state (`NADIF_SIM_READY`).
2. NADIF Client is expected to use `NADIF_modem_get_imsi()` to query IMSI.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.12.1.2.4 Manufacturer ID

NADIF Client is expected to use `NADIF_modem_get_manufacturer_id()` to query the manufacturer ID.

3.12.1.2.5 Model ID

NADIF Client is expected to use `NADIF_modem_get_model_id()` to query the model ID.

3.12.1.2.6 Phone Number

The following expectations must be met by NADIF Client in order to query the assigned voice phone number (MSISDN) as designed:

1. SIM must be in Ready state (`NADIF_SIM_READY`).
2. MSISDN must be provisioned on the SIM by the carrier.
3. NADIF Client is expected to use `NADIF_modem_get_phone_number()` to query the phone number.

3.12.1.2.7 Revision ID

NADIF Client is expected to use `NADIF_modem_get_revision_id()` to query the firmware revision ID.

3.12.1.2.8 Serial Number (IMEI)

NADIF Client is expected to use `NADIF_modem_get_serial_number()` to query the serial numbers of the modem.

3.12.1.2.9 SIM Serial Number

The following expectations must be met by NADIF Client in order to query the SIM serial number as designed:

1. SIM must be in Ready state (`NADIF_SIM_READY`).
2. NADIF Client is expected to use `NADIF_modem_get_sim_serial_number()` to query the ICCID.

3.12.1.2.10 Thermal mitigation device list

NADIF client is expected to use `NADIF_modem_get_thermal_mitigation_device_list` () to obtain the list of supported modem thermal mitigation devices.

3.12.1.2.11 Thermal mitigation level

NADIF Client is expected to use `NADIF_modem_get_thermal_mitigation_level()` to query the current mitigation level for all supported devices.

3.12.1.2.12 Thermal mitigation report

NADIF Client is expected to use `NADIF_modem_set_thermal_mitigation_report()` to enable or disable the thermal mitigation notification.

3.12.1.2.13 Get NAD Configuration

NADIF Client is expected to use NADIF_modem_get_nad_configuration() to query the value of NAD configuration items.

3.12.1.2.14 Set NAD Configuration

NADIF client is expected to use NADIF_modem_set_nad_configuration() to set the value of the NAD configuration items.

3.12.1.2.15 Get Modem Reset Counts

NADIF client is expected to use `NADIF_modem_get_modem_resets_info()` to get the value of modem reset counter in the current power cycle and it includes planned, requested and unexpected modem resets due to modem reset.

3.12.1.2.16 Get SAR RF State

NADIF client is expected to use `NADIF_modem_get_sar_rf_state`() to get the SAR RF state, The SAR RF state means Modem using the default value or the set value for TX power backoff

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.12.1.2.17 Activate SAR RF State

NADIF client is expected to use `NADIF_modem_activate_sar_rf_state`() to activate the specified SAR RF state.

3.12.1.2.18 Get SAR Tx Backoff

NADIF client is expected to use `NADIF_modem_get_sar_backoff`() to get the SAR Tx power backoff value of the supported bands of the specified network type.

3.12.1.2.19 Set SAR Tx Backoff

NADIF client is expected to use `NADIF_modem_set_sar_backoff`() to set the SAR Tx power backoff value of the specified bands.

3.12.1.2.20 Set Modem Configuration

NADIF client is expected to use `NADIF_modem_set_config` () to set the Modem configuration items.

3.12.1.2.21 Get Modem Configuration

NADIF Client is expected to use `NADIF_modem_get_config` () to query the Modem configuration items.

3.12.1.2.22 Traffic Channel Dormancy Status

NADIF Client is expected to be notified through the unsolicited notification NADIF_FEAT_MODEM_DORMANCY_STATUS whenever the state of traffic channel dormancy status is changed.

3.12.1.3 Extending Platform Functionality

Not applicable

3.12.2 Setup

3.12.2.1 Tools/Equipment Needed

Not applicable

3.12.2.2 Setup Instructions

Not applicable

3.12.2.3 Setup Troubleshooting

Not applicable

3.12.3 Usage

3.12.3.1 Basic Instructions

Not applicable

3.12.3.2 Detailed Instructions

3.12.3.2.1 Modem Status

The unsolicited notification `NADIF_FEAT_STATUS_MODEM` is reported whenever the modem status has changed.

The `NADIF_FEAT_STATUS_MODEM` includes the modem reset count information.

See more details in the following doxygen path:

Type Description	Doxygen Path

Structure of notification data	`Modules`
	`  |-> NADIF`
	`  |-> MODEM_STATUS`
	`  |-> NADIF_MODEM_STATUS`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

This unsolicited notification is typically sent when one of the following use cases occurs:

Use Case: A new NADIF Client has been registered but Modem is not initialized

Modem Status is sent whenever a new client has been registered, in this case, the modem is not ready to receive any request, so NADIF Server reports the status as `NADIF_MODEM_NOT_READY`.

Use Case: A new NADIF Client has been registered and Modem is ready

In this case, the modem is already initialized, so NADIF Server reports the modem status as `NADIF_MODEM_READY`.

NOTE: *After sending `NADIF_MODEM_READY`, NADIF Server reports one more `NADIF_FEAT_STATUS_MODEM` notification, indicating whether all services are available or not. See the use cases Service Status.*

Use Case: Modem has been initialized

Immediately after the modem is initialized, NADIF Server notifies the clients that modem status is `NADIF_MODEM_READY`.

NOTE: *After sending `NADIF_MODEM_READY`, NADIF Server reports one more `NADIF_FEAT_STATUS_MODEM` notification, indicating whether all services are available or not. See the use cases Service Status.*

Use Case: Service Status

This modem status notification is sent after NADIF Server reports *`NADIF_MODEM_READY`* if all services are available the modem status is reported as `NADIF_MODEM_FULLY_OPERATIONAL`, otherwise some of the services are not available and the modem status is reported `NADIF_MODEM_NOT_FULLY_OPERATIONAL`.

NOTE: *After this notification, NADIF Server will report a list with the status of each service, indicating whether it is available or not. See the unsolicited notification `NADIF_FEAT_STATUS_SERVICES`.*

Use Case: Network has changed the phone number

The modem status is reported as `NADIF_MODEM_ACTIVATED` when the network changes the mobile phone number, also known as MSISDN.

Use Case: Commination error with Modem

The modem status is reported as `NADIF_MODEM_FAILURE` when NADIF Server fails to communicate with Modem. For this modem status the modem reset count will be reported as invalid. This is a typical situation caused by some internal error, a system reset might be necessary to recover from this error.

Use Case: Modem has been reset

The modem status is reported as `NADIF_MODEM_RESET` when NADIF Server detects that modem has been reset for any reason. This modem status will include modem reset count information as additional information.

After the modem reset is complete, NADIF will report to the NADIF clients `NADIF_MODEM_FULLY_OPERATIONAL` or `NADIF_MODEM_NOT_FULLY_OPERATIONAL` and will provide updated modem services status (e.g., Modem status, Call status, Thermal mitigation status, etc). Refer to the following Message Sequence Chart from Doxygen path:

Type Description	Doxygen Path

MSC	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Message Sequence Charts`
	`|-> NADIF_Modem_Status`
	`|-> **NADIF Modem Reset**`

NOTE: If the information that NADIF client needs is not reported by the modem, the NADIF client has to query the information needed.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

When a modem reset is detected, the total modem reset detected count will be incremented. Additionally, a reset type counter will be incremented. So, the total modem reset count will be the sum of all reset type counters. The following reset type counters are supported:

- Planned Modem Reset Counter: incremented when a modem configuration change occurs that triggers a reset, such as when the modem hardware configuration is activated.
- Requested Modem Reset Counter: incremented when a modem reset is triggered via `nadif_modem_set_state()` by a NADIF Client.
- Unexpected Modem Reset Counter: incremented when a modem reset is detected that was not triggered by a Planned Modem Reset or Requested Modem Reset.

Use Case: Modem has been shut off

The modem status is reported as `NADIF_MODEM_OFF` when the modem is shut down. A power cycle is needed to re-establish the communication. This modem state is expected when the unit is shutting down.

3.12.3.2.2 Modem service status

The unsolicited notification `NADIF_FEAT_STATUS_SERVICES` reports the status of each NADIF service, informing if the service is available (`NADIF_SERVICE_AVAILABLE`) or unavailable (`NADIF_SERVICE_NOT_AVAILABLE`).

See more details in the following doxygen path:

Type Description	Doxygen Path

Structure of notification data	`Modules`
	`|-> NADIF`
	`|-> Modem Status`
	`|-> NADIF_SERVICES_STATUS`

NOTE: *The status `NADIF_SERVICE_LIMITED` should not be reported by any NADIF service*.

Each service status provides the availability of a NADIF API group based on the support of the installed HAL services, as well as Voice, Wireless Data, Device Management, Network Access, User Identify, IMS, Custom services, etc.

From the NADIF Client perspective, it is recommended to monitor this notification to make sure that all the services are up and running, especially during modem startup.

In case of a service failure, NADIF has a strategy to try to recover it every 10 seconds without affecting other services. This actions will continue indefinitely until the service recovery is successful. However, if the error persists for a long time a system reset might be necessary to recover from this situation.

This unsolicited notification is typically sent after registering a NADIF Client, see the Client Registration section for more details.

3.12.3.2.3 IMSI

API/Type Description	Doxygen Path

API to query call status	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Modem Status`
	`|-> NADIF_modem_get_imsi()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Voice`
	`|-> NADIF_GET_IMSI_RESP`

The notification of this API contains a string with IMSI to identify the SIM which is attached to the modem.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.12.3.2.4 Manufacturer ID

API/Type Description	Doxygen Path
API to query manufacturer ID	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_get_manufacturer_id()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_GET_MANUFACTURER_ID_RESP**

The notification of this API contains a string to identify the manufacturer.

The AT command equivalent is AT+GMI (refer to 3GPP TS 27.007).

3.12.3.2.5 Model ID

API/Type Description	Doxygen Path
API to query model ID	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_get_model_id()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_GET_MODEL_ID_RESP**

The notification of this API contains a string to identify the model of the device. This usually corresponds to the manufacturer's model name under which the device is marketed.

The AT command equivalent is AT+GMM (refer to 3GPP TS 27.007).

3.12.3.2.6 Phone Number

API/Type Description	Doxygen Path
API to query phone number	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_get_phone_number()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_GET_PHONE_NUMBER_RESP**

The notification of this API contains a string with MSISDN if it is available in the device provisioning.

The AT command equivalent is AT+CNUM (refer to TIA/EIA/IS-131).

3.12.3.2.7 Revision ID

API/Type Description	Doxygen Path
API to query revision ID	Modules |-> Modem

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|>> NADIF |-> Modem Status |-> **NADIF_modem_get_revision_id()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_GET_REVISION_ID_RESP**

The notification of this API contains a string with the firmware revision of the device. This usually corresponds to the manufacturer's software revision loaded on the device.

The AT command equivalent is AT+GMR (refer to 3GPP TS 27.007).

3.12.3.2.8 Serial Number

API/Type Description	Doxygen Path
API to query serial number	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_get_serial_number()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_GET_SERIAL_NUMBER_RESP**

The notification of this API contains a string with the IMEI of the modem and another string with the IMEI software version number.

The AT command equivalent is AT+GSN (refer to 3GPP TS 27.007).

3.12.3.2.9 SIM Serial Number

API/Type Description	Doxygen Path
API to query SIM serial number	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_get_sim_serial_number()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_GET_SIM_SERIAL_NUMBER_RESP**

The notification of this API contains a string with ICCID of the SIM.

3.12.3.2.10 Thermal mitigation device list

API/Type Description	Doxygen Path
API to query thermal mitigation level	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_thermal_mitigation_device_list()**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
Notification data type of API	Modules |-> Modem |-> NADIF |-> Modem Status |->**NADIF_MODEM_GET_THERMAL_MITIGATION_DEVICE_LIST_RESP**

The notification of this API reports the list of supported modem thermal mitigation devices..
See TP_F101U-00047 (Thermal Management User Guide) for more details.

3.12.3.2.11 Thermal mitigation level

API/Type Description	Doxygen Path
API to query thermal mitigation level	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_get_thermal_mitigation_level()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_MODEM_GET_THERMAL_MITIGATION_LEVEL_RESP**

The notification of this API reports the thermal mitigation level for all the supported devices.
See TP_F101U-00047 (Thermal Management User Guide) for more details.

3.12.3.2.12 Thermal mitigation report

API/Type Description	Doxygen Path
API to query thermal mitigation level	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_set_thermal_mitigation_report()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API enables or disable the unsolicited notification `NADIF_MODEM_THERMAL_MITIGATION_CHANGE`.
See TP_F101U-00047 (Thermal Management User Guide) for more details.

3.12.3.2.13 Get NAD Configuration

API/Type Description	Doxygen Path
API to query NAD configuration	Modules |-> Modem |-> NADIF |-> Modem Status |->**NADIF_modem_get_nad_configuration**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Voice

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|-> NADIF_GET_NAD_CONFIG_RESP

This API provides the value for the following parameters:

Parameter	Description	Valid Range	Default value
ecall_call_clr_dwn_timeout	eCall Call Clear-down Timeout	0 – 255 (in minutes)	0xffff
ecall_start_timeout	eCall Start Timeout	0 – 255 (in Seconds)	0xffff
ecall_al_ack_timeout	eCall Application Layer Acknowledgement Timeout	0 – 255 (in seconds)	0xffff
ecall_session_timeout	eCall Session Timeout	0 – 255 (in seconds)	0xffff
ecall_inv_signal_duration	eCall Invitation Signal Duration	0 – 255 (in milliseconds)	0xffff
hplmn_search_timer	HPLMN search timer	0x00000000-0xffffffff (in minutes)	0xffffffff means unused 0xfffffffe means use the SIM-defined timer
nad_features_mask	Mask values for the NAD features.	N/A	Unused:NADIF_MDM_FEAT_NAD_UNSUPPORTED Default mask value enabled by the modem is: NADIF_MDM_FEAT_CAS_REVISIONS
hw_config	NAD HWID	See below	See below

hw_config: NAD HWID which is composed of RF Card ID,MODEL_MINOR and PINOUT.

HWID Byte Identification			
Byte 3	Byte 2	Byte 1	Byte 0
RFC ID		MODEL MINOR	PINOUT

RF Card ID holds the index of the RFC HWID which tells the exact NAD model name, Continental's RFC IDs starts from 449 and its index is 0x0001h, RF Card ID will be set only if HWID is present.When HW ID is not present , the RF card ID will be set to 0xFF.Table below shows the RFC index and its corresponding RFC HWID. MODEL_MINOR tells the hardware revision number in decimal format (P1, P2, P3) PINOUT tells the GPIO pinout.

RF Card Index	RFC HWID	HW Module Name
0x0001h	449	FE415GLR00
0x0002h	450	FE415RW02
0x0003h	451	FE415EU02

HWID examples:

FE415EU02 P1: 0x00030100, here RFCID is ‘3’ , RFC HWID is 451 and model minor is ‘1’.

FE415EU02 P3: 0x00030300, here RFCID is ‘3’ , RFC HWID is 451 and model minor is ‘3’.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

NOTE: The following parameters have been duplicated in NADIF_ecall_get_cfg_parameters(), its preferable to use this API for these parameters.

When NADIF_modem_get_nad_configuration is called with the below parameters for the secondary subscription, the values for primary subscription will be shared in the response.

NADIF_modem_get_nad_configuration	NADIF_ecall_get_cfg_parameters
ecall_call_clr_dwn_timeout	nadif_ecall_timer_t2
ecall_start_timeout	nadif_ecall_timer_t5
ecall_al_ack_timeout	nadif_ecall_timer_t6
ecall_session_timeout	nadif_ecall_timer_t7
ecall_inv_signal_duration	nadif_ecall_timer_t3

3.12.3.2.14 Set NAD Configuration

API/Type Description	Doxygen Path

API to set NAD configuration	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> Modem Status`
	`  |->`**`NADIF_modem_set_nad_configuration`**

Notification data type of API	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> NADIF Common Types`
	`  |-> `**`NADIF_RESULT`**

This API allows the application to set the values of the NAD configuration items. The client should check the value of the notification for a success or a failure. Following are the NAD configuration items.

Parameter	Description	Valid Range	Default value
ecall_msd_len	eCall Length of Minimum Set of Data	1-140	0xff
ecall_msd	eCall Minimum Set of Data	Max. size 140 bytes,if <140 pad with '0'	0xffff
ecall_call_clr_dwn_timeout	eCall Call Clear-down Timeout	0 – 255 (in minutes)	0xffff
ecall_start_timeout	eCall Start Timeout	0 – 255 (in Seconds)	0xffff
ecall_al_ack_timeout	eCall Application Layer Acknowledgement Timeout	0 – 255 (in seconds)	0xffff
ecall_session_timeout	eCall Session Timeout	0 – 255 (in seconds)	0xffff

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

Parameter	Description	Valid Range	Default value
ecall_inv_signal_duration	eCall Invitation Signal Duration	0 – 255 (in millisecondes)	0xffff
hplmn_search_timer	HPLMN search timer	0x00000000-0xffffffff (in minutes)	0xffffffff means unused 0xfffffffe means use the SIM-defined timers
nad_features_mask	Mask values for the NAD features.	N/A	Unused:NADIF_MDM_FEAT_NAD_UNSUPPORTED Default mask value enabled by the modem is: NADIF_MDM_FEAT_CAS_REVISIONS

NOTE: The following parameters have been duplicated in NADIF_ecall_set_cfg_parameters(), its preferable to use this API for these parameters.

When NADIF_modem_set_nad_configuration is called with the below parameters for the secondary subscription, the values for primary subscription will be updated.

NADIF_modem_set_nad_configuration	NADIF_ecall_set_cfg_parameters
ecall_call_clr_dwn_timeout	nadif_ecall_timer_t2
ecall_start_timeout	nadif_ecall_timer_t5
ecall_al_ack_timeout	nadif_ecall_timer_t6
ecall_session_timeout	nadif_ecall_timer_t7
ecall_inv_signal_duration	nadif_ecall_timer_t3

3.12.3.2.15 Get Modem Reset Counts

API/Type Description	Doxygen Path
API to query Get Modem Reset	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF modem get modem reset info()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF FEAT MODEM GET MODEM RESET INFO Response**

The notification of this API contains the modem reset count information values which are planned modem reset, requested modem reset and unexpected modem reset.

See Use case: Modem has been reset under **Modem Status** for modem reset count definition.

3.12.3.2.16 Get SAR RF State

API/Type Description	Doxygen Path
API to get sar rf state	Modules |-> Modem |-> NADIF |-> Modem Status

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|-> **NADIF_modem_get_sar_rf_state()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Modem Status |->**NADIF_FEAT_MODEM_GET_SAR_RF_STATE_RSEP**

The notification of this API contains the SAR RF state, the state means which SAR TX backoff value be used for SAR TX power limit backoff.

3.12.3.2.17 Activate SAR RF State

API/Type Description	Doxygen Path
API to activate sar rf state	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_activate_sar_rf_state()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows the application to activate the SAR RF state.

sar_rf_state set to `NADIF_SAR_RF_STATE_DEFAULT`, this is the default state after modem power on, modem will take default sar value for sar tx power backoff;

sar_rf_state set to `NADIF_SAR_RF_STATE_APP_CONFIGURED`, modem will take the set sar value by NADIF_modem_set_sar_tx_backoff for sar tx power backoff. SAR RF state is not persistent across power cycles. Default at power up is always `NADIF_SAR_RF_STATE_DEFAULT`.

3.12.3.2.18 Get SAR Tx Backoff

API/Type Description	Doxygen Path
API to get sar tx power backoff value	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_get_sar_tx_backoff()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Modem Status |->**NADIF_FEAT_MODEM_GET_SAR_TX_BACKOFF_RESP**

The notification of this API contains the SAR TX backoff value of all supported bands of the specified network type.

3.12.3.2.19 Set SAR Tx Backoff

API/Type Description	Doxygen Path
API to set value for sar tx power backoff	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_set_sar_tx_backoff()**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows the application to set the SAR TX backoff value of supported bands of the specified network type.

The set SAR TX power backoff value will take effect after a modem power cycle. The set value is only used when SAR RF state is NADIF_SAR_RF_STATE_APP_CONFIGURED

3.12.3.2.20 Set Modem Configuration

API/Type Description	Doxygen Path
API to set Modem configuration	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_set_config**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows the application to set the values of the Modem configuration items. Following are the Modem configuration items supported(list not exhaustive):-

1. Active or Dormant modem state: Helps to save power.

Notes:

1. The maximum number of config items per request is NADIF_MAX_CONFIG_ITEMS_PER_REQUEST.
2. The maximum length of a item name is NADIF_ITEM_NAME_MAX.
3. The maximum data length of a item is NADIF_ITEM_DATA_LENGTH_MAX.

3.12.3.2.21 Get Modem Configuration

API/Type Description	Doxygen Path
API to query Modem configuration	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_modem_get_config**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_MODEM_GET_CONFIG_RESP**

The notification of this API contains a list of modem configurations which are set.

Code Example: Get Dormant State modem configuration

```
NOTIFICATION_REQUEST_DATA rqst;
NADIF_STATUS status;
NADIF_MODEM_GET_CONFIG_LIST_REQ get_config_list = {0};
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

```
rqst.sender_id = client_id;
strncpy(get_config_list.name[0],NADIF_MODEM_DORMANT_STATE_NAME,NADIF_ITEM_NAME_MAX
-1);
status = NADIF_modem_get_config(&rqst, &get_config_list);
```

The configuration details are shared via a single notification “NADIF_MODEM_GET_CONFIG_RESP” for the Get Dormant State modem configuration.

Code Example: Get all modem configuration items

```
NOTIFICATION_REQUEST_DATA rqst;
NADIF_STATUS status;
NADIF_MODEM_GET_CONFIG_LIST_REQ get_config_list = {0};

rqst.sender_id = client_id;
strncpy(get_config_list.name[0], NADIF_MODEM_ALL_ITEM,NADIF_ITEM_NAME_MAX-1);
status = NADIF_modem_get_config(&rqst, &get_config_list);
```

The configuration details are shared via the notification “NADIF_MODEM_GET_CONFIG_RESP” for the All items modem configuration.

The maximum number of items per notification is NADIF_MAX_CONFIG_ITEMS_PER_REQUEST, So multiple notifications are delivered when the number of supported configuration items is greater than NADIF_MAX_CONFIG_ITEMS_PER_REQUEST. Otherwise, a single notification is delivered containing all supported configuration items.

3.12.3.2.22 Traffic Channel Dormancy Status

API/Type Description	Doxygen Path
Structure of notification data	Modules |-> Modem |-> NADIF |-> Modem Status |-> **NADIF_FEAT_MODEM_DORMANCY_STATUS**

The unsolicited notification **NADIF_FEAT_MODEM_DORMANCY_STATUS** provides the state of Modem traffic channel dormancy status.

3.12.3.3 Testing/Error Reporting

Not applicable

3.13 Standard eCall

3.13.1 Overview

3.13.1.1 Platform Content Overview

This section describes the support provided by NADIF for handling standard eCalls. A standard eCall is an emergency call made to a Public Safety Answering Point (PSAP) following the European Union eCall standards or the Russia Federation ERA/GLONASS standards.

The following features are provided by NADIF to support standard eCalls:

- Initiate a standard eCall-->allows the application to start a standard eCall.
- Provide MSD information-->allows the application to provide a MSD (Minimum Set of Data) “string” to be provided to the call center upon request during the eCall.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

- Get emergency call configuration settings-->allows the application to obtain the values of the emergency call configuration parameters.
- Set emergency call configuration settings-->allows the application to set the values of the emergency call configuration parameters.
- Get the current emergency call state and status-->allows the application to obtain the current emergency call status.
- Answer an incoming standard eCall-->allows the application to answer an incoming standard eCall.
- Cancel an incoming standard eCall-->allows the application to reject an incoming standard eCall.
- (eCall Only) SPY mode enable timer (T3242)-->allows the application to configure the timeout for SPY mode enable failsafe timer

3.13.1.2 Product Expectations

NADIF Client must include the header file `nadif_eCall.h` to access any API described in this section.

3.13.1.2.1 Initiate a standard eCall

NADIF Client is expected to use `NADIF_ecall_initiate`() to start a standard eCall.

3.13.1.2.2 Provide MSD information

The following expectations must be met by NADIF Client in order to provide MSD information that will be used during a standard eCall as designed:

1. NADIF Client must provide a formatted MSD “string” (array of binary data).
2. NADIF Client is expected to use `NADIF_ecall_update_msd`() to provide MSD information.

3.13.1.2.3 Get emergency call configuration settings

NADIF Client is expected to use `NADIF_ecall_get_cfg_parameters`() to obtain emergency call configuration settings.

3.13.1.2.4 Set emergency call configuration settings

NADIF Client is expected to use `NADIF_ecall_set_cfg_parameters`() to set values of emergency call configuration settings.

3.13.1.2.5 Get the current emergency call state and status

NADIF Client is expected to use `NADIF_ecall_get_call_info()` to query the current eCall state and eCall status.

3.13.1.2.6 Answer an incoming standard eCall

The following expectations must be met by NADIF Client in order to answer an incoming ecall as designed:

1. NADIF Client must have received an unsolicited notification `NADIF_FEAT_ECALL_STATUS` with a status of `NADIF_ECALL_INCOMING` containing the call ID.
2. NADIF Client is expected to use `NADIF_ecall_answer`() to answer the incoming eCall using the call ID obtained from `NADIF_ECALL_INCOMING` status.

3.13.1.2.7 Cancel an incoming standard eCall

The following expectations must be met by NADIF Client in order to cancel/reject an incoming eCall as designed:

1. NADIF Client must have received an unsolicited notification `NADIF_FEAT_ECALL_STATUS` with a status of `NADIF_ECALL_INCOMING` containing the call ID.
2. NADIF Client is expected to use `NADIF_ecall_cancel`() to cancel the incoming eCall using the call ID obtained from `NADIF_ECALL_INCOMING` status.

3.13.1.2.8 SPY mode enable timer (T3242)

The following expectations must be met by NADIF Client in order to enable automatic T3242 SPY mode enable timer operation when an eCall ends:

1. Modem must be configured as eCall only or in eCall only emulation mode.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

2. Persistence key `T3242_AUTO_ENABLE` must be set to enabled (1) to enable automatic T3242 timer operation (see TP_F101U-00026), default value is disabled (0).
3. Persistence key `T3242_EXPIRY_DURATION` must be set to desired T3242 timeout in seconds (see TP_F101U-00026), default value is 65520.
4. NADIF Client can use the unsolicited notification `NADIF_FEAT_TIMER_EXPIRY_NOTIFICATION` with indication of `NADIF_TIMER_T3242` expiry to know when SPY mode is automatically enabled due to T3242 timer expiration.

Notes:

1. When modem is configured for eCall only or in eCall only emulation mode, the modem will stay in callback mode and NOT enter SPY mode after an eCall ends. The modem will start the T3242 timer with a value specified in 3GPP TS 24.301 when an eCall ends irrespective of eCall connection success or failure. SPY mode will be reenabled when the timer expires.
2. The T3242 timer will be started with a timeout specified by `T3242_EXPIRY_DURATION` when `NADIF_net_set_spy_mode`() is invoked with a request to disable SPY mode. SPY mode will be reenabled when the timer expires.
3. T3242 timer operation/behavior described in the above notes is irrespective of `T3242_AUTO_ENABLE` setting. `T3242_AUTO_ENABLE` setting is effectively ignored.

3.13.1.2.9 LTE and 5G Enable/Disable during callback

NADIF Client is expected to use `NADIF_ecall_callback_mode_rat_config()` to disable the LTE (and 5G if applicable) for the scenario where it is required to ensure that only CS voice calls get connected, this is to avoid scenarios where IMS could be available and the network provider could establish the calls via VoLTE, which does not support inband MSD transmission.

3.13.1.3 Extending Platform Functionality

The following describes the platform configuration provided for the SPY mode enable timer (T3242):

`T3242_AUTO_ENABLE`

Refer to notes in section 3.13.1.2.8.

`T3242_EXPIRY_DURATION`

Configuration Type:	Run-Time (Persistency) Key-Value
Description:	SPY mode enable timer (T3242) timeout. When enabled, the timeout of the SPY mode enable timer (T3242) is set to the specified value upon timer start/restart (e.g., at eCall termination when `T3242_AUTO_ENABLE` is enabled, when SPY mode is disabled via `NADIF_net_set_spy_mode`()).
Values:	Integer (time in seconds): 1 to 65520 – timer timeout Platform Default: 65520
Attributes:	- Product able to override at compile time - Run-time changes take effect immediately (no reboot required)

Include:	`#include "nadif_cmd_network.h"`
	Definition for key use in application code:
	`#define NADIF_PERS_KEY_T3242_EXPIRY_DURATION "T3242_EXPIRY_DURATION"`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.13.2 Setup

3.13.2.1 Tools/Equipment Needed

Not applicable

3.13.2.2 Setup Instructions

Not applicable

3.13.2.3 Setup Troubleshooting

Not applicable

3.13.3 Usage

3.13.3.1 Basic Instructions

Not applicable

3.13.3.2 Detailed Instructions

3.13.3.2.1 Initiate a standard eCall

API/Type Description	Doxygen Path

API to initiate standard eCall	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> ECall`
	`|-> NADIF_ecall_initiate()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_RESULT`

This API allows NADIF Client to initiate an standard eCall . When the call type is `NADIF_ECALL_TYPE_EMER`, the call is placed to an emergency PSAP and the dial_string is ignored. When the call type is `NADIF_ECALL_TYPE_TEST`, the call is placed to the provided dial string.

The `auto_start_MSD` parameter selects the MSD transmission mode, either Pull or Push mode. In Push mode, the modem automatically sends the MSD to the PSAP once the data modem connection is established. In Pull mode, the modem waits for a MSD request from the PSAP before sending the MSD.

Note that the notification `NADIF_FEAT_STATUS_ECALL` is sent whenever the eCall status changes.

3.13.3.2.2 Provide MSD information

API/Type Description	Doxygen Path

API to provide the MSD	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> ECall`
	`|-> NADIF_ecall_update_msd()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_RESULT`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

This API allows NADIF Client to provide a formatted Minimum Set of Data (MSD) for the eCall. The provided MSD is sent to the PSAP via inband modem during the eCall. Refer to CSN EN 15722 "Intelligent transport systems – ESafety – ECall minimum set of data" for details on MSD content and format.

3.13.3.2.3 Get emergency call configuration settings

API/Type Description	Doxygen Path

API to get the values of eCall configuration settings	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> ECall`
	`|->`
	`NADIF_ecall_get_cfg_parameters()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> ECall`
	`|->` **`NADIF_ECALL_CFG_PARAMS_RESP`**

eCall configuration values structure	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> ECall`
	`|->` **`NADIF_ECALL_CFG_PARAMS`**

This API allows NADIF Client to query the values of various eCall configuration parameters. Parameter settings are provided in `NADIF_ECALL_CFG_PARAMS` structure.

3.13.3.2.4 Set emergency call configuration settings

API/Type Description	Doxygen Path

API to set the values of eCall configuration settings	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> ECall`
	`|->`
	`NADIF_ecall_set_cfg_parameters()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|->` **`NADIF_RESULT`**

eCall configuration values structure	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> ECall`
	`|->` **`NADIF_ECALL_CFG_PARAMS`**

This API allows NADIF Client to set the values of various eCall configuration parameters.

NOTE: If any parameter does not need to be changed, set the corresponding element in `NADIF_ECALL_CFG_PARAMS` structure to a value of `NADIF_ECALL_CFG_PARAM_NOT_SET` prior to invoking `NADIF_ecall_set_cfg_parameters()`.

3.13.3.2.5 Get the current emergency call state and status

API/Type Description	Doxygen Path
API to query eCall state and status	`Modules` `|-> Modem`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|> NADIF |-> ECall |-> ***NADIF_ecall_get_call_info()***
Notification data type of API	Modules |-> Modem |-> NADIF |-> ECall |-> **NADIF_ECALL_INFO_RESP**

This API allows NADIF Client to obtain the current eCall state and status. NADIF Client must evaluate the `NADIF_ECALL_INFO_RESP` to identify the current state and status of the eCall.

The eCall status is reported by the unsolicited notification `NADIF_FEAT_STATUS_ECALL` or by querying via `NADIF_ecall_get_call_info()`.

3.13.3.2.6 Answer an incoming standard eCall

API/Type Description	Doxygen Path
API to answer incoming standard eCall	Modules |-> Modem |-> NADIF |-> ECall |-> ***NADIF_ecall_answer()***
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to answer an incoming standard eCall. Note the unsolicited notification `NADIF_FEAT_STATUS_ECALL` is reported when there is an incoming eCall (`NADIF_ECALL_INCOMING`). This notification also informs the call ID, which must be used to answer the eCall by `NADIF_ecall_answer()`.

3.13.3.2.7 Cancel an incoming standard eCall

API/Type Description	Doxygen Path
API to cancel an incoming standard eCall	Modules |-> Modem |-> NADIF |-> ECall |-> ***NADIF_ecall_cancel()***
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to cancel/reject an incoming standard eCall. Note the unsolicited notification `NADIF_FEAT_STATUS_ECALL` is reported when there is an incoming eCall (`NADIF_ECALL_INCOMING`). This notification also informs the call ID, which must be used to cancel the eCall by `NADIF_ecall_cancel()`.

3.13.3.2.8 SPY mode enable timer (T3242)

API/Type Description	Doxygen Path
T3242 expiry unsolicited notification	Modules |-> Modem |-> NADIF

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

API/Type Description	Doxygen Path
	|-> ECall |-> ***NADIF_FEAT_TIMER_EXPIRY_NOTIFICATION***

The following describes the platform configuration provided for this feature (see TP_F101U-00026):

Persistence Key	Description	Allowed values
T3242_AUTO_ENABLE	**Automatic SPY mode enable timer (T3242) enable.** A value of disabled indicates that NADIF will not start T3242 timer after eCall termination, so will not re-enable SPY mode automatically. A value of enabled indicates that NADIF will start T3242 timer automatically after eCall ends and will re-enable SPY mode upon timer expiry. Note: An out of range value will be treated as default value when NADIF reads this persistence key value.	0 [Disabled] 1 [Enabled] Default: 0
T3242_EXPIRY_DURATION	**SPY mode enable timer (T3242) timeout**. Timeout in seconds for the NADIF T3242 timer when enabled. Note: An out of range value will be treated as default value when NADIF reads this persistence key value.	1 to 65520 [timeout in seconds] Default: 65520

Automatic SPY mode enable timer operation

The automatic SPY mode enable fail-safe timer is a back up timer implemented in the modem to help ensure that SPY mode is re-enabled within a reasonable time period after an eCall ends when the modem is configured for eCall only mode or in eCall only emulation mode. A SPY mode enable timer (T3242) is started automatically when an eCall ends with a timeout value specified in 3GPP TS 24.301. SPY mode is enabled (i.e., modem network deregistration, scan for available networks but do not register) and an unsolicited NADIF_FEAT_TIMER_EXPIRY_NOTIFICATION notification is provided to NADIF clients upon timer expiry.

Notes regarding automatic SPY mode enable fail-safe timer behavior in eCall only mode or eCall only emulation mode:

1. The timer will be restarted (i.e., timeout reset to value specified in 3GPP TS 24.301) if the timer is already running when an eCall ends.
2. The timer will be started for all mobile originated or mobile terminated call terminations, whether or not there is a successful connection. Call terminations include call drop and call hang-up.

SPY mode enable fail-safe timer interaction with NADIF_net_set_spy_mode()

The SPY mode enable fail-safe timer is also utilized to prevent inadvertent disabling of SPY mode for extended periods of time. A SPY mode enable timer (T3242) will be started when NADIF_net_set_spy_mode() API is invoked with a request to disable SPY mode, an eCall is not active, and the timer is not running. The timer will be started with a timeout value specified by T3242_EXPIRY_DURATION persistence key. SPY mode is enabled (i.e., modem network deregistration, scan for available networks but do not register) and an unsolicited NADIF_FEAT_TIMER_EXPIRY_NOTIFICATION notification is provided to NADIF clients upon timer expiry.

Notes regarding SPY mode enable fail-safe timer behavior with NADIF_net_set_spy_mode():

1. A request to disable SPY mode via NADIF_net_set_spy_mode() when the timer is already running results in an error response and the timer is not restarted.
2. The timer will be stopped if the timer is running for any reason when NADIF_net_set_spy_mode() is invoked with an enable SPY mode request (e.g., timer is cancelled).

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.13.3.2.9 LTE and 5G Enable/Disable during callback

API/Type Description	Doxygen Path

API to cancel an incoming standard eCall	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> ECall`
	`|->`
	`NADIF_ecall_callback_mode_rat_config ()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_RESULT`

It is recommended NADIF Client is to use `NADIF_ecall_callback_mode_rat_config()` to disable the LTE (and 5G if applicable) for the scenario where T9 is running when there is a possibility for the PSAP to callback the IVS and it is mandatory to establish the call via CS network in order to be able to perform an MSD transmission via inband tones.

Note: If the modem chipset supports 5G cellular technology, this API will disable/enable both technologies together.

3.13.3.3 Testing/Error Reporting

Not applicable

3.14 Private eCall

3.14.1 Overview

3.14.1.1 Platform Content Overview

This section describes the support provided by NADIF for MSD inband modem transmission during private eCall.

This is used for private OEM emergency call use case.

See the following features supported by NADIF:

- Private eCall Voice Dial Inband Modem-->dial a voice with in-band modem feature.
- Set data to transmit via Inband Modem→ used for the in-band modem data payload where the data element is a binary string.
- Answer incoming private eCall→To answer incoming private eCall.

3.14.1.2 Product Expectations

NADIF Client must include the header file `nadif_voice.h` to access any API described in this section.

3.14.1.2.1 Private eCall Voice Dial Inband Modem

The following expectations must be met by NADIF Client in order to dial a private eCall as designed:

1. The modem must be registered on the cellular network.
2. NADIF Client is expected to use NADIF_voice_inband_modem_dial() to start a private eCall.

3.14.1.2.2 Set data to transmit via Inband Modem

The following expectations must be met by NADIF Client in order to provide private data that will be sent via inband modem during the private eCall as designed:

1. NADIF client must provide a formatted “string” for inband modem transmission (array of binary data).

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

2. NADIF Client is expected to use `NADIF_voice_inband_modem_set_data`() to provide private data information.

3.14.1.2.3 Answer incoming private eCall

The following expectations must be met by NADIF Client in order to answer an incoming private eCall as designed:

1. NADIF Client must have received an unsolicited notification `NADIF_FEAT_STATUS_VOICE` with the status of `NADIF_CALL_INCOMING`.
2. NADIF Client must determine that the incoming call is a private eCall requiring private data transmission via inband modem and not a standard voice call.
3. NADIF Client must know the call ID of the incoming private eCall. The incoming `call_id` is informed by `NADIF_FEAT_STATUS_VOICE`.
4. NADIF Client is expected to use `NADIF_voice_inband_modem_answer`() to answer an incoming private eCall.

3.14.1.3 Extending Platform Functionality

Not applicable

3.14.2 Setup

3.14.2.1 Tools/Equipment Needed

Not applicable

3.14.2.2 Setup Instructions

Not applicable

3.14.2.3 Setup Troubleshooting

Not applicable

3.14.3 Usage

3.14.3.1 Basic Instructions

Not applicable

3.14.3.2 Detailed Instructions

3.14.3.2.1 Private eCall Voice Dial Inband Modem

API/Type Description	Doxygen Path

API to dial an private eCall	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Voice`
	`|->`
	`NADIF_voice_inband_modem_dial()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|->` **`NADIF_RESULT`**

This API allows NADIF Client to initiate and dial a private eCall to a specific number. Note that the notification `NADIF_FEAT_STATUS_VOICE` is sent whenever the call status changes.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.14.3.2.2 Set data to transmit via Inband Modem

API/Type Description	Doxygen Path
API to set data to transmit	Modules |-> Modem |-> NADIF |-> Voice |-> ***NADIF_voice_inband_modem_set_data()***
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to provide private data that will be sent via inband modem during a private eCall. The private data is a binary array that must be formatted per OEM specifications before providing to NADIF.

3.14.3.2.3 Answer incoming private eCall

API/Type Description	Doxygen Path
API to answer incoming private eCall	Modules |-> Modem |-> NADIF |-> Voice |-> ***NADIF_voice_inband_modem_answer()***
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

This API allows NADIF Client to answer an incoming private eCall. Note the unsolicited notification `NADIF_FEAT_STATUS_VOICE` is reported when there is an incoming call. This notification also informs the call ID, which must be used to answer the call by `NADIF_voice_inband_modem_answer`().

Note: NADIF does not differentiate between a standard voice call and a private eCall. So, NADIF Client must determine when an incoming call is a private eCall and use `NADIF_voice_inband_modem_answer`().

3.14.3.3 Testing/Error Reporting

Not applicable

3.15 Phone Book Manager

3.15.1 Overview

3.15.1.1 Platform Content Overview

This section describes the support provided by NADIF related to phone book interfaces. See the following features supported by NADIF:

- Read all the emergency number list from the phonebook.

3.15.1.2 Product Expectations

NADIF Client must include the header file nadif_pbm.h to access any API described in this section.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.15.1.2.1 Indication for emergency list update

NADIF Client is expected to be notified through the unsolicited notification NADIF_FEAT_PBM_EMERGENCY_LIST_INFO whenever an emergency number list gets updated in phonebook.

3.15.1.2.2 Read Emergency Number

NADIF Client is expected to use NADIF_pbm_get_emergency_list() to read all emergency numbers list from the Phonebook.

3.15.1.3 Extending Platform Functionality

Not applicable

3.15.2 Setup

3.15.2.1 Tools/Equipment Needed

Not applicable

3.15.2.2 Setup Instructions

Not applicable

3.15.2.3 Setup Troubleshooting

Not applicable

3.15.3 Usage

3.15.3.1 Basic Instructions

Not applicable

3.15.3.2 Detailed Instructions

3.15.3.2.1 Indication for emergency list update

API/Type Description	Doxygen Path

Structure of notification data	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> PBM`
	`|-> NADIF_PBM_EMERGENCY_LIST`

This API allows NADIF Client to receive unsolicited indication whenever an emergency number list gets updated in phonebook.

3.15.3.2.2 Read Emergency number

API/Type Description	Doxygen Path

API to read all emergency numbers	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> PBM`
	`|-> NADIF_pbm_get_emergency_list()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> PBM`
	`|-> NADIF_PBM_GET_EMERGENCY_LIST_RESP`

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

This API allows NADIF Client to read all the supported emergency numbers list from the Phonebook. “NADIF_FEAT_PBM_GET_EMERGENCY_LIST” response is received when this API is processed by Modem.

3.15.3.3 Testing/Error Reporting

Not applicable

3.16 Network Recovery Mechanism

3.16.1 Overview

3.16.1.1 Platform Content Overview

This section describes the support provided by NADIF related to Recovery Mechanism in scenarios of attach failures due to reject causes. See the following scenarios of Recovery Mechanism supported by NADIF which might not be compliant with 3GPP:

- Power save mode from no coverage.
- SIM Invalid due to reject causes.
- Re attach to already Forbidden PLMN.

3.16.1.2 Product Expectations

3.16.1.2.1 Power Save mode from no coverage.

The product application should toggle airplane mode to attempt recovery when the modem enters power save mode.

3.16.1.2.2 SIM Invalid due to reject causes.

The product application should perform the SIM power cycle mechanism when SIM invalid cases occurs due to various reject causes.

3.16.1.2.3 Re attach to already Forbidden PLMN.

The product application should perform the SIM power cycle mechanism when MNO reject allows the connection to the Modem, there will be a need to update FPLMN list.

3.16.1.3 Extending Platform Functionality

Not applicable

3.16.2 Setup

3.16.2.1 Tools/Equipment Needed

Not applicable

3.16.2.2 Setup Instructions

Not applicable

3.16.2.3 Setup Troubleshooting

Not applicable

3.16.3 Usage

3.16.3.1 Basic Instructions

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.16.3.2 Detailed Instructions

3.16.3.2.1 Power save mode from no coverage

API/Type Description	Doxygen Path
API to set modem operating mode	Modules |-> Modem |-> NADIF |-> **NADIF_modem_set_operating_mode()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> **NADIF_SET_OPERATING_MODE_RESP**
Notification data type of API	Modules |-> Modem |-> NADIF |-> **NADIF_FEAT_STATUS_NETWORK**

This scenario for NAD Recovery Mechanism explains that, as per the 3GPP standard, the Modem can enter the Power Save Mode whenever it is unable to register into a network – e.g. in areas of no network coverage.

Power Save Mode status will be provided in response to NADIF get registration status request and via an unsolicited NADIF_FEAT_STATUS_NETWORK notification whenever there is a change in the network registration status.

If the registration status is NADIF_REG_STAT_NOT_REGISTERED, powersave_mode will indicate TRUE when the modem is in deep sleep i.e., it stops searching for network to regain the lost service to save battery consumption. Then no matter which network type is registered,powersave_mode will be true if modem gets into power save mode.

So in order to achieve this:

a. Modem enters the Power Save Mode in case of no network coverage.
b. Toggle Airplane mode.

NADIF_modem_set_operating_mode API allows NADIF Client to set the operating mode and receives the current mode in response.

3.16.3.2.2 SIM invalid due to reject causes

API/Type Description	Doxygen Path
API to set SIM Power	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_set_power()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**
Notification data type of API	Modules |-> Modem |-> NADIF |-> **NADIF_FEAT_STATUS_NETWORK**

This scenario for NAD Recovery Mechanism explains about SIM invalid cases occurs due to various reject causes.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

These reject cause code info are fetched from NADIF_FEAT_STATUS_NETWORK, through Notification Data Type NADIF_REG_STATUS_DATA.

Reject Cause Code 2:IMSI unknown in HLR (Causes related to MS identification)

Reject Cause Code 3:Illegal MS (Causes related to MS identification)

Reject Cause Code 6:Illegal ME (Causes related to MS identification)

Reject Cause Code 7:GPRS services not allowed (Additional cause codes for GMM)

Reject Cause Code 8:GPRS services and non-GPRS services not allowed (Additional cause codes for GMM)

Reject Cause Code 32:"Service option not supported" (GPRS specific cause values for session management)

The reject cause code is sent from the MNO to the Modem when the SIM card information is not recognized by the MNO.

So in order to achieve this:

a. Sync with MNO if SIM Card is allowed to have the service, if SIM Card is just allowed then perform the Recovery mechanism.
b. SIM requires a power cycle[Recovery Mechanism by SIM RESET].
c. Turn OFF the SIM card. Execute NADIF_sim_set_power API with NADIF_SIM_POWER_DOWN.
d. NADIF_FEAT_STATUS_SIM unsolicited notification will be send on power down completion.
e. Turn ON the SIM card. Execute NADIF_sim_set_power API with NADIF_SIM_POWER_UP.
f. NADIF_FEAT_STATUS_SIM unsolicited notification will be send on power on completion to indicate NADIF_SIM_READY.

NADIF_sim_set_power API allows NADIF Client to power up or down the SIM. The SIM is powered by default at modem power up.

The SIM power down can be performed when the modem is switched off or when it is set to Airplane mode.

Refer: Section 3.9.3.2.1

3.16.3.2.3 Re-attach to already forbidden PLMN

API/Type Description	Doxygen Path
API to update FPLMN list	Modules |-> Modem |-> NADIF |-> SIM |->**NADIF_sim_write_transparent_file()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> SIM |->**NADIF_SIM_WRITE_TRANSPARENT_FILE_RESP**
Notification data type of API	Modules |-> Modem |-> NADIF |-> **NADIF_FEAT_STATUS_NETWORK**
API to set SIM Power	Modules |-> Modem |-> NADIF |-> SIM |-> **NADIF_sim_set_power()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_RESULT**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

A MNO attachment rejection may result in the MNO being added to the Forbidden PLMN list (FPLMN).

In this scenario, the modem will no longer attempt to attach to the MNO network. If it is known that this is a temporary condition and the MNO should allow modem connection, the FPLMN list needs to be cleared.

So in order to achieve this:

a. When the modem is registered to a MNO and NADIF_IS_SYS_FORBIDDEN in Notification Data Type NADIF_REG_STATUS_DATA indicates that NADIF is not allowed to work anymore in the MNO, which will be received in unsolicited notification through NADIF_FEAT_STATUS_NETWORK with forbidden item NADIF_IS_SYS_FORBIDDEN.
b. At this point the sequence to clear FPLMN list can be initiated.
c. Then update FPLMN by writing file_id=0x6F7B (FPLMN) in NADIF_sim_write_transparent_file API
d. SIM requires a power cycle[Recovery Mechanism by SIM RESET].
e. Turn OFF the SIM card. Execute NADIF_sim_set_power API with NADIF_SIM_POWER_DOWN.
f. NADIF_FEAT_STATUS_SIM unsolicited notification will be send on power down completion.
g. Turn ON the SIM card. Execute NADIF_sim_set_power API with NADIF_SIM_POWER_UP.
h. NADIF_FEAT_STATUS_SIM unsolicited notification will be send on power on completion to indicate NADIF_SIM_READY.

NADIF_sim_write_transparent_file API allows NADIF Client to have write access to any elementary transparent file in the SIM card and access by the path. NADIF client can use this API for writing/updating FPLMN by using file_id=0x6F7B(FPLMN).

Refer: Section 3.9.3.2.7

Note: When multiple FPLMN scan is attempted, it has to be stopped.

3.16.3.3 Testing/Error Reporting

Not applicable

3.17 Power-up Operating Mode

3.17.1 Overview

3.17.1.1 Platform Content Overview

This section describes the support provided by NADIF to configure on demand the modem power up operating mode. The selected power up operating mode will be used on modem power up.

See the following features supported by NADIF:

- Set the Power-up operating mode.
- Get the Power-up operating mode.

3.17.1.2 Product Expectations

NADIF Client must include the header file `notification.h` to access any API described in this section.

3.17.1.2.1 Set Power-up operating mode

The following expectations must be met by NADIF Client in order to set to NADIF_MODE_O using API NADIF_modem_set_power_up_operating_mode:

- By default, power up mode will be in NADIF_MODE_LOW mode.
- Power up mode is set to NADIF_MODE_ON using API NADIF_modem_set_power_up_operating_mode.
- After a reboot the modem will be set to NADIF_MODE_ON, even if the current mode is NADIF_MODE_LOW.

3.17.1.2.2 Get Power-up operating mode.

The following expectations must be met by NADIF Client in order get the power-up operating mode setting using API NADIF_modem_get_power_up_operating_mode:

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

- The power-up mode selected will be returned from the NADIF_modem_get_power_up_operating_mode API.

3.17.1.3 Extending Platform Functionality

Not applicable

3.17.2 Setup

3.17.2.1 Tools/Equipment Needed

Not applicable

3.17.2.2 Setup Instructions

Not applicable

3.17.2.3 Setup Troubleshooting

Not applicable

3.17.3 Usage

3.17.3.1 Basic Instructions

Not applicable

3.17.3.2 Detailed Instructions

3.17.3.2.1 Set power-up operating mode.

API/Type Description	Doxygen Path

API to set power-up operating mode	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF_modem_set_power_up_operating_mode()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_RESULT`

This API allows NADIF Client to set the Power-up operating mode using API `NADIF_modem_set_power_up_operating_mode`, which will be by default set in NADIF_MODE_LOW mode. After a reboot the modem will be set to selected power-up operating mode.

There are 2 modes available for setting the modem power-up operating mode:

- NADIF_MODE_LOW(Default)
- NADIF_MODE_ON

NOTES:

1. The default modem operating mode on first power up after a fresh flashing of the modem will be Low power mode,
2. Modem Factory test mode (FTM) has priority over this configuration. If the modem is in FTM mode, the modem will remain in FTM mode at power up.
3. If the modem does not have a proper RF Calibration and selected power-up operating mode is NADIF_MODE_ON, periodic modem crashes might be observed at power up.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.17.3.2.2 Get Power-up operating mode.

API/Type Description	Doxygen Path

API to get power-up operating mode	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF_modem_get_power_up_operating_mode()`

Notification data type of API	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF_GET_OPERATING_MODE_RESP`

This API allows NADIF Client to get the selected Power-up operating mode using API NADIF_modem_get_power_up_operating_mode for which we get the response through NADIF_GET_OPERATING_MODE_RESP.

3.17.3.3 Testing/Error Reporting

Not applicable

3.18 RTT (Real Time Text)

3.18.1 Overview

Real-time text (RTT) is text transmitted instantly as it is typed or created. Recipients can immediately read the message while it is being written, without waiting.

Real-time text is used for conversational text, in collaboration, and in live captioning. Real-time text is frequently used by the deaf. Real-time text allows the other person to read immediately, without waiting for the sender to finish composing his or her sentence/message. This allows conversational use of text, much like a hearing person can listen to someone speaking in real-time.

3.18.1.1 Platform Content Overview

This section describes the support provided by NADIF for incoming and outgoing RTT (Real Time Text) calls via cellular network. See the following features supported by NADIF:

- Enable or disable Real Time Text(RTT) device setting - request to enable or disable RTT feature.
- Initiate a voice call with RTT - request to initiate a RTT call specifying a calling number.
- Answer a RTT call - request to answer an incoming call (ringing).
- Send RTT messages during an active RTT call - request to send Real Time Text(RTT) messages while a call is active.
- Get notified about incoming RTT messages during an active RTT call – receive a notification for incoming RTT messages during an active RTT call.
- Upgrade/Downgrade a voice call to RTT call and vice versa – request to upgrade to an RTT call from normal voice call or downgrade to normal voice call from an active RTT call.
- Get notified about incoming upgrade/downgrade request from peer during an active call – request to receive a notification about incoming RTT call upgrade/downgrade request.

3.18.1.2 Product Expectations

NADIF Client must include the header file `nadif_voice.h` to access any API described in this section.

3.18.1.2.1 Dial an outgoing RTT call

The following expectations must be met by NADIF Client in order to dial a RTT call as designed:

1. The modem must be registered with the LTE or 5G network.
2. NADIF Client is expected to use `NADIF_voice_dial_ext()` to dial an outgoing RTT call.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.18.1.2.2 Answer an incoming RTT call

The following expectations must be met by NADIF Client in order to answer an incoming call as designed:

1. NADIF Client must have received an unsolicited notification `NADIF_FEAT_STATUS_VOICE` with the status of `NADIF_CALL_INCOMING`.
2. NADIF Client must know the call ID of the incoming call. The incoming `call_id` is informed by `NADIF_FEAT_STATUS_VOICE`.
3. NADIF Client must know the call type of the incoming call. The incoming `NADIF_VOICE_CALL_TYPE_RTT` is informed by `NADIF_FEAT_STATUS_VOICE`.
4. NADIF Client is expected to use `NADIF_voice_answer_ext()` to answer an incoming RTT call.

3.18.1.2.3 Send/Receive a RTT message during active RTT call

The following expectations must be met by NADIF Client in order to send/receive a RTT message during an ongoing RTT call as designed:

1. NADIF Client is expected to use `NADIF_voice_rtt_send_msg()` to send a RTT message during a call.
2. NADIF Client is is expected to be notified through the unsolicited notification `NADIF_FEAT_STATUS_VOICE_RTT_MSG` when an incoming RTT message is received during a call.

3.18.1.2.4 Send/Receive a request to upgrade active voice call to RTT call

The following expectations must be met by NADIF Client in order to send a request to upgrade an active voice call to RTT call as designed:

1. NADIF Client is expected to be notified through the unsolicited notification `NADIF_FEAT_STATUS_VOICE` when a request to upgrade a voice call to RTT call is received. NADIF Client is expected to use `NADIF_voice_manage_calls()` to accept/reject the upgrade request.
2. NADIF Client is expected to use `NADIF_voice_manage_calls()` to request upgrade of an ongoing voice call to RTT call.

3.18.1.2.5 Send/Receive a request to downgrade active RTT call to voice call

The following expectations must be met by NADIF Client in order to send a request to downgrade an active RTT call to voice call as designed:

1. NADIF Client is expected to be notified through the unsolicited notification `NADIF_FEAT_STATUS_VOICE` when a RTT call is downgraded to a voice call by the network. NADIF Client may use `NADIF_voice_manage_calls()` to request upgrade to RTT call if downgrade is not desired.
2. NADIF Client is expected to use `NADIF_voice_manage_calls()` to request downgrade of an ongoing RTT call to voice call.

3.18.1.3 Extending Platform Functionality

Not applicable

3.18.2 Setup

3.18.2.1 Tools/Equipment Needed

Not applicable

3.18.2.2 Setup Instructions

Not applicable

3.18.2.3 Setup Troubleshooting

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.18.3 Usage

3.18.3.1 Basic Instructions

Not applicable

3.18.3.2 Detailed Instructions

3.18.3.2.1 Dial an outgoing RTT call

API/Type Description	Doxygen Path

API to dial	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> Voice`
	`  |-> NADIF_voice_dial_ext()`

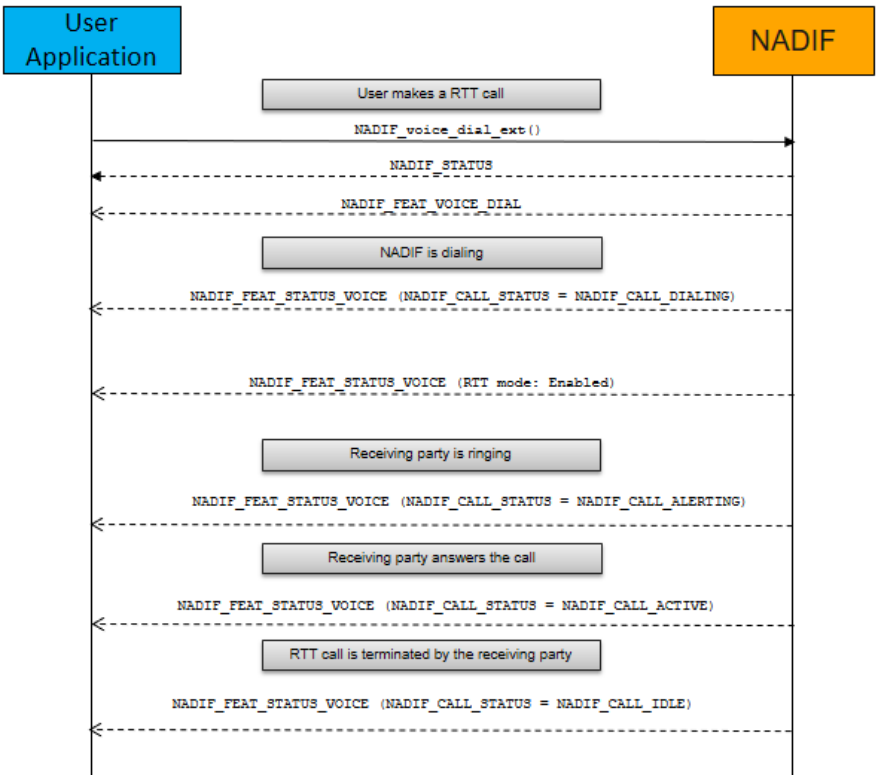
Notification data type of API	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> NADIF Common Types`
	`  |-> NADIF_FEAT_STATUS_VOICE`

This API allows NADIF Client to initiate and dial an outgoing RTT call to a specific number. Note that the notification `NADIF_FEAT_STATUS_VOICE` is sent whenever the call status changes.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

See a typical use case for an outgoing RTT call in the following MSC:



3.18.3.2.2 Receive an incoming RTT call

API/Type Description	Doxygen Path

API to dial	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> Voice`
	`  |-> **NADIF_voice_answer_ext()**`

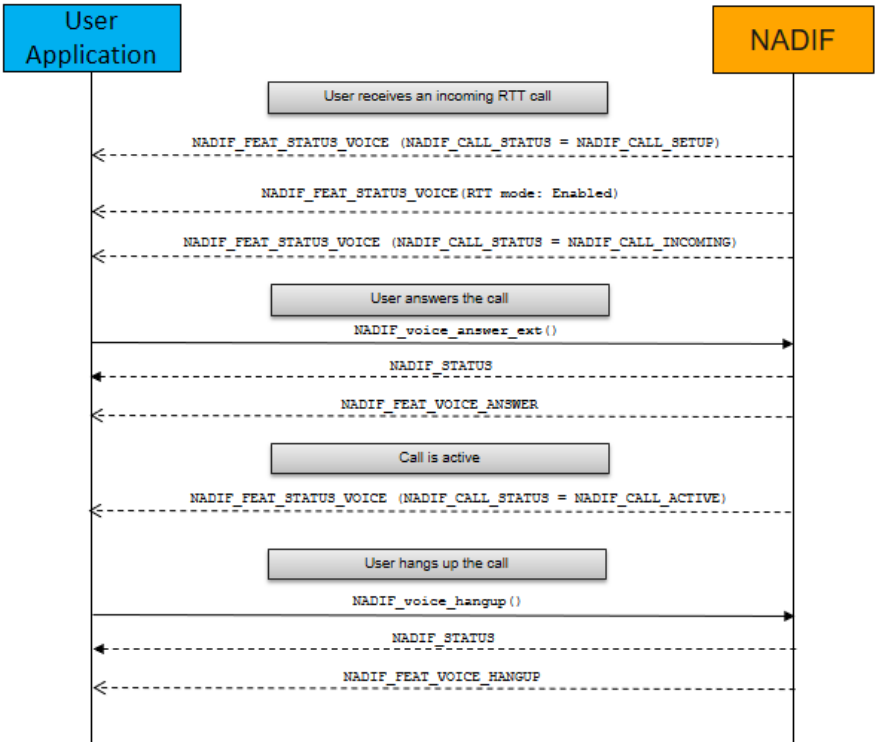
Notification data type of API	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> NADIF Common Types`
	`  |-> **NADIF_FEAT_STATUS_VOICE**`

This API allows NADIF Client to answer an incoming RTT call. Note the unsolicited notification `NADIF_FEAT_STATUS_VOICE` is reported when there is an incoming RTT call (ringing). This notification also informs the call ID, which must be used to answer the call by `NADIF_voice_answer_ext()`.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

See a typical use case for an incoming RTT call in the following MSC:



3.18.3.2.3 Receive an incoming RTT call while a voice call is active

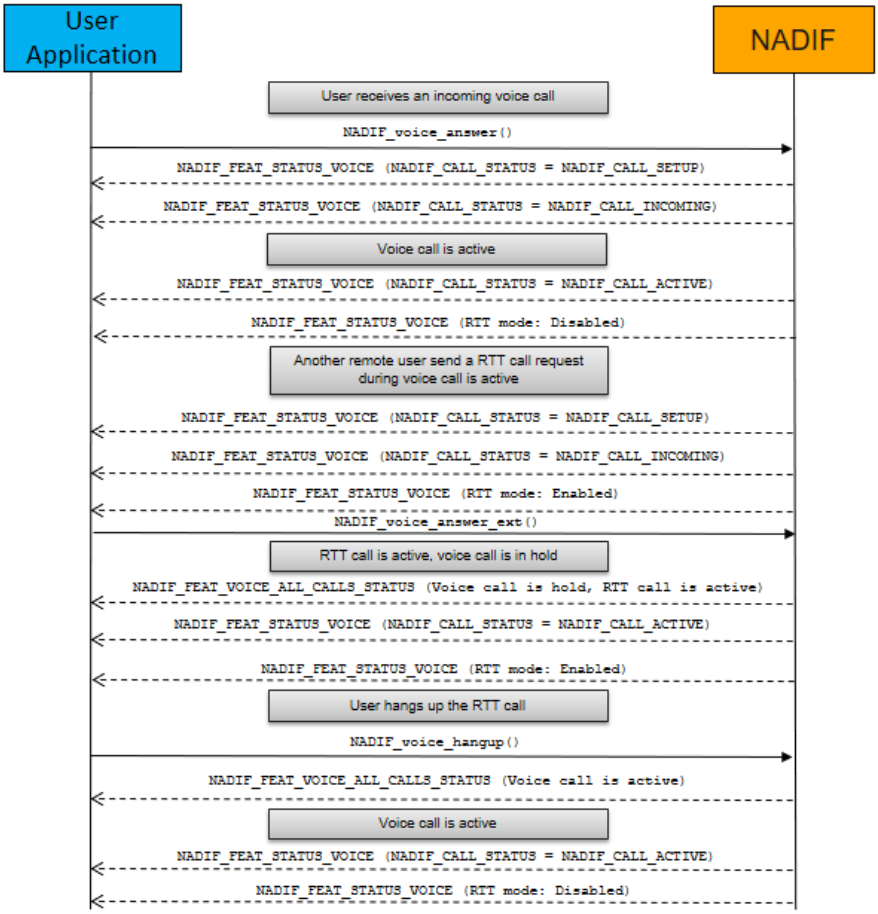
API/Type Description	Doxygen Path
API to dial	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_voice_answer_ext()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_FEAT_STATUS_VOICE**

This API allows NADIF Client to answer an incoming RTT call. Note the unsolicited notification `NADIF_FEAT_STATUS_VOICE` is reported when there is an incoming RTT call (ringing). This notification also informs the call ID, which must be used to answer the call by `NADIF_voice_answer_ext()`.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

See a typical use case for an incoming RTT call in the following MSC:



3.18.3.2.4 Send/Receive a RTT message during an active RTT call

API/Type Description	Doxygen Path
API to dial	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_voice_rtt_send_msg()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF_Common_Types

© Continental Automotive GmbH 2019
Confidential and Proprietary

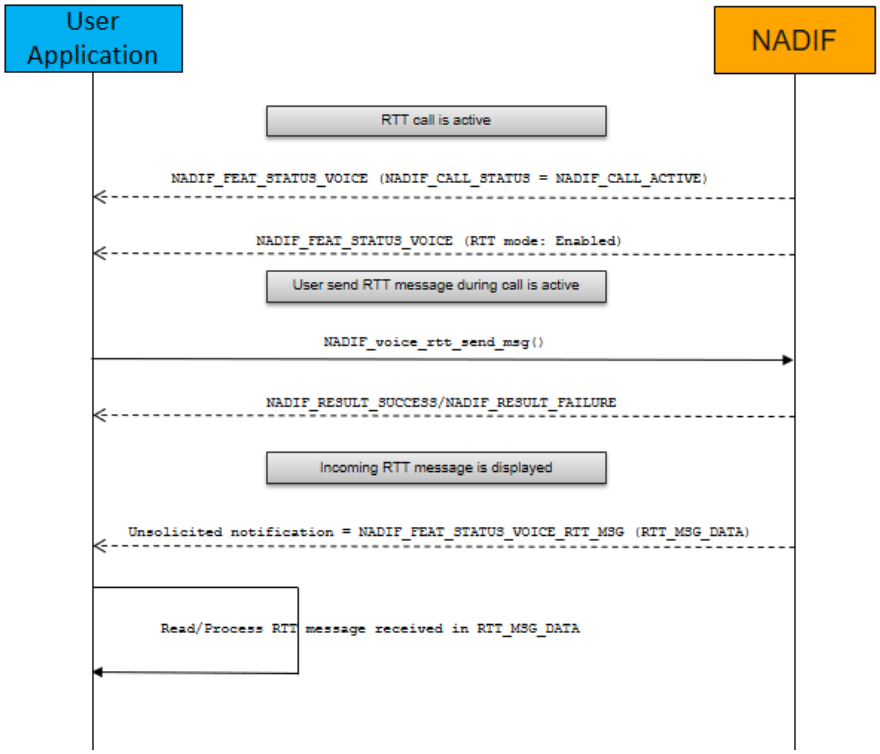
Internal

API/Type Description	Doxygen Path
	`|-> NADIF_FEAT_STATUS_VOICE_RTT_MSG`

Call status notification	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> NADIF Common Types`
	`|-> NADIF_FEAT_STATUS_VOICE`

This API allows NADIF Client to send a RTT message during an active RTT call. Note the notification `NADIF_FEAT_STATUS_VOICE_RTT_MSG` is reported when there is an incoming RTT message.

See a typical use case for sending/receiving RTT message during an active RTT call in the following MSC:



3.18.3.2.5 Upgrade to RTT call from active voice call

API/Type Description	Doxygen Path

API to dial	`Modules`
	`|-> Modem`
	`|-> NADIF`
	`|-> Voice`
	`|-> NADIF_voice_manage calls()`

© Continental Automotive GmbH 2019
Confidential and Proprietary

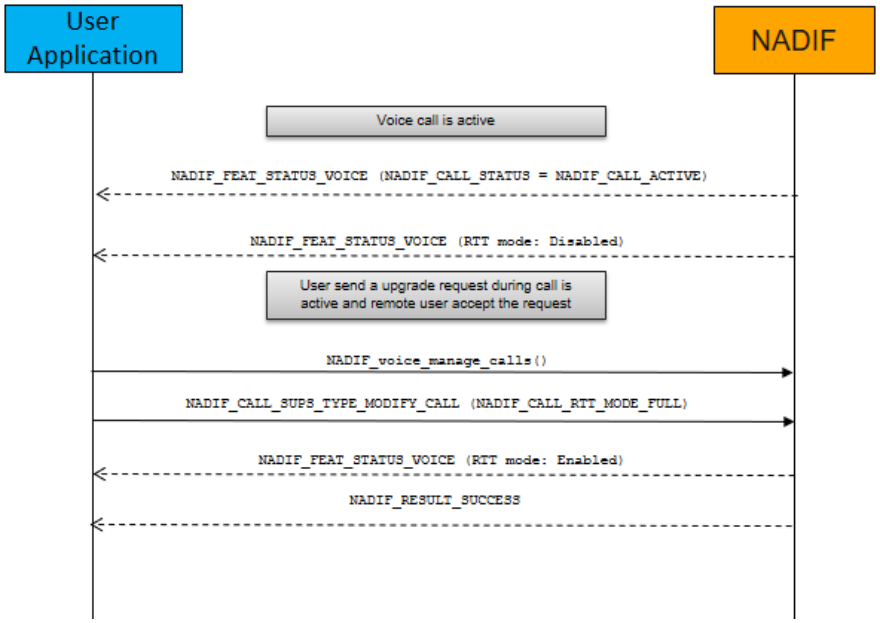
Internal

API/Type Description	Doxygen Path

Notification data type of API	`Modules`
	`  |-> Modem`
	`  |-> NADIF`
	`  |-> NADIF Common Types`
	`  |-> NADIF_FEAT_STATUS_VOICE`

This API allows NADIF Client to upgrade an active voice call to RTT call. Note: the unsolicited notification `NADIF_FEAT_STATUS_VOICE` is reported when there is an incoming RTT upgrade request.

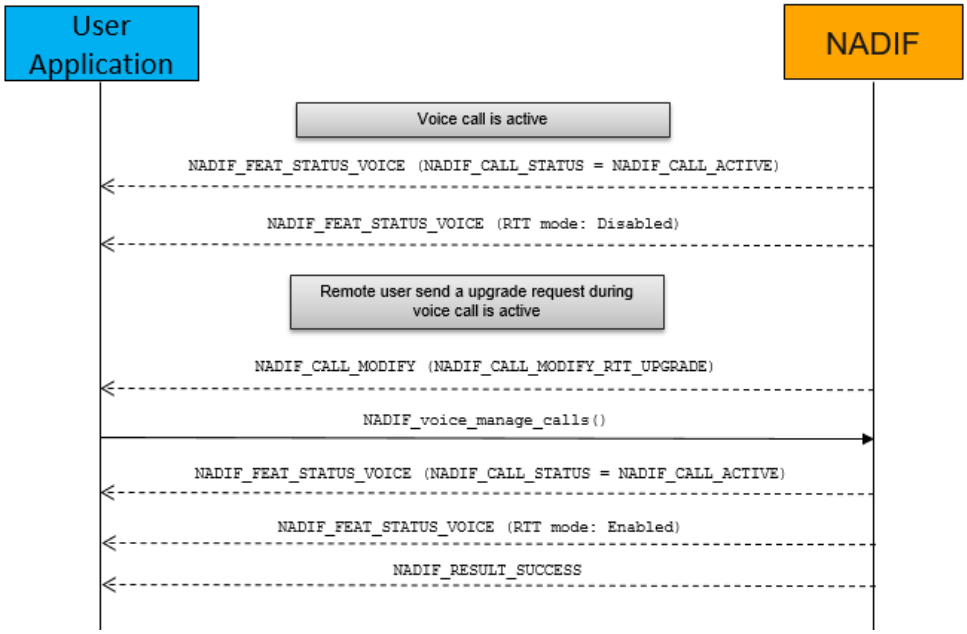
See a typical use case for an outgoing request to upgrade voice call to RTT call in the following MSC:



© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

See a typical use case for an incoming request to upgrade voice call to RTT call in the following MSC:



3.18.3.2.6 Downgrade to voice call from active RTT call

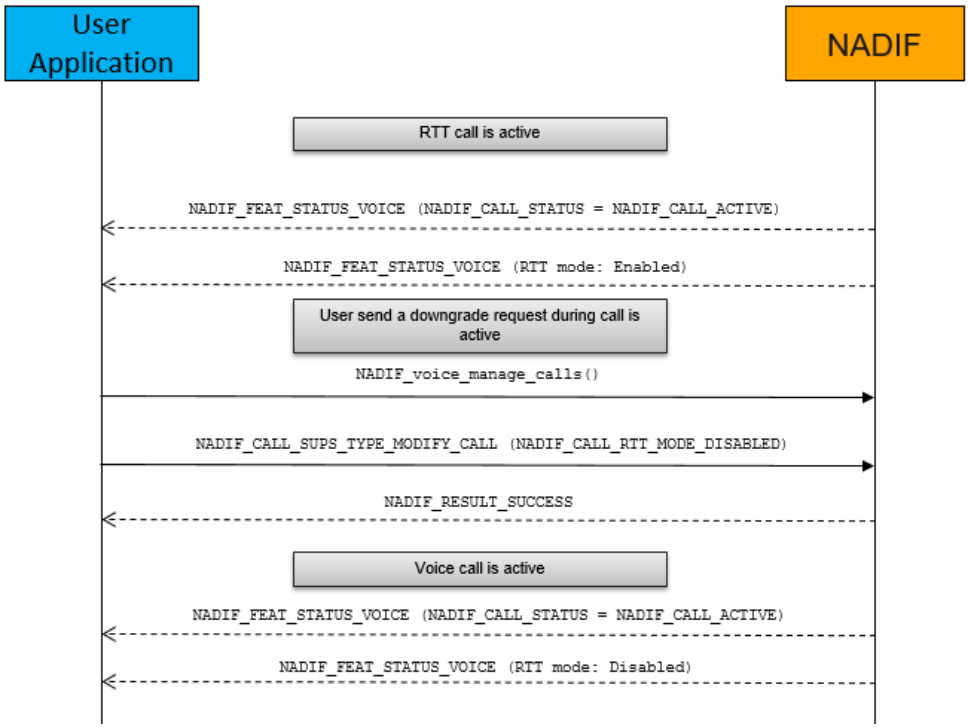
API/Type Description	Doxygen Path
API to dial	Modules |-> Modem |-> NADIF |-> Voice |-> **NADIF_voice_manage_calls()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> **NADIF_FEAT_STATUS_VOICE**

This API allows NADIF Client to downgrade an active RTT call to voice call. Note: the unsolicited notification `NADIF_FEAT_STATUS_VOICE` is reported when there is an incoming call.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

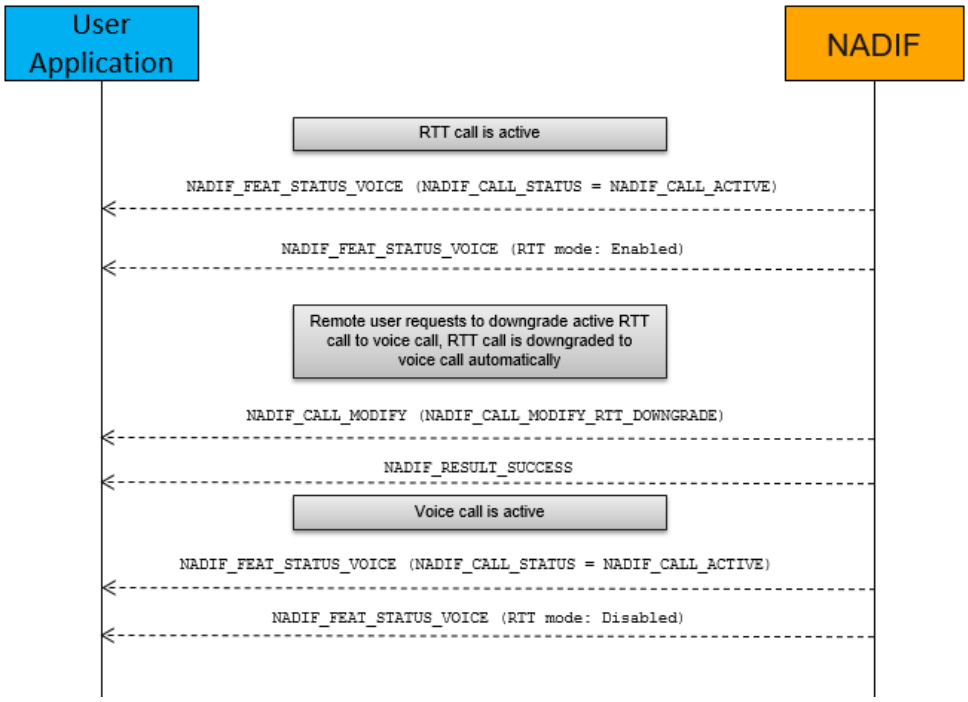
See a typical use case for an outgoing request to downgrade RTT call to voice call in the following MSC:



© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

See a typical use case for an incoming notification of RTT call downgraded to voice call in the following MSC:



3.18.3.3 Testing/Error Reporting

Not applicable

3.19 Persistence configuration

3.19.1 Overview

3.19.1.1 Platform Content Overview

This section contains the APIs provided by NADIF to read and write NADIF persistence items. This persistence items allow customatization of NADIF features like the Modem Recovery feature. In order to know which persistence items can be access by NADIF client please refer to nadif_control.h.

3.19.1.2 Product Expectations

NADIF Client must include the header file nadif_control.h to access any API described in this section.

3.19.1.3 Extending Platform Functionality

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

3.19.2 Setup

3.19.2.1 Tools/Equipment Needed

Not Applicable

3.19.2.2 Setup Instructions

Not Applicable

3.19.2.3 Setup Troubleshooting

Not Applicable

3.19.3 Usage

3.19.3.1 Basic Instructions

Not Applicable

3.19.3.2 Detailed Instructions

3.19.3.2.1 Write Persistence items

API/Type Description	Doxygen Path
API to check write persistence items	Modules |-> Modem |-> NADIF |-> Control |-> **NADIF_control_write_persistence_item ()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> NADIF Common Types |-> ***NADIF_RESULT***

The purpose of API *NADIF_control_write_persistence_item()* is to allow to write the persistence items that storage NADIF persistence settings.

3.19.3.2.2 Read Persistence items

API/Type Description	Doxygen Path
API to check read persistence items	Modules |-> Modem |-> NADIF |-> Control |-> **NADIF_control_read_persistence_item()**
Notification data type of API	Modules |-> Modem |-> NADIF |-> Control |-> **NADIF_PERSISTENCE_READ_RESP**

The purpose of API *NADIF_control_read_persistence_item()* is to read the persistence items that store NADIF persistence settings.

3.19.3.3 Testing/Error Reporting

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

4 Known Issues

Not applicable

5 FAQs

Not applicable

6 Troubleshooting/Tips and Tricks

API to query Phone Number is not reporting the phone number

Problem: The notification of `NADIF_modem_get_phone_number()` is reporting a null string in `phone_number` element.

Possible Cause: MSISDN is not provisioned on the SIM by the carrier.

7 References

7.1 Abbreviations/Acronyms/Definitions

Not applicable

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary

TP_F101U-00009: Telematics Common Service User Guide

TP_F101U-00023: Common Library User Guide

TP_F101U-00024: Connection Manager User Guide

TP_F101U-00026: Persistence User Guide

TP_F101U-00047: Thermal Management User Guide

7.3 Website links

https://www.gsma.com/newsroom/all-documents/sgp-02-v3-2-remote-provisioning-architecture-for-embedded-uicc-technical-specification/

3GPP specifications: http://www.etsi.org

- 3GPP TS 24.008, release 12:
  https://www.etsi.org/deliver/etsi_ts/124000_124099/124008/12.15.00_60/ts_124008v121500p.pdf
- 3GPP TS 24.301, release 12:
  https://www.etsi.org/deliver/etsi_ts/124300_124399/124301/12.13.00_60/ts_124301v121300p.pdf
- 3GPP TS 27.005, release 12:
  https://www.etsi.org/deliver/etsi_ts/127000_127099/127005/12.00.00_60/ts_127005v120000p.pdf
- 3GPP TS 27.007, release 12:
  https://www.etsi.org/deliver/etsi_ts/127000_127099/127007/12.12.00_60/ts_127007v121200p.pdf

RTT specifications: https://www.fcc.gov/real-time-text

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal

8 Revision History

Version	Date	Description
01.00	Aug 16, 2019	Initial release
02.00	Nov 20, 2019	Add security group information
03.00	May 4, 2020	Added SMS Transparent PDU, MIPI switch
04.00	Jun 9, 2020	Added get NAD configuration, cell broadcast messages, SMS delivery reports, and get last reject causes
05.00	Aug 4, 2020	Replace tpSYS service notify API with tpSVCMON API, added more use cases for `NADIF_MODEM_RESET` case, and added support to read emergency numbers from SIM
06.00	Jan 06, 2021	Removed references to 3GPP2 specifications, changed description of automatic SPY mode timer operation, added Network Recovery Mechanism, added Power-up Operating Mode and updated the Thermal mitigation
07.00	Jan 15, 2021	Updated Modem Status with new API for modem reset counts
08.00	Apr 5, 2021	Added support for Multiple SIM subscriptions
09.00	Apr 16, 2021	Added support for Persistence configuration
10.00	Jul 19, 2021	Added multiple SIM subscription support Extended NADIF_voice_dial_custom_data_over_ims() API parameters Removed PDN reject causes Added SAR transmit power back-off
11.00	Sep 8, 2021	Added support for high RF capability configuration
12.00	Oct 1, 2021	Removal of product specific content (Data Services and MIPI Switch) Added support to Block ecall request from subscription1 on nadif
		== ConMod-specific content from here ==
13.00	Nov 26, 2021	Added support to ignoring SIM slot parameter and using binded subscription for SIM API
14.00	Apr 04, 2022	Added support to NADIF set, get Modem Configuration API and notification for Dormant status.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Internal